

Revisiting Differential-Linear Cryptanalysis via a Walsh-Transform Perspective

Jiahui He^{1,2}, Kai Hu^{2,3,4}, Zhongfeng Niu⁵, Bart Preneel¹, and Meiqin Wang^{2,3}(✉)

¹ COSIC, KU Leuven, Leuven, Belgium

`jhe@esat.kuleuven.be`, `bart.preneel@esat.kuleuven.be`

² School of Cyber Science and Technology, Shandong University, Qingdao, China

`kai.hu@sdu.edu.cn`, `mqwang@sdu.edu.cn`

³ State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, China

⁴ Suzhou Research Institute, Shandong University, Suzhou, 215123, China

⁵ School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore

`zhongfeng.niu@ntu.edu.sg`

Abstract. Standard differential-linear (SDL) cryptanalysis has matured rapidly, now backed by the SDL connectivity table (SDLCT) framework, multi-round middle-part extensions, and automated CP-based search models. The rotational differential-linear (RDL) variant lacks even an SDLCT counterpart, and the analogous combination of internal differentials with linear approximations, which we call internal differential-linear (IDL) cryptanalysis, has never been formulated.

We unify SDL, RDL, and IDL in a single Walsh-transform framework on the difference transition function (DTF). Existing SDL machinery, including recent multi-round middle-part search models, transfers to RDL and IDL, yielding efficient correlation computation and distinguisher search.

For ARX primitives, we derive a 2×2 matrix-product formula for the SDL and RDL correlations of a single modular addition. The formula exposes a rank-1 structure on the input-difference matrix that decouples the running product into independent factors, making a CP approximation with only linear inequalities and table lookups practical. This partially resolves the open problem of Niu et al. (CRYPTO 2022).

We apply the framework to *Xoodoo-p*, *Ascon-p*, *Alzette*, *SipHash*, and *SPECK64*. For *Xoodoo-p*, we obtain a 6-round RDL distinguisher and the first IDL distinguisher in the literature (both at 6 rounds). For *Ascon-p*, the first RDL distinguisher (6 rounds). For *Alzette*, an 8-round RDL distinguisher, doubling the previous 4-round best. For *SipHash*, the longest known distinguisher (5 rounds). For *SPECK64*, the longest known SDL distinguisher (14 rounds).

Keywords: Standard differential-linear, Rotational differential-linear, Internal differential-linear, ARX, Xoodoo, Ascon, SipHash, SPECK, Chaskey, Alzette

1 Introduction

The security of symmetric primitives is judged against known cryptanalytic techniques. Standard differential cryptanalysis [14] and linear cryptanalysis [41] are the two most established, with three decades of mature theory. Modern designs resist both in their basic form by avoiding long differential trails of high probability and long linear trails of high correlation, pushing research toward variants of the two.

Standard Differential-Linear Cryptanalysis. Langford and Hellman [33] introduced the standard differential-linear (SDL) attack in 1994⁶. Given a decomposition $E = E_L \circ E_U$, it chains a high-probability differential of E_U with a high-correlation linear approximation of E_L , breaking ciphers that resist either technique alone.

The initial SDL theory assumed that E_U and E_L are independent and that the output difference of E_U is uniform. Biham et al. [13] showed both can fail in practice and proposed an enhanced attack. In 2017, Blondeau et al. [16] gave a closed-form expression for the SDL correlation, removing the second assumption. Bar-On et al. [5] later removed the independence assumption via the standard differential-linear connectivity table (SDLCT) framework, which decomposes E into E_U , E_M , E_L with E_M a single S-box layer. Extending SDLCT to a multi-round E_M remained open. At CRYPTO 2024, Hadipour et al. [25] and Peng et al. [46] resolved this independently. Peng et al. reduced it to the propagation of truncated standard differentials; Hadipour et al. generalized the SDLCT framework. At CRYPTO 2025, Hu et al. [27] extended Beyne’s geometric approach [12] and recast SDL analysis in a mixed-basis framework. At EUROCRYPT 2026, Hu, Niu and Wang [26] approximated the (higher-order) SDL correlation round by round from the geometric perspective, complementing the SDLCT line’s three-part decomposition. SDL is now supported by multiple complementary frameworks and a range of automated CP/MILP/SAT search models built on top.

Rotational Differential-Linear Cryptanalysis. Khovratovich and Nikolic [28] introduced rotational cryptanalysis, which tracks rotational pairs $(x, x \lll t)$ through the cipher. It has been effective against ARX designs such as Skein [29], Chaskey [30], and ChaCha [6]. Ashur and Liu [2] generalized rotational pairs to data pairs (x, x') with rotational-xor (RX) difference $x \oplus (x' \lll t) = \Delta$, yielding RX cryptanalysis, which has been applied to many ARX and AND-RX designs [37,38,39]. We use *rotational difference* and *rotational differential cryptanalysis* as synonyms for RX difference and RX cryptanalysis throughout.

At EUROCRYPT 2021, Liu et al. [36,35] introduced the rotational differential-linear (RDL) attack, the SDL counterpart for rotational differentials, and showed that Morawiecki et al.’s rotational cryptanalysis [42] is a special case. Under independence of E_U and E_L , they gave a closed-form for the RDL correlation of

⁶ To distinguish from RDL and IDL analyses, we use the term ‘standard differential-linear’ instead of ‘differential-linear’ in this paper.

$E = E_L \circ E_U$, but their computation restricted the output linear mask to a unit vector. At CRYPTO 2022, Niu et al. [45] lifted this restriction for modular additions via a chain of 4×4 matrix multiplications, but how to turn this formula into a CP/MILP/SAT-friendly model remained open. Lv et al. [40] later reduced the matrices to 2×2 but as an MIQCP, where the quadratic constraints limit scalability. No effective CP/MILP/SAT model is known for the SDL/RDL correlation of a single modular addition.

RDL has not kept pace with SDL. No SDLCT-style framework exists for RDL, let alone a counterpart of [25,46] for a multi-round middle part. Hadipour et al. themselves leave the RDL extension as an open question [25].

Internal Differential-Linear Cryptanalysis. Internal differentials have been studied since Peyrin [47], generalized by Dinur et al. [21] for collision attacks on Keccak, and pushed further on SHA-3 by recent work [52,53,54]. The technique tracks how internal slices of a single plaintext diverge through the cipher. Yet despite the success of SDL and RDL, an internal differential-linear (IDL) attack has never been formulated. The corresponding hybrid technique is absent from the literature.

Our Contributions. The contrast above motivates this work. A unification of SDL, RDL, and IDL would let the mature SDL toolkit flow directly into RDL and IDL, where almost no analogous machinery exists. We achieve this via the Walsh transform. On the ARX side, we resolve a longstanding bottleneck on modeling a single modular addition for automated SDL/RDL distinguisher search.

Unifying SDL, RDL, and IDL. We interpret the three analyses as instances of a single Walsh-transform structure on what we call the difference transition function (DTF). Under this unification, the existing SDL theory, including Hadipour et al.’s and Peng et al.’s methods for multi-round E_M , transfers directly to RDL and IDL. Efficient computation of RDL and IDL correlations and distinguisher search for primitives with a multi-round middle part follow as immediate consequences.

Automated Search for ARX Primitives. We derive a 2×2 matrix-product formula for the SDL and RDL correlations of a single modular addition from the Walsh-transform perspective. The per-bit matrix factors as $C \cdot B$, and the input-difference matrix C degenerates to rank 1 whenever the two input differences differ at that bit. This rank-1 property disconnects the running matrix product into independent factors, making a CP approximation with only linear inequalities and table lookups practical. The independently derived 2×2 formulation of Lv et al. [40] does not surface this structure and yields a less scalable MIQCP model. Combined with standard models for (standard, rotational) differential and linear propagation in ARX primitives, this gives a CP search for SDL and RDL distinguishers. The heuristic is simple. A distinguisher containing one SDL/RDL trail of high absolute correlation tends to have high overall absolute correlation.

We apply the techniques to S-box-based permutations (Xoodoo [20], Ascon [22]) and ARX primitives (Alzette [8], SPECK [7], SipHash [3], Chaskey [43]), yielding new SDL/RDL/IDL distinguishers:

New RDL and IDL distinguishers (Table 1).

- For Xoodoo- p , a 6-round RDL distinguisher of correlation $2^{-44.10}$ (prior best 4-round trivial), and a 6-round IDL distinguisher of correlation 2^{-45} . The IDL distinguisher is the first in the literature.
- For Ascon- p , the first RDL distinguisher, covering 6 rounds at correlation 2^{-21} .
- For Alzette, an 8-round RDL distinguisher of correlation $2^{-24.30}$, doubling the previous 4-round best.

New SDL distinguishers (Table 2).

- For SipHash, a 5-round SDL distinguisher of correlation $2^{-20.15}$, the longest known (prior 4 rounds at $2^{-6.03}$).
- For SPECK64, a single-key 14-round SDL distinguisher of correlation $2^{-31.33}$, the longest known (prior 13 rounds at $2^{-26.59}$) in terms of SDL analysis.

Comparison with distinguishers of other types. For Xoodoo- p and Ascon- p , integral distinguishers covering more than 6 rounds exist owing to the low algebraic degree of their S-boxes, so our contribution there is a new type of distinguisher rather than a larger number of rounds. For Alzette, the 8-round RDL distinguisher doubles the number of rounds of the previous RDL distinguisher [45]; in Appendix K.3 we additionally give a 13-round SDL distinguisher. For SPECK64, the best known distinguisher is the 15-round standard differential of [49], so our 14-round result is the best one within SDL analysis. For SipHash, to the best of our knowledge, the 5-round distinguisher covers the most rounds among the known distinguishers of any type for its finalization. For Chaskey, differential-linear distinguishers [34,10,24] remain the best known ones, and ours slightly improves the correlation.

All source code for modeling and computation is publicly available at <https://github.com/viocently/Zodkej1Slz3Uq>.

Outline. Section 2 introduces notations and preliminaries for SDL, RDL and IDL analyses. Section 3 presents the unified Walsh-transform framework. Section 4 applies it to S-box-based primitives. Section 5 develops the 2×2 matrix-product machinery for modular addition. Section 6 applies it to ARX primitives. Section 7 compares our theory with prior work. Section 8 concludes this work.

2 Background

In this paper, we use italic Latin or Greek letters to denote binary vectors, whose elements may represent either binary values or binary variables, depending on the context. In matrix multiplication, binary vectors are treated as column vectors

Table 1: New RDL and IDL distinguishers. Our results in shaded rows. Note that there exist integral distinguishers beyond 6 rounds for Xoodoo- p and Ascon- p due to low (quadratic) algebraic degree of their S-boxes.

Primitive	Type	#Rounds/Total	Absolute Cor.	Ref.
Xoodoo- p	RDL	4/12	1	[36,35]
	RDL	6/12	$2^{-44.10}$	Section 4.1
	IDL	6/12	2^{-45}	Section 4.1
Ascon- p	RDL	6/12	2^{-21}	Section 4.2
Alzette	RDL	4/4*	$2^{-3.14}$	[45]
	RDL	8/8*	$2^{-24.30}$	Section 6.1

* 4-round Alzette is typically treated as a single S-box, so rounds for Alzette are counted in multiples of 4.

Table 2: New SDL distinguishers. Our results in shaded rows; the prior comparison row is the best previously known result. Note that there exists a 15-round standard differential distinguisher for SPECK64 in [49].

Primitive	#Rounds/Total	Absolute Cor.	Ref.
SipHash	3/- [‡]	$2^{-0.78}$	[45]
	4/- [‡]	$2^{-6.03}$	[45]
	5/-[‡]	$2^{-20.15}$	Section 6.3
SPECK64	11/26	$2^{-19.66}$	[40]
	12/26	$2^{-23.72}$	[40]
	13/26	$2^{-28.15}$	[18]
	13/26	$2^{-26.59}$	[24]
	14/26	$2^{-31.33}$	Section 6.2
SPECK48	11/22	$2^{-17.55}$	[40]
	11/22	$2^{-17.27}$	[18]
	12/22	$2^{-23.40}$	[24]
	12/22	$2^{-22.43}$	Section 6.2
Chaskey	7/8	$2^{-36.11}$	[34,10]
	7/8	$2^{-33.65}$	[24]
	7/8	$2^{-33.48}$	Section 6.4

[‡] Number of rounds for SipHash’s finalization, which is variable [3].

by default. We use the standard notations $\oplus, \wedge, \vee, \neg$ for bit-wise operations, $M^\top$ for transpose, $wt(x)$ for Hamming weight, $\langle x, y \rangle = x^\top y = \bigoplus_i x[i] \cdot y[i]$ for the dot product, where $x[i]$ denotes the i -th bit of x with $x[0]$ being the least significant bit (LSB), and $0^n, 1^n$ for the all-zero and all-one vectors of length n (length omitted when clear from context). The remaining notations are summarized in Table 3.

Table 3: Summary of Notations

Notation	Description
$x y$	Concatenation of binary vectors x and y , with x at the lower bit positions and y at the higher bit positions
$x[i]$	The i -th bit of x (indexed modulo n); $x[0]$ is the LSB
$x[i]_m$	The i -th m -bit slice of x , which is the sub-vector of x from index mi to $mi + m - 1$
$[x]^t$	The most significant t bits of $x \in \mathbb{F}_2^n$, i.e., $(x[n-t], \dots, x[n-1])$; if $t = 0$, it is an empty vector
$[x]_t$	The least significant t bits of $x \in \mathbb{F}_2^n$, i.e., $(x[0], \dots, x[t-1])$; if $t = 0$, it is an empty vector
$x \lll t$ ($x \ggg t$)	Left (resp. right) circular rotation by t bits, where bit $x[i]$ is moved to position $(i+t) \bmod n$ (resp. $(i-t) \bmod n$)
$\boxplus_n^{(b)}(x, y)$	The n -bit modular addition function of $x, y \in \mathbb{F}_2^n$, defined as $(x + y + b) \bmod 2^n$, where $b \in \mathbb{F}_2$ is a constant
$x \boxplus_n y$	shorthand for $\boxplus_n^{(0)}(x, y)$

Standard Differential, Rotational Differential and Internal Differential Cryptanalyses. For two states $x, x' \in \mathbb{F}_2^n$, the *standard difference* is $x \oplus x'$ and, given a rotation offset t , the *rotational difference* is $x \oplus (x' \lll t)$. Both summarize the relation between two states by a single element of $\mathbb{F}_2^n$.

The internal difference instead describes how a single state x deviates from being slice-wise uniform, and does not have a single-state form by default. Given a slice size m that divides n , internal differential cryptanalysis tracks the propagation of the set $\{v \oplus u \mid u[0]_m = u[1]_m = \dots = u[\frac{n}{m}-1]_m\}$ parameterized by a single state $v \in \mathbb{F}_2^n$. To make this parameterization unique, we impose $v[\frac{n}{m}-1]_m = 0^m$. The representative of x is then $(x[0]_m \oplus x[\frac{n}{m}-1]_m) \parallel \dots \parallel (x[\frac{n}{m}-2]_m \oplus x[\frac{n}{m}-1]_m) \parallel 0^m$, whose first $\frac{n}{m}-1$ slices form the *internal difference* of x . This map is linear, denoted $\mathcal{L}x$ with $\mathcal{L}$ an $(n-m) \times n$ matrix. The transpose satisfies $\mathcal{L}^\top y = y[0]_m \parallel \dots \parallel y[\frac{n}{m}-2]_m \parallel \bigoplus_{i=0}^{\frac{n}{m}-2} y[i]_m$ for $y \in \mathbb{F}_2^{n-m}$.

We uniformly use $\Delta_i \xrightarrow{p} \Delta_o$ to represent the standard, rotational or internal differential transition with probability p through a cryptographic primitive $E(x), x \in \mathbb{F}_2^n$, depending on the context:

- Standard differential cryptanalysis: $p = \Pr[y \oplus y' = \Delta_o \mid x \oplus x' = \Delta_i]$, where $y = E(x), y' = E(x')$.

- Rotational differential cryptanalysis:

$$p = \Pr[y \oplus (y' \lll t) = \Delta_o \mid x \oplus (x' \lll t) = \Delta_i],$$

where $y = E(x)$, $y' = E(x')$. $x \oplus (x' \lll t)$ and $y \oplus (y' \lll t)$ are called the *rotational difference*. When the rotational offset t is clear from context or can be omitted, we write $\text{rot}(x)$ and $\text{rot}^{-1}(x)$ for $x \lll t$ and $x \ggg t$, respectively.

- Internal differential cryptanalysis:

$$p = \Pr[\mathcal{L}y = \Delta_o \mid \mathcal{L}x = \Delta_i],$$

where $y = E(x)$.

For brevity, we may use the compact notation $\mathcal{P}_E(\Delta_i, \Delta_o)$ to represent the standard, rotational, or internal differential probability.

Linear Cryptanalysis and Walsh Transform. For a cryptographic primitive $E : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^{n'}$, we denote a linear approximation by $\lambda_i \xrightarrow{q} \lambda_o$, which represents that the relation $\langle \lambda_i, x \rangle \oplus \langle \lambda_o, E(x) \rangle = 0$ holds with correlation $q = 2 \cdot \Pr[\langle \lambda_i, x \rangle \oplus \langle \lambda_o, E(x) \rangle = 0] - 1$. Here, $\lambda_i \in \mathbb{F}_2^n$ and $\lambda_o \in \mathbb{F}_2^{n'}$ are referred to as the *input and output linear masks*, respectively. We also use the compact notation $\mathcal{C}_E(\lambda_i, \lambda_o)$ to represent this correlation.

The linear correlation can also be understood from the perspective of the Walsh transform [17, 48]. Real-valued functions $f : \mathbb{F}_2^n \rightarrow \mathbb{R}$ form a 2^n -dimensional inner-product space under $\langle f, g \rangle = \frac{1}{2^n} \sum_{x \in \mathbb{F}_2^n} f(x)g(x)$, with the parity functions $(-1)^{\langle u, x \rangle}$, $u \in \mathbb{F}_2^n$ as an orthonormal basis. Any f thus admits the expansion

$$f(x) = \sum_{u \in \mathbb{F}_2^n} \hat{f}(u) (-1)^{\langle u, x \rangle}, \quad \hat{f}(u) = \frac{1}{2^n} \sum_{x \in \mathbb{F}_2^n} f(x) (-1)^{\langle u, x \rangle},$$

where $\hat{f}(u)$ is the *Walsh coefficient* of f at u , and the collection $\{\hat{f}(u)\}_{u \in \mathbb{F}_2^n}$ is the *Walsh spectrum* of f . The linear correlation $\mathcal{C}_E(\lambda_i, \lambda_o)$ is the Walsh coefficient of $(-1)^{\langle \lambda_o, E(x) \rangle}$ at λ_i , leading to the *correlation matrix* framework [19]. For an S-box S , the *standard difference distribution table* (SDDT) and the *linear approximation table* (LAT) tabulate $\mathcal{P}_S(\Delta, \Delta')$ for all standard differences (Δ, Δ') and $\mathcal{C}_S(\lambda, \lambda')$ for all masks (λ, λ') , respectively.

Standard Differential-Linear (SDL) Cryptanalysis. Let E be a cryptographic primitive, with Δ_i and λ_o denoting the input standard difference and output linear mask, respectively. SDL analysis distinguishes E from a random function using $\mathcal{O}(r^{-2})$ chosen inputs, provided that the correlation $r = 2 \cdot \Pr[\langle \lambda_o, E(x) \rangle \oplus \langle \lambda_o, E(x \oplus \Delta_i) \rangle = 0] - 1$ is sufficiently large. We denote this SDL correlation as $\mathcal{R}_E^{\text{SDL}}(\Delta_i, \lambda_o)$.

Consider a decomposition of E into two parts, E_U and E_L , such that $E = E_L \circ E_U$. Let $\Delta_i \xrightarrow{p} \Delta_m$ be a standard differential for E_U and $\lambda_m \xrightarrow{q} \lambda_o$ be a linear approximation for E_L . Under the assumptions that E_U and E_L are independent, and that $\langle \lambda_m, E_U(x) \rangle \oplus \langle \lambda_m, E_U(x \oplus \Delta_i) \rangle = 0$ holds in half the cases where

$E_U(x) \oplus E_U(x \oplus \Delta_i) \neq \Delta_m$, the correlation $\mathcal{R}_E^{\text{SDL}}(\Delta_i, \lambda_o)$ is traditionally estimated using Matsui’s Piling-up Lemma [41] as $\mathcal{R}_E^{\text{SDL}}(\Delta_i, \lambda_o) = (-1)^{\langle \lambda_m, \Delta_m \rangle} pq^2$. Without the second assumption, Blondeau et al. [16] provided a closed formula for the SDL correlation: $\mathcal{R}_E^{\text{SDL}}(\Delta_i, \lambda_o) = \sum_{\lambda_m} \mathcal{R}_{E_U}^{\text{SDL}}(\Delta_i, \lambda_m) \cdot \mathcal{C}_{E_L}^2(\lambda_m, \lambda_o)$.

In the standard differential-linear connectivity table (SDLCT) framework (see Appendix A for the definition of SDLCT) proposed by Bar-On et al. [5], E is decomposed further into three parts: $E = E_L \circ E_M \circ E_U$. Assuming that E_U , E_M , and E_L are independent, the correlation $\mathcal{R}_E^{\text{SDL}}(\Delta_i, \lambda_o)$ can be expressed as:

$$\mathcal{R}_E^{\text{SDL}}(\Delta_i, \lambda_o) = \sum_{\Delta_m, \lambda_m} \mathcal{P}_{E_U}(\Delta_i, \Delta_m) \cdot \mathcal{R}_{E_M}^{\text{SDL}}(\Delta_m, \lambda_m) \cdot \mathcal{C}_{E_L}^2(\lambda_m, \lambda_o). \quad (1)$$

When applying the above formula in practice, Bar-On et al. only consider the case where E_M consists of one S-box layer. The problem of computing SDL correlation of a multi-round E_M was solved independently by Hadipour et al. and Peng et al. [25, 46] at CRYPTO 2024. Their work can be viewed as an extension of the SDLCT framework, and we defer the details to Appendix D. In particular, Hadipour et al. developed a CP model (see Algorithm 4) for SDL distinguisher search on S-box-based primitives.

Two further frameworks complement the SDLCT line. At CRYPTO 2025, Hu et al. [27] recast SDL analysis in Beyne’s mixed-basis framework [12]. At EUROCRYPT 2026, Hu, Niu and Wang [26] approximated the SDL correlation round by round from the geometric perspective. Our work builds on the SDLCT line.

Rotational Differential-Linear (RDL) Cryptanalysis. Given the input rotational difference Δ_i and the output linear mask λ_o of E , RDL analysis [36, 35] can distinguish E from a random function using $\mathcal{O}(r^{-2})$ chosen inputs, if the correlation $r = 2 \cdot \Pr[\langle \lambda_o, E(x) \rangle \oplus \langle \lambda_o, \text{rot}(E(\text{rot}^{-1}(x \oplus \Delta_i))) \rangle = 0] - 1$ is sufficiently high. We denote this RDL correlation as $\mathcal{R}_E^{\text{RDL}}(\Delta_i, \lambda_o)$.

Dividing E into two consecutive parts E_U and E_L , let $\Delta_i \xrightarrow{p} \Delta_m$ be a rotational differential for E_U , and $\lambda_m \xrightarrow{q} \lambda_o, \text{rot}^{-1}(\lambda_m) \xrightarrow{q'} \text{rot}^{-1}(\lambda_o)$ be two linear approximations for E_L . Assuming that E_U and E_L are independent, and that $\langle \lambda_m, E_U(x) \rangle \oplus \langle \lambda_m, \text{rot}(E_U(\text{rot}^{-1}(x \oplus \Delta_i))) \rangle = 0$ holds in half of the cases when $E_U(x) \oplus \text{rot}(E_U(\text{rot}^{-1}(x \oplus \Delta_i))) \neq \Delta_m$, we can estimate $\mathcal{R}_E^{\text{RDL}}(\Delta_i, \lambda_o)$ as $(-1)^{\langle \lambda_m, \Delta_m \rangle} pqq'$ using Matsui’s Piling-up lemma [41], analogous to SDL cryptanalysis. Without the second assumption, a closed formula for the overall RDL correlation is given by Liu et al. [35] as

$$\mathcal{R}_E^{\text{RDL}}(\Delta_i, \lambda_o) = \sum_{\lambda_m} \mathcal{R}_{E_U}^{\text{RDL}}(\Delta_i, \lambda_m) \cdot \mathcal{C}_{E_L}(\lambda_m, \lambda_o) \cdot \mathcal{C}_{E_L}(\text{rot}^{-1}(\lambda_m), \text{rot}^{-1}(\lambda_o)). \quad (2)$$

RDL lacks a three-part SDLCT-style decomposition; we close this gap in Section 3.

Internal Differential-Linear (IDL) Cryptanalysis. IDL cryptanalysis has not been formulated in the literature. We give the first definition in Section 3, alongside the unified Walsh-transform treatment of SDL, RDL and IDL.

3 A Unified Walsh-Transform Framework for SDL, RDL and IDL Cryptanalysis

This section presents one of our main theoretical contributions: a unified framework, grounded in the Walsh transform, that encompasses SDL, RDL and IDL analyses under a single umbrella. We begin by formally defining IDL cryptanalysis.

Definition 1 (IDL cryptanalysis). *Let $E : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be a cryptographic primitive where the state is partitioned into slices of size m . Given an input internal difference $\Delta_i \in \mathbb{F}_2^{n-m}$ and a linear mask $\lambda_o \in \mathbb{F}_2^{n-m}$ on the output internal difference, the IDL correlation of E , denoted by $\mathcal{R}_E^{\text{IDL}}(\Delta_i, \lambda_o)$, is defined as*

$$\mathcal{R}_E^{\text{IDL}}(\Delta_i, \lambda_o) = 2 \cdot \Pr[\langle \lambda_o, \mathcal{L}y \rangle = 0 \mid \mathcal{L}x = \Delta_i] - 1,$$

where $y = E(x)$, and the probability is taken over all $x[\frac{n}{m} - 1]_m \in \mathbb{F}_2^m$.

IDL analysis targets a different kind of structure than SDL and RDL: it exploits the symmetry of a primitive under a shift of its slices, which round constants can only perturb but never fully destroy, and can thus be regarded as a new security metric for permutations. It is also the natural extension of internal differential cryptanalysis, which underlies collision and preimage attacks on SHA-3 [21, 52, 53, 54]; since those attacks live on keyless permutations, IDL distinguishers are a promising addition to that toolbox, although turning them into concrete attacks requires further investigation.

Throughout this section, we use the unified notation $\mathcal{R}_E(\Delta, \lambda)$ to denote any of $\mathcal{R}_E^{\text{SDL}}(\Delta, \lambda)$, $\mathcal{R}_E^{\text{RDL}}(\Delta, \lambda)$, or $\mathcal{R}_E^{\text{IDL}}(\Delta, \lambda)$ when the specific variant is clear from the context or when the discussion applies uniformly to all three.

3.1 The Difference Transition Function

The framework rests on a single auxiliary function. For an input x and a difference Δ , this function returns the corresponding output difference of F , with the type of “difference” (standard, rotational, or internal) fixed by the variant. Differential propagation through F is then an ordinary vectorial Boolean function of (x, Δ) , and the Walsh transform applies directly. We call this auxiliary function the *difference transition function* (DTF). For standard differences, this function is the classical derivative of F in the direction Δ [31]; what is new here is to treat Δ as a second input and to work with the Walsh transform of the resulting two-input function.

Definition 2 (DTF). *For a vectorial Boolean function F with n input variables, its DTF ∂_F is defined according to the chosen difference type:*

- *RDL analysis (with rotational offset t ; SDL is a special case $t = 0$): For $x \in \mathbb{F}_2^n$ and $\Delta \in \mathbb{F}_2^n$, $\partial_F(x, \Delta) = F(x) \oplus \text{rot}(F(\text{rot}^{-1}(x \oplus \Delta)))$.*
- *IDL analysis (with slice size m): For $x \in \mathbb{F}_2^m$ and $\Delta \in \mathbb{F}_2^{n-m}$, $\partial_F(x, \Delta) = \mathcal{L}F(y)$, where $y = (\Delta[0]_m \oplus x) \parallel \dots \parallel (\Delta[\frac{n}{m} - 2]_m \oplus x) \parallel x$.*

The first input of ∂_F corresponds to the free slice (in IDL analysis) or the full state (in SDL/RDL analysis). The second input and the output of ∂_F correspond to the input and output differences of F , respectively.

Under this abstraction, the differential probability and the differential-linear correlation both have clean and uniform expressions:

$$\begin{aligned}\mathcal{P}_F(\Delta_i, \Delta_o) &= \Pr[\partial_F(x, \Delta_i) = \Delta_o], \\ \mathcal{R}_F(\Delta_i, \lambda_o) &= 2 \cdot \Pr[\langle \lambda_o, \partial_F(x, \Delta_i) \rangle = 0] - 1,\end{aligned}$$

where x is drawn uniformly at random from its domain.

These two quantities are related by the following fundamental lemma, which asserts that for a fixed input difference, the differential probabilities and the differential-linear correlations are Walsh-spectrum counterparts of each other. In its SDL form, the lemma was proved in [25, Proposition 1]; the DTF abstraction makes it valid for all three difference types.

Lemma 1. *For any λ_o , we have $\mathcal{R}_F(\Delta_i, \lambda_o) = \sum_{\alpha} \mathcal{P}_F(\Delta_i, \alpha) (-1)^{\langle \lambda_o, \alpha \rangle}$.*

Proof. By the relation of DTF to the differential probability and the differential-linear correlation, we have

$$\begin{aligned}\mathcal{R}_F(\Delta_i, \lambda_o) &= \Pr[\langle \lambda_o, \partial_F(x, \Delta_i) \rangle = 0] - \Pr[\langle \lambda_o, \partial_F(x, \Delta_i) \rangle = 1] \\ &= \sum_{\alpha} \Pr[\partial_F(x, \Delta_i) = \alpha] \cdot (\Pr[\langle \lambda_o, \alpha \rangle = 0] - \Pr[\langle \lambda_o, \alpha \rangle = 1]) \\ &= \sum_{\alpha} \mathcal{P}_F(\Delta_i, \alpha) (-1)^{\langle \lambda_o, \alpha \rangle},\end{aligned}$$

as asserted. $\square$

The lemma above reveals that for any fixed Δ_i , the set of all $\mathcal{P}_F(\Delta_i, \alpha)$ is simply the Walsh spectrum of $\mathcal{R}_F(\Delta_i, \lambda_o)$ with respect to the basis functions $(-1)^{\langle \lambda_o, \alpha \rangle}$. As we will see, this relation serves as a key technical tool in the derivations that follow.

3.2 Decomposition of Correlation: Extending the SDLCT Framework

We now show how to decompose the differential-linear correlation of a primitive $E = E_L \circ E_U$ (and later $E = E_L \circ E_M \circ E_U$) into contributions from each part. The decomposition relies on the following independence assumption, which is standard in SDL analysis.

Definition 3 (Differential independence). E_U and E_L are differentially independent if for all Δ_i and Δ_o , the following relation holds:

$$\mathcal{P}_E(\Delta_i, \Delta_o) = \sum_{\alpha} \mathcal{P}_{E_U}(\Delta_i, \alpha) \mathcal{P}_{E_L}(\alpha, \Delta_o),$$

where the difference type can be standard, rotational or internal, depending on the context.

This notion was originally formalized for SDL analysis by Blondeau et al. [16]. As proved in [4], rounds of a Markov cipher [32] are differentially independent with respect to standard differences; this does not carry over to rotational or internal differences, since XORing a round key k shifts a rotational difference by $k \oplus \text{rot}(k)$ and an internal difference by $\mathcal{L}k$ (see Remarks 2 and 3). While it has not been explicitly formalized for RDL and IDL analysis, it has been implicitly relied upon throughout the existing literature.

Two-Part Decomposition. Under the differential independence between E_U and E_L , the differential-linear correlation of E can be decomposed by traversing the intermediate differences or the masks on these differences.

Lemma 2. *Assuming that E_U and E_L are differentially independent, we have*

$$\mathcal{R}_E(\Delta_i, \lambda_o) = \sum_{\alpha} \mathcal{P}_{E_U}(\Delta_i, \alpha) \cdot \mathcal{R}_{E_L}(\alpha, \lambda_o).$$

Proof. By Lemma 1 and the assumption, we have

$$\begin{aligned} \mathcal{R}_E(\Delta_i, \lambda_o) &= \sum_{\beta} \mathcal{P}_E(\Delta_i, \beta) (-1)^{\langle \lambda_o, \beta \rangle} \\ &= \sum_{\alpha} \mathcal{P}_{E_U}(\Delta_i, \alpha) \cdot \sum_{\beta} \mathcal{P}_{E_L}(\alpha, \beta) (-1)^{\langle \lambda_o, \beta \rangle}. \end{aligned}$$

Since for each α , $\sum_{\beta} \mathcal{P}_{E_L}(\alpha, \beta) (-1)^{\langle \lambda_o, \beta \rangle}$ is exactly $\mathcal{R}_{E_L}(\alpha, \lambda_o)$, we obtain the lemma. $\square$

Lemma 2 sums over intermediate differences. Its Walsh-spectrum counterpart, stated next, sums over masks on these differences and is the form that feeds the three-part decomposition (Proposition 1). In the RDL case, it is exactly Formula (2) of Liu et al. [35].

Lemma 3. *Assuming that E_U and E_L are differentially independent, we have*

$$\mathcal{R}_E(\Delta_i, \lambda_o) = \sum_{\lambda_m} \mathcal{R}_{E_U}(\Delta_i, \lambda_m) \cdot \mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o),$$

where $\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o)$ is the Walsh coefficient of $(-1)^{\langle \lambda_o, \partial_{E_L} \rangle}$ at $(0, \lambda_m)$.

Proof. From the proof of Lemma 2, we have $\mathcal{R}_E(\Delta_i, \lambda_o) = \sum_{\alpha} \mathcal{P}_{E_U}(\Delta_i, \alpha) \cdot \sum_{\beta} \mathcal{P}_{E_L}(\alpha, \beta) (-1)^{\langle \lambda_o, \beta \rangle}$. Inverting the formula in Lemma 1, we have

$$\mathcal{P}_{E_U}(\Delta_i, \alpha) = 2^{-N} \sum_{\lambda_m} \mathcal{R}_{E_U}(\Delta_i, \lambda_m) (-1)^{\langle \lambda_m, \alpha \rangle},$$

where N is the bit-length of the difference, i.e., $N = n$ for SDL/RDL analysis and $N = n - m$ for IDL analysis (with slice size m). Substituting this into the first equation yields:

$$\mathcal{R}_E(\Delta_i, \lambda_o) = 2^{-N} \sum_{\lambda_m} \mathcal{R}_{E_U}(\Delta_i, \lambda_m) \cdot \sum_{\alpha, \beta} \mathcal{P}_{E_L}(\alpha, \beta) (-1)^{\langle \lambda_m, \alpha \rangle \oplus \langle \lambda_o, \beta \rangle}. \quad (3)$$

By the definition of Walsh Transform, with d denoting the bit-length of the first input of ∂_{E_L} ($d = n$ for SDL/RDL analysis and $d = m$ for IDL analysis),

$$\begin{aligned}\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o) &= 2^{-N-d} \sum_x \sum_{\alpha} (-1)^{\langle \partial_{E_L}(x, \alpha), \lambda_o \rangle \oplus \langle \lambda_m, \alpha \rangle} \\ &= 2^{-N-d} \sum_{\alpha} \sum_{\beta} \sum_{x: \partial_{E_L}(x, \alpha) = \beta} (-1)^{\langle \lambda_m, \alpha \rangle \oplus \langle \lambda_o, \beta \rangle}, \\ &= 2^{-N} \sum_{\alpha, \beta} \mathcal{P}_{E_L}(\alpha, \beta) (-1)^{\langle \lambda_m, \alpha \rangle \oplus \langle \lambda_o, \beta \rangle}\end{aligned}$$

where the last step uses that the inner sum over x has exactly $2^d \mathcal{P}_{E_L}(\alpha, \beta)$ terms. Substituting the above equation into Eqn. (3) completes the proof of the lemma. $\square$

Setting E_U as the identity function in the lemma, we immediately obtain the following corollary, whose RDL form is equivalent to [35, Theorem 2]:

Corollary 1. $\mathcal{R}_E(\Delta_i, \lambda_o) = \sum_{\lambda_m} \mathcal{C}_{\partial_E}((0, \lambda_m), \lambda_o) (-1)^{\langle \lambda_m, \Delta_i \rangle}.$

Remark 1. Let us emphasize the role of masks in $\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o)$. Here, λ_m appears as a mask on the second input of ∂_{E_L} , which is the input difference of E_L . Likewise, λ_o is a mask on the output of ∂_{E_L} , which is the output difference of E_L . These are masks on differences, not on state bits. In the special case of SDL analysis, we will see shortly (Proposition 2 with $t = 0$) that these masks on differences coincide with linear masks on the state, but this coincidence does not hold in general for RDL and IDL analyses.

Three-part Decomposition. Decomposing E into three parts E_U, E_M and E_L , we obtain:

Proposition 1 (Unified decomposition of SDL/RDL/IDL correlation). *Under the assumption that E_U , E_M , and E_L are differentially independent of each other, we have*

$$\mathcal{R}_E(\Delta_i, \lambda_o) = \sum_{\Delta_m, \lambda_m} \mathcal{P}_{E_U}(\Delta_i, \Delta_m) \cdot \mathcal{R}_{E_M}(\Delta_m, \lambda_m) \cdot \mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o). \quad (4)$$

Proof. Applying Lemma 2 on E_U and $E_L \circ E_M$ yields

$$\mathcal{R}_E(\Delta_i, \lambda_o) = \sum_{\Delta_m} \mathcal{P}_{E_U}(\Delta_i, \Delta_m) \cdot \mathcal{R}_{E_L \circ E_M}(\Delta_m, \lambda_o).$$

For each $\mathcal{R}_{E_L \circ E_M}(\Delta_m, \lambda_o)$ appearing in the above equation, applying Lemma 3 yields

$$\mathcal{R}_{E_L \circ E_M}(\Delta_m, \lambda_o) = \sum_{\lambda_m} \mathcal{R}_{E_M}(\Delta_m, \lambda_m) \cdot \mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o),$$

thus completing the proof. $\square$

Remark 2 (Key models and the scope of Definition 3). Proposition 1 is conditional on Definition 3 alone and holds in any key model in which the three parts are differentially independent; what differs between key models is the support for this assumption. (i) In the averaged-key model with independent round keys, Definition 3 is provable for standard differences (Markov ciphers [32,4]), but not for rotational or internal differences, as explained above. (ii) For real key schedules, even SDL analysis additionally relies on a hypothesis of stochastic-equivalence type [32]. (iii) In the fixed-key or fixed-constant model there is no averaging, and Definition 3 is a working assumption, as in [16,35]; all our targets except SPECK are unkeyed permutations with fixed constants, which is why we verify our results experimentally (Appendix L). The related-key setting carries through by treating the key difference as part of the input to ∂_E ; see Appendix I for details.

Remark 3 (Constant and key dependence of rotational and internal differences). Rotational and internal differences are sensitive to constants and keys [28,2]: every XOR with a constant c shifts a rotational difference by $c \oplus \text{rot}(c)$ and an internal difference by $\mathcal{L}c$. Our models of E_U take this into account explicitly, so the reported probabilities are specific to the actual constants. If the RDL or IDL correlation in the averaged-key model is required, Corollary 1 gives an assumption-free exact expression at a fixed key, whose average over the keys is well defined and imposes no related-key or weak-key condition.

Remark 4 (Connection to geometric approach). Our DTF/Walsh perspective extends the SDL machinery to RDL and IDL analyses, at the cost of requiring a differential-independence assumption. The geometric approach of [27,12], in contrast, avoids such independence assumptions altogether, but has not yet been developed for RDL or IDL analyses.

The search and evaluation methods for (standard, rotational, internal) differentials have been well-established in existing literature (e.g., [50,37,52]). Thus, the primary challenge in applying Proposition 1 in practice lies in computing the values of $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$ and $\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o)$ for specific Δ_m and λ_m . While the computation of $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$ will be addressed in Section 3.4, the third factor $\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o)$ remains an abstract quantity defined on the DTF, and it is not immediately clear how to evaluate it using existing tools. In the next subsection, we show that this quantity can be expressed entirely in terms of the linear correlations of E_L , thereby completing the unified framework and making it fully operational without any new analysis machinery.

3.3 Reducing Masks on Differences to Linear Masks on the State

The central technical insight of this subsection is that the Walsh coefficient $\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o)$ for the DTF, which characterizes how masks on differences propagate through ∂_{E_L} , can be expressed entirely in terms of linear correlations of E_L . That is to say, to apply our three-part decomposition in practice, one does

not need any new analysis machinery: the well-established methods for finding and evaluating linear trails suffice.

The following proposition generalizes [35, Corollary 2], which corresponds to the case $\alpha = \beta$.

Proposition 2 (Walsh coefficients of ∂_F in RDL analysis). *For a vectorial Boolean function F , the Walsh coefficient $\mathcal{C}_{\partial_F}((\alpha \oplus \beta, \beta), \lambda)$ of ∂_F in RDL analysis for any α, β and λ is given by:*

$$\mathcal{C}_{\partial_F}((\alpha \oplus \beta, \beta), \lambda) = \mathcal{C}_F(\alpha, \lambda) \cdot \mathcal{C}_F(\text{rot}^{-1}(\beta), \text{rot}^{-1}(\lambda)).$$

Proposition 3 (Walsh coefficients of ∂_F in IDL analysis). *For a vectorial Boolean function $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^{n'}$, for any $\alpha \in \mathbb{F}_2^n$ and $\lambda \in \mathbb{F}_2^{n'-m}$, we have the following relation in IDL analysis with a slice size m :*

$$\mathcal{C}_F(\alpha, \mathcal{L}^\top \lambda) = \mathcal{C}_{\partial_F} \left(\left(\sum_{i=0}^{\frac{n}{m}-1} \alpha[i]_m, \alpha[0]_m \parallel \cdots \parallel \alpha[\frac{n}{m}-2]_m \right), \lambda \right).$$

Due to page limits, the proofs of the above propositions are provided in Appendix E.

Instantiating the Decomposition Formula. Having established the general reduction in Propositions 2 and 3, we now instantiate $\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o)$ for each difference type, revealing how the unified formula of Proposition 1 specializes to recover known results and yield new ones.

Setting $t = 0$ in Proposition 2 gives $\mathcal{C}_{\partial_F}((\alpha \oplus \beta, \beta), \lambda) = \mathcal{C}_F(\alpha, \lambda) \cdot \mathcal{C}_F(\beta, \lambda)$. Setting the first input mask to 0 (i.e., $\alpha = \beta$) yields $\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o) = \mathcal{C}_{E_L}^2(\lambda_m, \lambda_o)$. This is the crucial simplification in SDL analysis: the masks λ_m, λ_o on the difference map to the same masks λ_m, λ_o as linear masks on the state of E_L . In other words, in SDL analysis, the distinction between ‘mask on the difference’ and ‘mask on the state’ is invisible. Substituting the equation into Proposition 1 recovers the well-known SDLCT formula (1).

In RDL analysis with a non-zero rotational offset t , Proposition 2 with $\alpha = \beta$ gives the decomposition of $\mathcal{R}_E(\Delta_i, \lambda_o)$ as

$$\sum_{\Delta_m, \lambda_m} \mathcal{P}_{E_U}(\Delta_i, \Delta_m) \cdot \mathcal{R}_{E_M}(\Delta_m, \lambda_m) \cdot \mathcal{C}_{E_L}(\lambda_m, \lambda_o) \cdot \mathcal{C}_{E_L}(\text{rot}^{-1}(\lambda_m), \text{rot}^{-1}(\lambda_o)). \quad (5)$$

Here is where the distinction matters: the single mask λ_m (resp. λ_o) on the difference gives rise to two linear masks on the state, namely λ_m and $\text{rot}^{-1}(\lambda_m)$ (resp. λ_o and $\text{rot}^{-1}(\lambda_o)$). Note that if E_U is the identity function, this simplifies to the closed formula (2) presented by Liu et al. [35].

In IDL analysis, we have $\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o) = \mathcal{C}_{E_L}(\mathcal{L}^\top \lambda_m, \mathcal{L}^\top \lambda_o)$ according to Proposition 3, yielding the following decomposition:

$$\mathcal{R}_E(\Delta_i, \lambda_o) = \sum_{\Delta_m, \lambda_m} \mathcal{P}_{E_U}(\Delta_i, \Delta_m) \cdot \mathcal{R}_{E_M}(\Delta_m, \lambda_m) \cdot \mathcal{C}_{E_L}(\mathcal{L}^\top \lambda_m, \mathcal{L}^\top \lambda_o). \quad (6)$$

Here, the masks λ_m and λ_o on the difference map to the linear mask $\mathcal{L}^\top \lambda_m$ and $\mathcal{L}^\top \lambda_o$ on the state. For the special case where the state is partitioned into exactly two slices, we have $\mathcal{L}^\top \lambda = \lambda \parallel \lambda$, thus

$$\mathcal{R}_E(\Delta_i, \lambda_o) = \sum_{\Delta_m, \lambda_m} \mathcal{P}_{E_U}(\Delta_i, \Delta_m) \cdot \mathcal{R}_{E_M}(\Delta_m, \lambda_m) \cdot \mathcal{C}_{E_L}(\lambda_m \parallel \lambda_m, \lambda_o \parallel \lambda_o). \quad (7)$$

Formulas (1), (5), (6) and (7) share a three-factor structure and all rely solely on the differential independence assumption between E_U , E_M and E_L . The only difference between SDL, RDL and IDL lies in the third factor, which describes how a mask on the intermediate difference relates to the mask on the output difference through E_L . This demonstrates that the SDLCT framework naturally extends to both RDL and IDL analyses, with the conceptual bridge being Proposition 2 and 3 which translate masks on differences into linear masks on the state.

Remark 5 (Mask on the difference vs. mask on the state). In the DTF view, the third factor $\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o)$ is naturally indexed by masks on the input and output *differences* of E_L . Propositions 2 and 3 translate these into linear masks on the *state* of E_L :

- SDL: $(\lambda_m, \lambda_o) \mapsto (\lambda_m, \lambda_o)$, the identity translation.
- RDL with rotational offset $t \neq 0$: $(\lambda_m, \lambda_o) \mapsto \{(\lambda_m, \lambda_o), (\text{rot}^{-1}(\lambda_m), \text{rot}^{-1}(\lambda_o))\}$, a pair of state masks.
- IDL with slice size m : $(\lambda_m, \lambda_o) \mapsto (\mathcal{L}^\top \lambda_m, \mathcal{L}^\top \lambda_o)$.

SDL is the only variant where the translation is trivial. This is why the historical SDL literature could conflate “mask on the difference” with “mask on the state” without consequence; the conflation breaks for RDL and IDL, and the unified framework must track the two notions separately.

Practical Estimation of the Correlation. In practice, directly computing $\mathcal{R}_E(\Delta_i, \lambda_o)$ via Eqn. (4) is often computationally infeasible because it involves a sum over all possible Δ_m and λ_m . Instead, one typically estimates $|\mathcal{R}_E(\Delta_i, \lambda_o)|$ by finding a single dominant trail:

- Identify a (standard, rotational or internal) differential trail through E_U from Δ_i to some specific Δ_m , with high probability p .
- For SDL analysis, find a single linear trail from λ_m to λ_o with high absolute correlation q ; for RDL analysis, find a linear trail from λ_m to λ_o with high absolute correlation q , and another from $\text{rot}^{-1}(\lambda_m)$ to $\text{rot}^{-1}(\lambda_o)$ with high absolute correlation q' ; for IDL analysis, find a single linear trail from $\mathcal{L}^\top \lambda_m$ to $\mathcal{L}^\top \lambda_o$ with high absolute correlation q .
- Theoretically or experimentally compute the correlation $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$ as r . The estimate is $|prq^2|$ for SDL analysis, $|prqq'|$ for RDL analysis and $|prq|$ for IDL analysis.

We refer to the concatenation of these components as an SDL/RDL/IDL trail, and its correlation is defined as prq^2 , $prqq'$ or prq , respectively. An SDL trail is a special case of an RDL trail with zero rotational offset.

3.4 Computing the Middle-Part Correlation $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$

We now present a unified method for computing $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$, which generalizes the SDL-specific approaches proposed by Peng et al. and Hadipour et al. (detailed in Appendix D).

Backward Reduction. Consider the decomposition $E_M = E_{ML} \circ E_{MU}$. For ease of demonstration, suppose that given the input difference of E_{MU} as Δ_m , the output difference of E_{MU} is always of the form $0^s \|1^t\| *^w$ ⁷, where $*^w$ represents w bits whose specific values are arbitrary and non-deterministic. This shape arises naturally from the boundary S-box layer of E_{MU} , with bits forced by Δ_m forming the deterministic 0^s and 1^t parts and the unconstrained bits forming $*^w$; this is the structure exploited by Peng et al. and Hadipour et al. in the SDL case (Appendix D). Then

Proposition 4. *Assuming that E_{ML} and E_{MU} are differentially independent, we can decompose $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$ as*

$$\mathcal{R}_{E_M}(\Delta_m, \lambda_m) = \sum_{\beta \in \mathbb{F}_2^w} \mathcal{TC}_{\partial_{E_{ML}}}(\beta, \lambda_m) \cdot \mathcal{R}_{E_{MU}}(\Delta_m, 0^{s+t} \|\beta\|),$$

where $\mathcal{TC}_{\partial_{E_{ML}}}(\beta, \lambda_m) = \sum_{\alpha \in \mathbb{F}_2^{s+t}} \mathcal{C}_{\partial_{E_{ML}}}((0, \alpha \|\beta\|), \lambda_m) (-1)^{\bigoplus_{i=s}^{s+t-1} \alpha[i]}$.

The proof of the above proposition is provided in Appendix F.1. When E_{ML} is a single S-box layer, $\mathcal{TC}_{\partial_{E_{ML}}}(\beta, \lambda_m)$ can be precomputed for each $\beta \in \mathbb{F}_2^w$. Consequently, this proposition essentially reduces the problem of computing $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$ to 2^w simpler sub-problems, i.e., computing $\mathcal{R}_{E_{MU}}(\Delta_m, 0^{s+t} \|\beta\|)$ for each $\beta \in \mathbb{F}_2^w$. We refer to this reduction as *backward reduction*.

Forward Reduction. Symmetrically, suppose that given the mask on the output difference of E_{ML} as λ_m , the mask on the input difference of E_{ML} is always of the form $0^s \|1^t\| *^w$, then

Proposition 5. *Assuming that E_{ML} and E_{MU} are differentially independent, we can decompose $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$ as*

$$\mathcal{R}_{E_M}(\Delta_m, \lambda_m) = \sum_{\beta \in \mathbb{F}_2^w} \mathcal{TP}_{E_{MU}}(\Delta_m, \beta) \cdot \mathcal{R}_{E_{ML}}(0^{s+t} \|\beta\|, \lambda_m).$$

where $\mathcal{TP}_{E_{MU}}(\Delta_m, \beta) = \sum_{\alpha \in \mathbb{F}_2^{s+t}} \mathcal{P}_{E_{MU}}(\Delta_m, \alpha \|\beta\|) (-1)^{\bigoplus_{i=s}^{s+t-1} \alpha[i]}$.

The proof of the above proposition is provided in Appendix F.2. When E_{MU} is a single S-box layer, $\mathcal{TP}_{E_{MU}}(\Delta_m, \beta)$ can be precomputed for each $\beta \in \mathbb{F}_2^w$. Thus, the proposition also reduces the problem of computing $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$ to 2^w sub-problems, namely computing $\mathcal{R}_{E_{ML}}(0^{s+t} \|\beta\|, \lambda_m)$ for each $\beta \in \mathbb{F}_2^w$. We refer to this reduction as *forward reduction*.

⁷ Writing the deterministic and free bits as the contiguous blocks $0^s \|1^t\| *^w$ is only for notational convenience; in general they are interleaved across the $s+t+w$ positions

In practice, we compute $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$ by applying the forward and backward reductions alternately and recursively, peeling off one round at a time, until the sub-problems are computationally tractable. The choice of which reduction to apply at each step is guided by which direction yields fewer sub-problems. These two reduction methods generalize Peng et al. and Hadipour et al.’s methods shown in Appendix D and extend them to RDL and IDL analyses.

Determining Determinism. Both the forward and backward reductions rely on identifying the determinism for the output difference of E_{MU} and the corresponding mask on this difference. That is, we have to determine which bits of the output difference of E_{MU} , and the corresponding masks on that difference, are deterministically 0 or 1, given that an input difference Δ_m and an output mask λ_m . For the SDL case, we have detailed in Appendix D how to track the propagation of determinism for standard differences and their masks through (inverse of) S-boxes and XOR operations. We next show that utilizing the structure of specific primitives, the determinism propagation of (rotational or internal) differences and their masks can be reduced to the SDL case.

Taking Xoodoo as an example to illustrate this in IDL analysis, we consider an internal difference generated by XORing S-boxes at z -coordinates 0-15 with those at z -coordinates 16-31. Consequently, all possible output internal differences of E_{MU} can be characterized from Δ_m using SDDT (through S-box) and XOR operations (through linear layer), and all possible masks on the output internal difference of E_{MU} can be characterized from λ_m using LAT (through the inverse of S-box) and XOR operations (through the inverse of linear layer). We can therefore determine the determinism of the output difference of E_{MU} and the determinism of the mask on this difference using the same methodology as in SDL analysis.

Effectiveness under Low-Correlation Conditions. One may question whether the method in this subsection still remains effective under low-correlation conditions. To this end, we give an example of a 5-round Xoodoo RDL distinguisher in Appendix K.1, whose correlation is 2^{-33} and cannot be experimentally verified. However, our method can compute this correlation in minutes on a laptop.

4 Applications on S-Box-Based Primitives

In this section, we apply the unified framework of Section 3 to the S-box-based permutations Xoodoo- p and Ascon- p . The CP models are solved using the CP solver OR-Tools [1] (invoked in Minizinc [51] with 32 threads) on a personal laptop equipped with an Intel 13900HQ CPU, requiring anywhere from a few minutes to a few hours to complete. The source files for modeling and computing the correlations are available at our [github repository](#). Throughout this section, hexadecimal constants are written in the standard MSB-on-the-left form (e.g., 0x80000000 denotes the vector with only $x[31] = 1$).

Search Strategy for S-Box-Based Primitives. From Section 3.4, it can be heuristically claimed that the computational cost of evaluating $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$

is governed by the number of difference bits throughout E_M that simultaneously have non-deterministic values and non-deterministic masks. Therefore, we adapt Hadipour et al.'s CP model for SDL distinguisher search into our unified framework, which seeks to achieve a balance between the differential probability of E_U , the linear correlation between masks-on-difference of E_L , and the complexity for computing the SDL/RDL/IDL correlation of E_M . The details of this model are provided in Appendix J (see Algorithm 5). The CP solver is configured to output all found solutions during the optimization. For each solution with a sufficiently small objective value, we read the values of $\mathbf{x}, \mathbf{y}, \Delta_i, \Delta_m, \lambda_m, \lambda_o$ as $p, q, \Delta_i, \Delta_m, \lambda_m, \lambda_o$. Then, we compute $\mathcal{R}_{E_M}(\Delta_m, \lambda_m)$ as r using the method in Section 3.4, and estimate $|\mathcal{R}_E(\Delta_i, \lambda_o)|$ as $|prq|$. If this estimated value is large enough, it identifies an effective distinguisher of E .

4.1 Xoodoo

The structure of Xoodoo is described in Appendix K.1. We decompose the 6-round Xoodoo permutation E into three components: 2-round $E_U = (L \circ S)^2$, 3-round $E_M = S \circ (L \circ S)^2$, and 1-round $E_L = S \circ L$, where L is the linear layer.

For RDL analysis, the left rotation offset along the z -axis direction is set to 1. Since the propagation of linear masks is invariant along the z -axis, we have $\mathcal{C}_{\partial_{E_L}}((0, \alpha), \beta) = \mathcal{C}_{E_L}^2(\alpha, \beta)$ for any α, β . Using our CP model, the rotational differential of E_U is identified as

$$\begin{array}{ccc}
 & y=0 & y=1 & y=2 & & y=0 & y=1 & y=2 \\
 x=0 : & 02000060, 01000012, 00000042 & & & \xrightarrow{2^{-24}} & 00000490, 00000000, 00000000 \\
 x=1 : & 00000000, 00000000, 00001000 & & & & 00040240, 00000000, 20140000 \\
 x=2 : & 00000000, 00000000, 04000000 & & & & 00000000, 000402c0, 00000000 \\
 x=3 : & 00000000, 00000000, 00000000 & & & & 00000000, 00000000, 00000000
 \end{array}$$

The linear approximation of E_L is identified as

$$\begin{array}{ccc}
 00000000, 00000000, 00000000 & & 00000000, 00000000, 00000000 \\
 00000800, 00000400, 00000000 & \xrightarrow{2^{-2}} & 00000800, 00000800, 00000800 \\
 00000000, 00000000, 00000000 & & 00000800, 00000800, 00000000 \\
 00000000, 00000000, 00000000 & & 00000000, 00000000, 00000000
 \end{array}$$

The RDL correlation of E_M is computed as $2^{-16.10}$ using our techniques. Thus, the RDL correlation of E is estimated as $2^{-24-16.10-4} = 2^{-44.10}$. Using 2^{37} pairs, we experimentally evaluated the RDL correlation of E_M as $2^{-15.95}$, which aligns closely with the theoretical value. We also compute the RDL correlation of $E_L \circ E_M$ as $2^{-20.0931}$ using the techniques in Section 3.4, which matches the theoretical estimate $2^{-16.10-4} = 2^{-20.10}$.

For IDL analysis, we XOR the S-boxes at z -coordinates 0-15 with those at z -coordinates 16-31. Using our CP model, the internal differential of E_U is identified as

$$\begin{array}{ccc}
 0000, 0016, 0000, & & 8280, 0000, 0000, \\
 0000, 0000, 0000, & \xrightarrow{2^{-32}} & 0000, 0000, 0000, \\
 1020, 0810, 2000, & & 0000, 0000, 0000, \\
 0000, 2040, 8000, & & 0000, 0000, 0020,
 \end{array}$$

For E_L , we identified a linear approximation $\lambda_m \parallel \lambda_m \xrightarrow{2^{-4}} \lambda_o \parallel \lambda_o$, where

$$\lambda_m = \begin{matrix} 0000, 0000, 0000, \\ 0000, 8000, 0000, \\ 0000, 0000, 0000, \\ 0000, 0000, 0100, \end{matrix} \lambda_o = \begin{matrix} 0000, 0000, 0000, \\ 0000, 0000, 0800, \\ 0000, 0001, 0000, \\ 0000, 0000, 0000. \end{matrix}$$

The IDL correlation of E_M is computed as 2^{-9} using our techniques. Thus, the IDL correlation of E is estimated as $2^{-32-9-4} = 2^{-45}$. Using 2^{31} pairs, we experimentally evaluated the IDL correlation of E_M as 2^{-9} , aligning perfectly with the theoretical value. Moreover, we compute the IDL correlation of $E_L \circ E_M$ as 2^{-13} using the techniques in Section 3.4, which also aligns with the experimental value $2^{-12.99}$.

4.2 Ascon

The structure of Ascon is described in Appendix K.2.

For the RDL analysis on 6-round Ascon permutation, the rotation is performed inside each 64-bit word with a rotational offset 2. We decompose the permutation into 2-round $E_U = (L \circ S)^2$ (L is the linear layer of Ascon), 3-round $E_M = (L \circ S)^3$ and 1-round $E_L = L \circ S$. Using our CP model, the rotational differential of E_U is identified as

$$\begin{matrix} 0000000000000140, & 0000000000000000, \\ 0000000000000000, & 0000000000000000, \\ 0000000000000000, & \xrightarrow{2^{-8}} 0000000000000050, . \\ 0000000000000140, & a140000000000280, \\ 0000000000000140 & 0000000000000000. \end{matrix}$$

The linear approximation of E_L is identified as

$$\begin{matrix} 0000000000000000, & 2db492dbb6926489, \\ 0000000000000000, & 6e221eea5a47ccba, \\ 0000000000004000, & \xrightarrow{2^{-1}} 0000000000000000, . \\ 0000000000000000, & 0000000000000000, \\ 0000000000000000, & 7cc63cdc4b8fec29. \end{matrix}$$

The RDL correlation of E_M is computed as 2^{-11} using our techniques, perfectly aligning with the experimental value $2^{-11.12}$. Thus, the correlation of the overall 6-round RDL distinguisher is $2^{-8-11-2} = 2^{-21}$. We also experimentally evaluated the RDL correlation of $E_L \circ E_M$ as $2^{-13.14}$, which closely matches the theoretical value $2^{-11-2} = 2^{-13}$.

5 SDL/RDL Analysis on ARX Primitives

In the last section, the unified framework already yields effective bit-wise CP models for SDL/RDL/IDL distinguisher search on S-box-based primitives. In this section, we turn our attention to ARX primitives, for which we focus primarily on SDL and RDL analyses. Note that the approach for S-box-based primitives

will lose its effectiveness on ARX primitives, because modular addition does not have a compact SDDT/LAT structure amenable to determinism tracking. Instead, we adopt a different strategy: directly model an SDL/RDL trail through the entire primitive $E = E_L \circ E_M \circ E_U$. The theories for modeling the propagation of (standard or rotational) differences and linear masks through ARX operations have been well-established in previous work [15, 37, 44, 23], thus the primary challenge lies in modeling the SDL/RDL correlation of the modular addition operation.

In Appendix M, we summarize the theoretical formulas established by Niu et al. [45] for evaluating the SDL and RDL correlations of modular addition. These formulas are derived from a perspective of (standard or rotational) differential propagation and involve chains of 4×4 matrix multiplications. Each matrix is determined by two bits of the input standard or rotational difference and one bit of the output linear mask, yielding eight distinct cases in total. Niu et al. raised the integration of this formula into an automated search model as an open problem, which we partially resolve in this section.

5.1 SDL Correlation Computation for Modular Addition

Setup. Consider the modular addition function $z = \boxplus_n^{(b)}(x, y) = (x + y + b) \bmod 2^n$ with initial carry bit b , the carry vector $c \in \mathbb{F}_2^{n+1}$ satisfies:

$$c[0] = b; c[i+1] = \text{maj}(x[i], y[i], c[i]), \text{ for } 0 \leq i < n,$$

where $\text{maj}(a, b, c) = ab \oplus ac \oplus bc$ is the majority function, and $\lfloor c \rfloor^n = x \oplus y \oplus z$.

Given a fixed input standard difference (Δ_x, Δ_y) and a mask λ_z on the output standard difference, our goal is to compute

$$\mathcal{R}_{\boxplus_n^{(b)}}^{\text{SDL}}((\Delta_x, \Delta_y), \lambda_z) = \frac{1}{2^{2n}} \sum_{x, y} (-1)^{\langle \lambda_z, \Delta_z \rangle},$$

where $\Delta_z = \boxplus_n^{(b)}(x, y) \oplus \boxplus_n^{(b)}(x \oplus \Delta_x, y \oplus \Delta_y)$. Define $\Delta_c \in \mathbb{F}_2^n$ as the standard difference on the carry bits that satisfy $\Delta_c[0] = 0$, and for $0 \leq i < n-1$,

$$\Delta_c[i+1] = \text{maj}(x[i], y[i], c[i]) \oplus \text{maj}(x[i] \oplus \Delta_x[i], y[i] \oplus \Delta_y[i], c[i] \oplus \Delta_c[i]). \quad (8)$$

Since $\Delta_z = \Delta_x \oplus \Delta_y \oplus \Delta_c$, we obtain:

$$\mathcal{R}_{\boxplus_n^{(b)}}^{\text{SDL}}((\Delta_x, \Delta_y), \lambda_z) = \frac{(-1)^{\langle \lambda_z, \Delta_x \oplus \Delta_y \rangle}}{2^{2n}} \sum_{x, y} (-1)^{\langle \lambda_z, \Delta_c \rangle}. \quad (9)$$

Thus, the problem reduces to computing $\sum_{x, y} (-1)^{\langle \lambda_z, \Delta_c \rangle}$, inspiring us to investigate the Walsh transform of $(-1)^{\lambda_z[i+1] \cdot \Delta_c[i+1]}$ for $0 \leq i < n-1$. When $\lambda_z[i+1] = 0$, $(-1)^{\lambda_z[i+1] \cdot \Delta_c[i+1]} = 1$. Thus, we consider $(-1)^{\Delta_c[i+1]}$.

Lemma 4 (Walsh transform of $(-1)^{\Delta_c[i+1]}$). For each $\Delta_c[i+1], 0 \leq i < n-1$ mentioned above,

$$(-1)^{\Delta_c[i+1]} = \sum_{\alpha, \beta \in \mathbb{F}_2^3} \epsilon_{\alpha, \beta} (-1)^{\langle \alpha, x[i] \| y[i] \| c[i] \rangle \oplus \langle \beta, \Delta_x[i] \| \Delta_y[i] \| \Delta_c[i] \rangle},$$

where

$$\epsilon_{\alpha, \beta} = \begin{cases} \frac{1}{4}, & \text{if } wt(\alpha \oplus \beta) \text{ and } wt(\beta) \text{ are both odd and equal,} \\ -\frac{1}{4}, & \text{if } wt(\alpha \oplus \beta) \text{ and } wt(\beta) \text{ are both odd but unequal,} \\ 0, & \text{otherwise.} \end{cases}$$

Lemma 5. For each $\Delta_c[i+1], 0 \leq i < n-1$ mentioned above,

$$\sum_{x[i], y[i]} (-1)^{\Delta_c[i+1]} = (-1)^{\Delta_x[i] \oplus \Delta_y[i] \oplus \Delta_c[i]} + (-1)^{\Delta_x[i]} + (-1)^{\Delta_y[i]} + (-1)^{\Delta_c[i]}.$$

In particular,

$$\sum_{x[i], y[i]} (-1)^{\Delta_c[i+1]} = \begin{cases} 0, & \text{if } \Delta_x[i] \neq \Delta_y[i], \\ 2(-1)^{\Delta_c[i]} + 2(-1)^{\Delta_x[i]}, & \text{if } \Delta_x[i] = \Delta_y[i]. \end{cases}$$

The proofs for the above two lemmas are provided in Appendix [G](#).

Computing SDL Correlation of Modular Addition by Recurrence. We now set up a recurrence to compute $\sum_{x,y} (-1)^{\langle \lambda_z, \Delta_c \rangle}$. For $0 \leq i \leq n-1$, define:

$$S_1^i = \sum_{[x]^{i+1}, [y]^{i+1}} (-1)^{\langle \lfloor \lambda_z \rfloor^i \| 1, \lfloor \Delta_c \rfloor^{i+1} \rangle}, S_0^i = \sum_{[x]^{i+1}, [y]^{i+1}} (-1)^{\langle \lfloor \lambda_z \rfloor^i \| 0, \lfloor \Delta_c \rfloor^{i+1} \rangle}.$$

The initial values are $S_1^0 = 4(-1)^{\Delta_c[0]}$ and $S_0^0 = 4$. The idea is to express the target sum $\sum_{x,y} (-1)^{\langle \lambda_z, \Delta_c \rangle}$ as a linear combination of S_1^{n-1} and S_0^{n-1} , and then compute the latter via a recurrence.

Lemma 6. For $0 \leq i \leq n-1$,

$$\sum_{[x]^{i+1}, [y]^{i+1}} (-1)^{\langle \lfloor \lambda_z \rfloor^{i+1}, \lfloor \Delta_c \rfloor^{i+1} \rangle} = [\lambda_z[i] \ 1 - \lambda_z[i]] \begin{bmatrix} S_1^i \\ S_0^i \end{bmatrix}.$$

This follows directly from the definition: depending on whether $\lambda_z[i] = 1$ or 0 , we select either S_1^i or S_0^i .

Lemma 7 (Recurrence). For $1 \leq i \leq n-1$,

$$\begin{bmatrix} S_1^i \\ S_0^i \end{bmatrix} = 2^2 C_{\Delta_x[i-1], \Delta_y[i-1]} B_{\lambda_z[i-1]} \begin{bmatrix} S_1^{i-1} \\ S_0^{i-1} \end{bmatrix},$$

where $B_{\lambda_z[i-1]} = \begin{bmatrix} 1 - \lambda_z[i-1] & \lambda_z[i-1] \\ \lambda_z[i-1] & 1 - \lambda_z[i-1] \end{bmatrix}$, and

$$C_{\Delta_x[i-1], \Delta_y[i-1]} = \begin{bmatrix} \frac{1}{4}(-1)^{\Delta_x[i-1] \oplus \Delta_y[i-1]} + \frac{1}{4} & \frac{1}{4}(-1)^{\Delta_x[i-1]} + \frac{1}{4}(-1)^{\Delta_y[i-1]} \\ 0 & 1 \end{bmatrix}.$$

Due to page limits, the derivation of this lemma is given in Appendix H.1.

Finally, assembling the full product gives:

Proposition 6 (SDL correlation of $\boxplus_n^{(b)}$). *We can compute the SDL correlation $\mathcal{R}_{\boxplus_n^{(b)}}^{\text{SDL}}((\Delta_x, \Delta_y), \lambda_z)$ in Eqn. (9) as*

$$(-1)^{\langle \lambda_z, \Delta_x \oplus \Delta_y \rangle} \begin{bmatrix} 0 & 1 \end{bmatrix} \prod_{i=n-1}^0 (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} (-1)^{\Delta_c[0]} \\ 1 \end{bmatrix}, \quad (10)$$

where $\Delta_c[0]$ is the XOR of initial carry bits of the two modular additions, and the order of the product is from $i = n - 1$ down to $i = 0$.

Proof. By Eqn. (9) and Lemma 6 with $i = n - 1$:

$$\sum_{x,y} (-1)^{\langle \lambda_z, \Delta_c \rangle} = [\lambda_z[n-1] \ 1 - \lambda_z[n-1]] \begin{bmatrix} S_1^{n-1} \\ S_0^{n-1} \end{bmatrix} = \begin{bmatrix} 0 & 1 \end{bmatrix} C_{\Delta_x[n-1], \Delta_y[n-1]} B_{\lambda_z[n-1]} \begin{bmatrix} S_1^{n-1} \\ S_0^{n-1} \end{bmatrix}.$$

By Lemma 7, iterating the recurrence from $i = n - 1$ down to $i = 1$ gives

$$\begin{bmatrix} S_1^{n-1} \\ S_0^{n-1} \end{bmatrix} = 2^{2n-2} \prod_{i=n-2}^0 (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} S_1^0 \\ S_0^0 \end{bmatrix}.$$

Substituting $S_1^0 = 4(-1)^{\Delta_c[0]}$ and $S_0^0 = 4$ completes the proof. $\square$

The derivation above depends only on the value of $\Delta_c[0]$, not on the individual carry bits. This yields:

Corollary 2. *If we generalize to $\Delta_z = \boxplus_n^{(b)}(x, y) \oplus \boxplus_n^{(b')}(x \oplus \Delta_x, y \oplus \Delta_y)$ with $\Delta_c[0] = b \oplus b'$, then the entire derivation above still holds, and the correlation $\frac{1}{2^{2n}} \sum_{x,y} (-1)^{\langle \lambda_z, \Delta_z \rangle}$ can still be computed using Expression (10). We denote this generalized correlation as $\mathcal{R}_{\boxplus_n^{(b,b')}}^{\text{SDL}}((\Delta_x, \Delta_y), \lambda_z)$.*

Modeling the SDL Correlation of Modular Addition. We now translate the matrix multiplication of Proposition 6 into constraints suitable for a CP model. The goal is to model the negative logarithm (base 2) of the absolute value of the following general expression:

$$\begin{bmatrix} 0 & 1 \end{bmatrix} \prod_{i=n-1}^0 (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} a \\ 1 \end{bmatrix}, \quad (11)$$

where the constant $a \in \{0, 1, -1\}$ (Expression (10) is a special case $a = (-1)^{\Delta_c[0]}$).

Two variants of CP model are developed. The basic workflow of the first variant is depicted in Fig. 1, where at each iteration i , the matrices $C_{\Delta_x[i], \Delta_y[i]}$ and $B_{\lambda_z[i]}$ are determined by the values of $(\Delta_x[i], \Delta_y[i])$ and $\lambda_z[i]$ respectively, and right-multiplied onto X . Because the final X is typically not a power of 2, we approximate $-\log_2 |X|$ by rounding it down, with an error of at most 1.

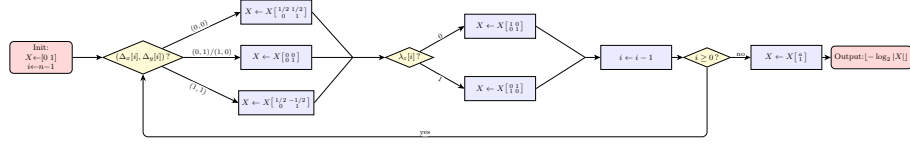


Fig. 1: Workflow of First Variant

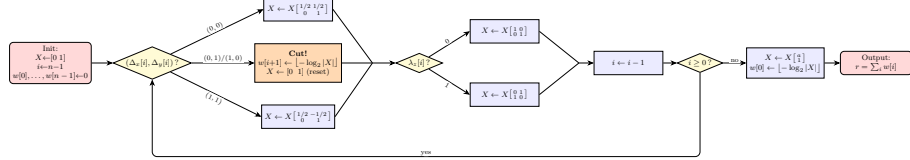


Fig. 2: Workflow of Second Variant

Since $C_{\Delta_x[i], \Delta_y[i]}$ contains entries of $\frac{1}{2}$, the division-by-2 can be performed $n-1$ times in the worst case, which exceeds the largest precision modern CP solvers support if n is large (e.g., $n = 64$). Thus, we develop the second variant, whose basic workflow is depicted in Fig. 2 based on the following proposition. The key observation is that when $\Delta_x[i] \neq \Delta_y[i]$, the matrix $C_{\Delta_x[i], \Delta_y[i]}$ becomes rank-1, which disconnects the running multiplication into independent factors.

Proposition 7. *For Expression (11), if there exists $0 \leq j_0 < j_1 < \dots < j_{m-1} < j_m = n-1$ such that $\Delta_x[j_i] \neq \Delta_y[j_i]$ for $0 \leq i < m$, then the expression can be factored as $p_m p_{m-1} \dots p_0$, where $p_s, 1 \leq s \leq m$ is given as*

$$p_s = [0 \ 1] \prod_{i=j_s}^{j_{s-1}+1} (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

$$\text{and } p_0 = [0 \ 1] \prod_{i=j_0}^0 (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} a \\ 1 \end{bmatrix}.$$

Proof. When $\Delta_x[j_i] \neq \Delta_y[j_i]$, $C_{\Delta_x[j_i], \Delta_y[j_i]} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} [0 \ 1] C_{\Delta_x[j_i], \Delta_y[j_i]}$. Substitute this into Expression (11) yields the claimed factorization. $\square$

An intuitive illustration for this proposition is provided in Appendix C (see Example 1). The factors in the proposition occupy disjoint bit positions from $\{0, 1, \dots, n-1\}$: p_0 contains indices 0 to j_0 , while $p_s, 1 \leq s \leq m$ contains indices $j_{s-1} + 1$ to j_s . Consequently, the negative logarithm of the absolute value of Expression (11) can be decomposed into a sum of $m+1$ independent terms: $\sum_{s=0}^m (-\log_2 |p_s|)$, and the error of the second variant is at most $m+1$, which is dynamic because we do not know Δ_x, Δ_y and λ_z in the modeling process. In our experiment, the error of the second variant is usually less than 2 on $n = 64$.

We recommend the first variant for moderate block sizes (e.g., $n = 32$) and the second one for larger block sizes (e.g., $n = 64$). The two variants are specified by Algorithm 1 and 2 in Appendix B.1. All the constraints used in the two variants are CP/SAT/MILP friendly and can be efficiently handled by modern CP solvers. We defer the implementation details of the two variants to Appendix B.1.

5.2 RDL Correlation Computation for Modular Addition

We now extend the analysis to the RDL setting, where the rotational offset $t > 0$.

Setup. Given input rotational difference (Δ_x, Δ_y) with offset t , let $x' = \text{rot}^{-1}(x \oplus \Delta_x)$, $y' = \text{rot}^{-1}(y \oplus \Delta_y)$, i.e., $\Delta_x = x \oplus \text{rot}(x')$ and $\Delta_y = y \oplus \text{rot}(y')$ (note that $\Delta_x \neq x \oplus x'$ in general; in particular, $\lfloor \Delta_x \rfloor^t = \lfloor x \rfloor^t \oplus \lceil x' \rceil^t$). The output rotational difference is $\Delta_z = \boxplus_n^{(b)}(x, y) \oplus \text{rot}(\boxplus_n^{(b)}(x', y')) = \Delta_x \oplus \Delta_y \oplus \Delta_c$, where $\Delta_c = \lfloor c \rfloor^n \oplus \text{rot}(\lfloor c' \rfloor^n)$, and $c \in \mathbb{F}_2^{n+1}$ and $c' \in \mathbb{F}_2^{n+1}$ are the carry vectors of $\boxplus_n^{(b)}(x, y)$ and $\boxplus_n^{(b)}(x', y')$, respectively. The RDL correlation is thus

$$\mathcal{R}_{\boxplus_n^{(b)}}^{\text{RDL}}((\Delta_x, \Delta_y), \lambda_z) = \frac{(-1)^{\langle \lambda_z, \Delta_x \oplus \Delta_y \rangle}}{2^{2n}} \sum_{x, y \in \mathbb{F}_2^n} (-1)^{\langle \lambda_z, \Delta_c \rangle}. \quad (12)$$

Unlike the SDL case, Δ_c here involves two carry vectors c and c' that share the same (x, y) but through a rotational relationship. The carries decompose into two groups: $\lfloor \Delta_c \rfloor^t$ depends only on $\lfloor x \rfloor^t, \lfloor y \rfloor^t$ and $\Delta_c[0]$, while $\lceil \Delta_c \rceil^{n-t}$ depends only on $\lceil x \rceil^{n-t}, \lceil y \rceil^{n-t}$ and $\Delta_c[t]$. Indeed, $\lfloor \Delta_c \rfloor^t$ is the carry difference of the two t -bit additions $\lfloor x \rfloor^t + \lfloor y \rfloor^t + b$ and $\lceil x' \rceil^t + \lceil y' \rceil^t + c'[n-t]$, where $\lceil x' \rceil^t = \lfloor x \rfloor^t \oplus \lfloor \Delta_x \rfloor^t$, $\lceil y' \rceil^t = \lfloor y \rfloor^t \oplus \lfloor \Delta_y \rfloor^t$ and $\Delta_c[0] = b \oplus c'[n-t]$; the lower $n-t$ bits of x', y' (i.e., the upper bits $\lceil x \rceil^{n-t}, \lceil y \rceil^{n-t}$) influence $\lfloor \Delta_c \rfloor^t$ only through the single carry bit $c'[n-t]$, that is, through $\Delta_c[0]$. Crucially, $\Delta_c[0]$ depends on $c'[n-t]$ (hence on $\lceil x \rceil^{n-t}, \lceil y \rceil^{n-t}$), while $\Delta_c[t]$ depends on $c[t]$ (hence on $\lfloor x \rfloor^t, \lfloor y \rfloor^t$). This cross-dependence means the sum does not simply factor into two independent halves.

Computing RDL Correlation of Modular Addition by Marginal Sums.

To handle this, we write

$$\begin{aligned} (-1)^{\langle \lfloor \lambda_z \rfloor^t, \lfloor \Delta_c \rfloor^t \rangle} &= f_0(\lfloor x \rfloor^t, \lfloor y \rfloor^t) (-1)^{\Delta_c[0]} + f_1(\lfloor x \rfloor^t, \lfloor y \rfloor^t), \\ (-1)^{\langle \lceil \lambda_z \rceil^{n-t}, \lceil \Delta_c \rceil^{n-t} \rangle} &= g_0(\lceil x \rceil^{n-t}, \lceil y \rceil^{n-t}) (-1)^{\Delta_c[t]} + g_1(\lceil x \rceil^{n-t}, \lceil y \rceil^{n-t}). \end{aligned}$$

Although f_0, f_1, g_0 and g_1 do not have simple closed forms, their marginal sums can be computed using the SDL machinery established above.

Lemma 8 (Marginal sums via SDL formula). *For f_0, f_1, g_0 and g_1 above,*

$$\begin{aligned} \frac{1}{2^{2t}} \sum_{\lfloor x \rfloor^t, \lfloor y \rfloor^t} [f_0 \ f_1] &= [0 \ 1] \prod_{i=t-1}^0 (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \\ \frac{1}{2^{2n-2t}} \sum_{\lceil x \rceil^{n-t}, \lceil y \rceil^{n-t}} [g_0 \ g_1] &= [0 \ 1] \prod_{i=n-1}^t (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}), \end{aligned}$$

where the matrices $C_{\Delta_x[i], \Delta_y[i]}, B_{\lambda_z[i]}$ for $0 \leq i < n$ are defined as in Lemma 7.

Due to page limits, the derivation of this lemma is given in Appendix H.2.

Lemma 9 (Cross terms). *The ‘cross’ marginals involving both halves satisfy*

$$\begin{aligned} \frac{1}{2^{2n-2t}} \sum_{\lceil x \rceil^{n-t}, \lceil y \rceil^{n-t}} \left((-1)^{\Delta_c[0]} [g_0 \ g_1] \right) &= \begin{cases} \begin{bmatrix} \frac{1}{2^{n-t}} & 0 \\ 0 & \frac{1}{2^{n-t}} \end{bmatrix}, & \text{if } wt(\lceil \lambda_z \rceil^{n-t}) \text{ is odd,} \\ 0 & \text{otherwise;} \end{cases} \\ \frac{1}{2^{2t}} \sum_{\lfloor x \rfloor^t, \lfloor y \rfloor^t} \left((-1)^{\Delta_c[t]} [f_0 \ f_1] \right) &= \begin{cases} \begin{bmatrix} \frac{1}{2^t} & 0 \\ 0 & \frac{1}{2^t} \end{bmatrix}, & \text{if } wt(\lfloor \lambda_z \rfloor^t) \text{ is odd,} \\ 0 & \text{otherwise.} \end{cases} \end{aligned}$$

Due to page limits, the derivation of this lemma is given in Appendix H.3.

Finally, we decompose the RDL correlation of modular addition into the sum of marginals.

Proposition 8 (Computing the RDL correlation of $\boxplus_n^{(b)}$). *We can compute $\mathcal{R}_{\boxplus_n^{(b)}}^{\text{RDL}}((\Delta_x, \Delta_y), \lambda_z)$ as $(-1)^{\langle \lambda_z, \Delta_x \oplus \Delta_y \rangle} (pq + \alpha_{01} + \alpha_{10} + \alpha_{00})$, where*

$$\begin{aligned} \alpha_{00} &= \begin{cases} \frac{1}{2^n}, & \text{if both } wt(\lceil \lambda_z \rceil^{n-t}) \text{ and } wt(\lfloor \lambda_z \rfloor^t) \text{ are odd,} \\ 0, & \text{otherwise;} \end{cases} \\ \alpha_{01} &= \begin{cases} \frac{1}{2^t} \begin{bmatrix} 0 & 1 \end{bmatrix} \prod_{i=n-1}^t (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} 1 \\ 0 \end{bmatrix}, & \text{if } wt(\lfloor \lambda_z \rfloor^t) \text{ is even,} \\ 0, & \text{otherwise;} \end{cases} \\ \alpha_{10} &= \begin{cases} \frac{1}{2^{n-t}} \begin{bmatrix} 0 & 1 \end{bmatrix} \prod_{i=t-1}^0 (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} 1 \\ 0 \end{bmatrix}, & \text{if } wt(\lceil \lambda_z \rceil^{n-t}) \text{ is even,} \\ 0, & \text{otherwise;} \end{cases} \end{aligned}$$

$$p = \begin{bmatrix} 0 & 1 \end{bmatrix} \prod_{i=n-1}^t (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} 0 \\ 1 \end{bmatrix}, q = \begin{bmatrix} 0 & 1 \end{bmatrix} \prod_{j=t-1}^0 (C_{\Delta_x[j], \Delta_y[j]} B_{\lambda_z[j]}) \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Proof (Proof sketch). The proof partitions $\sum_{x,y} (-1)^{\langle \lambda_z, \Delta_c \rangle}$ into four sums, factors each sum over $\lfloor x \rfloor^t, \lfloor y \rfloor^t$ and $\lceil x \rceil^{n-t}, \lceil y \rceil^{n-t}$, and then applies Lemma 8 and 9 to evaluate each sum. See Appendix H.4 for details.

Remark 6 (Comparison with Niu et al.’s Formulas). While Niu et al. derived the SDL/RDL correlation of modular addition using 4×4 matrix chains based on differential propagation (see Appendix M), our formulas employ 2×2 matrices derived from the Walsh transform perspective, reducing the matrix size by 75%. Furthermore, each matrix in our chain is determined by either two bits of the input difference (standard or rotational), yielding four cases, or one bit of the output mask, yielding two cases. This reduced complexity, with at most four cases per matrix compared to eight in Niu et al.’s formulas, can be readily handled by modern automated solvers.

Modeling the RDL Correlation of Modular Addition. When both t and $n - t$ are large, the terms $\alpha_{00}, \alpha_{10}, \alpha_{01}$ in Proposition 8 become negligible since they contain factors of $(\frac{1}{2})^t$ or $(\frac{1}{2})^{n-t}$. Thus,

$$\mathcal{R}_{\boxplus_n^{(b)}}^{\text{RDL}}((\Delta_x, \Delta_y), \lambda_z) \approx (-1)^{\langle \lambda_z, \Delta_x \oplus \Delta_y \rangle} pq.$$

Under this approximation, $-\log_2 |\mathcal{R}_{\boxplus_n^{(b)}}^{\text{RDL}}((\Delta_x, \Delta_y), \lambda_z)|$ splits as $-\log_2 |p|$ and $-\log_2 |q|$, where p and q are special cases of Expression (11). We can therefore model each half by calling the CP model of the previous subsection and then summing them up. The implementation details are specified by Algorithm 3 in Appendix B.2.

Error Analysis of the $|pq|$ Approximation. Since the approximation drops the three terms $\alpha_{00}, \alpha_{01}, \alpha_{10}$ in Proposition 8, it is important to quantify when this is safe. From the definitions, $|\alpha_{00}| \leq 2^{-n}$, $|\alpha_{01}| \leq 2^{-t}$ and $|\alpha_{10}| \leq 2^{-(n-t)}$, hence the absolute error satisfies $|\alpha_{00} + \alpha_{01} + \alpha_{10}| \leq 2^{1-\min(t, n-t)}$. The error is significant only when it dominates $|pq|$. A simple sufficiency condition is: the relative error $\frac{|\alpha_{00} + \alpha_{01} + \alpha_{10}|}{|pq|}$ is at most ε whenever $\min(t, n - t) \geq -\log_2 |pq| + \log_2(1/\varepsilon) + 1$. In cryptanalysis, what matters in practice is the negative logarithm of the correlation; we therefore consider the approximation reliable when the negative logarithms of the approximate and exact RDL correlations differ by at most 1. This yields the more practical condition

$$\min(t, n - t) \geq -\log_2 |pq| + 2. \quad (13)$$

The failure regime is when $\min(t, n - t)$ is small (e.g., $t = 1$) or $|pq|$ is very small. In that regime, the dropped terms can dominate, and the CP model may overlook useful distinguishers, or, equivalently, the correlation it reports for a returned trail may differ noticeably from the truth. As a safeguard for any RDL trail returned by the CP model, one can recompute the exact RDL correlation via Proposition 8 to check accuracy. Empirically we observe that for RDL trails with high overall correlations, each modular addition in E_M tends to have $|pq|$ close to 1 or 2^{-1} , in which case Eqn. (13) holds for a wide range of t . For example, $|pq| = 2^{-1}$ requires only $\min(t, n - t) \geq 3$. This is precisely why we adopt $t = 20$ for Alzette, placing the approximation well within its reliable regime.

Remark 7 (Comparison with Lv et al.’s Formulas). A 2×2 matrix product formula for the ordinary SDL correlation of modular addition was also obtained independently by Lv et al. [40] through a different, purely algebraic route: they observe that in the SDL case, Niu et al.’s 4×4 matrices collapse onto the parity $\Delta_c[0]$ of the two carry vectors, and give explicit 2×2 matrices indexed by three bits. Their formulation and ours track different but equivalent state vectors and compute the same scalar.

Beyond the difference in derivation route, our formulation adds the following:

- (a) The factored $C \cdot B$ structure of Proposition 6, in which the input-difference matrix C is decoupled from the mask matrix B . When $\Delta_x[i] \neq \Delta_y[i]$, C

degenerates to rank 1, which is the structural property that enables the chain decomposition of Proposition 7 and the dynamic splitting of Algorithm 2. In contrast, the rank degeneration never surfaces in the framework of [40], let alone is exploited. Moreover, as the authors themselves acknowledge, their MIQCP model is practical only for small modular additions ($n \leq 32$), and they resort to an ad hoc fixed scaling factor that sacrifices accuracy.

- (b) The extension to RDL (Proposition 8 and Algorithm 3), which [40] does not address.
- (c) Our CP models do not involve quadratic constraints, thus can be solved efficiently.

6 Applications on ARX Primitives

We now apply the techniques of Section 5 to the ARX primitives Alzette, SPECK, SipHash and Chaskey. As before, all CP models are solved using OR-Tools (invoked in Minizinc) on a personal laptop.

Search Strategy on ARX Primitives. We illustrate the overall search strategy using the 8-round RDL distinguisher search on Alzette as a concrete example. Decompose 8-round Alzette into 2-round E_U , 1-round E_M , and 5-round E_L . Construct a CP model that searches for an RDL trail through $E_L \circ E_M \circ E_U$, where a rotational differential trail through E_U , two linear trails through E_L whose masks are rotationally related, and RDL correlation of modular additions in E_M are modeled. The objective is to minimize the negative logarithm of this RDL trail’s absolute correlation.

The CP solver outputs a rotational differential trail through E_U : $\Delta_0 \xrightarrow{p_0} \Delta_1 \xrightarrow{p_1} \Delta_2$, and two linear trails through E_L : $\lambda_0 \xrightarrow{q_0} \lambda_1 \xrightarrow{q_1} \dots \xrightarrow{q_4} \lambda_5$ and $\text{rot}^{-1}(\lambda_0) \xrightarrow{q'_0} \text{rot}^{-1}(\lambda_1) \xrightarrow{q'_1} \dots \xrightarrow{q'_4} \text{rot}^{-1}(\lambda_5)$, where Δ_2 and λ_0 are the input difference and the output mask of E_M . To account for the clustering effect and potential violations of the round independence assumption in ARX primitives, we experimentally evaluate the 3-round RDL correlation of $E_M \circ E_U$ from Δ_0 to λ_0 , denoted as r . The absolute correlation of the entire 8-round RDL distinguisher is then estimated as $|rq_0q_1q_2q_3q_4q'_0q'_1q'_2q'_3q'_4|$. Note that the number of middle rounds evaluated experimentally may vary depending on the target primitive and the specific trail obtained.

6.1 Alzette

The structure of Alzette, along with a 13-round SDL distinguisher, is presented in Appendix K.3.

For the RDL analysis on 8-round Alzette with $c = b7e15162$, the left rotation is performed inside each 32-bit word and the rotational offset is chosen as 20. We decompose the primitive into 2-round E_U , 1-round E_M and 5-round E_L . We identify a rotational differential $(80001fc8, 00000f80) \xrightarrow{2^{-15}} (20003f69, 083007e5)$ for E_U , and for E_L we identify two linear approximations

$(00080c06, 08040c0c) \xrightarrow{2^{-7}} (16181406, 0c061018)$ and $(80c06000, 40c0c080) \xrightarrow{2^{-7}} (81406161, 610180c0)$. The RDL correlation of E_M is experimentally evaluated as $2^{-0.91}$ using 2^{31} pairs, thus the trail-based estimate of the overall 8-round RDL correlation is $2^{-15-0.91-14} = 2^{-29.91}$. In particular, the negative logarithm of the absolute RDL correlation of E_M is approximated as 0 by Algorithm 3, which invokes `SDLModelV2` with $r_{\max}$ being 7. Moreover, the error between the exact absolute RDL correlation of E_M (computed via Proposition 8 with all four terms) and the $|pq|$ approximation is verified to be 0.

The trail-based estimate turns out to be conservative: the rotational differential of E_U alone has a measured probability of $2^{-16.06}$ (with 2^{36} pairs) instead of 2^{-15} , a mild violation of the round-independence assumption, whereas the RDL correlation of the 3-round $E_M \circ E_U$ measured with 2^{40} pairs is $2^{-10.30}$, far above the product $2^{-16.06-0.91} = 2^{-16.97}$, indicating a strong clustering effect. Combining the measured correlation of $E_M \circ E_U$ with the two linear trails of E_L , we estimate the overall 8-round RDL correlation as $2^{-10.30-14} = 2^{-24.30}$.

6.2 SPECK

The structure of SPECK is described in Appendix K.4.

For 12-round SPECK48, we first model an SDL trail through 6-round E_U , 1-round E_M and 5-round E_L , where Algorithm 1 is applied on E_M . This gives an SDL trail whose correlation is approximated as 2^{-30} by the model. Then, we extend the E_M to 6 rounds, such that the decomposition becomes 3-round E_U , 6-round E_M and 3-round E_L . The final SDL distinguisher consists of a standard differential $(020082, 120200) \xrightarrow{2^{-6}} (000080, 000080)$ of E_U and a linear approximation $(003000, 000030) \xrightarrow{2^{-3}} (000809, 000808)$ of E_L , with the SDL correlation of the middle part being experimentally evaluated as $2^{-9.90}$. Thus, the trail-based estimate of the correlation of this SDL distinguisher is $2^{-6-9.90-3 \times 2} = 2^{-21.90}$. Measuring the 9-round $E_L \circ E_M$ directly with the real key schedule (averaged over 1024 keys with 2^{38} pairs each, see Appendix L) gives $2^{-16.44}$ instead of $2^{-15.90}$, while the differential probability of E_U is confirmed as $2^{-5.99}$. We therefore estimate the correlation of the 12-round SDL distinguisher as $2^{-5.99-16.44} = 2^{-22.43}$, which still improves on the previous best $2^{-23.40}$ [24].

For 14-round SPECK64, we first model an SDL trail through 7-round E_U , 1-round E_M and 6-round E_L , where Algorithm 1 is applied on E_M . This gives an SDL trail whose correlation is approximated as 2^{-41} by the model. Then, we extend the E_M to 7 rounds, such that the decomposition becomes 3-round E_U , 7-round E_M and 4-round E_L . The final SDL distinguisher consists of a standard differential $(00820200, 00001202) \xrightarrow{2^{-6}} (80000000, 80000000)$ of E_U and a linear approximation $(00000100, 00000001) \xrightarrow{2^{-6}} (00034903, 00430803)$ of E_L , with the SDL correlation of E_M being experimentally evaluated as $2^{-13.33}$. Thus, the correlation of this SDL distinguisher is estimated as $2^{-6-13.33-6 \times 2} = 2^{-31.33}$. This is the longest SDL distinguisher for SPECK64 so far.

6.3 SipHash

The structure of SipHash is described in Appendix K.5.

For 5-round finalization of SipHash, we model an SDL trail through 3.5-round E_U , 0.5-round E_M and 1-round E_L , where Algorithm 2 is applied on E_M . This gives an SDL trail whose correlation is approximated as 2^{-37} by the model. We then extend E_M to 4 rounds, such that the decomposition becomes 1-round E_U , 4-round E_M and 0-round E_L . The final SDL distinguisher consists of the following standard differential of E_U :

$$\begin{array}{ccc} 8010042000010000 & & 0000000080040000 \\ 0014002020010000 & \xrightarrow{2^{-15}} & 0000000000040000 \\ 0402200000000002 & & 0000000000000000 \\ 0402200000000000 & & 0000000000000000 \end{array},$$

and the output linear mask of E_L is identified as 0300000003000000. The SDL correlation of E_M is experimentally evaluated as $2^{-5.15}$. Thus, the overall SDL correlation of the 5-round finalization is estimated as $2^{-15-5.15} = 2^{-20.15}$.

6.4 Chaskey

The structure of Chaskey, along with our 7-round SDL distinguisher, is presented in Appendix K.6.

7 Comparison to Prior Work

SDLCT machinery. The SDLCT framework [5] and its multi-round middle-part extensions [46,25] are confined to the SDL setting. Our Proposition 1 reproduces the SDLCT formula in the SDL case and lifts the same three-part structure to RDL and IDL. The relation between differential probabilities and DL correlations via the Walsh transform (Lemma 1) was proved for SDL in [25]; under the DTF abstraction the statement holds for all three variants without modification.

RDL closed formulas. Liu et al. [35] proved a closed two-factor RDL formula and a Walsh-coefficient identity for the rotational difference transition. Both appear as specializations of Proposition 1 and Proposition 2. Our contribution is the three-part structure and the explicit mask-on-difference / mask-on-state translation, which becomes essential when moving to IDL.

IDL formulation. Internal differential-linear cryptanalysis has not been formulated in the literature. The definition in Section 3, together with the DTF treatment, gives the first such formulation and yields the first IDL distinguisher (6-round Xoodoo- p).

2×2 matrix formula for modular addition. Niu et al. [45] derived a 4×4 matrix-product formula for the SDL correlation of modular addition and posed its integration into an automated search model as an open problem. Independently of our work, Lv et al. [40] obtained a 2×2 matrix formula for the same

quantity. Our derivation reaches the same 2×2 form, with the additional contributions of (i) extending it to RDL and (ii) exposing the rank-1 structure of the input-difference matrix, which decouples the running product into independent factors and makes a CP approximation with only linear inequalities and table lookups practical. This partially resolves Niu et al.’s open problem.

Geometric approach. The mixed-basis framework of Beyne [12], extended to SDL by Hu et al. [27] and to round-by-round approximation by Hu, Niu and Wang [26], avoids the differential-independence assumption that our DTF approach requires. The two lines are complementary: DTF/Walsh extends seamlessly to RDL and IDL but assumes independence; the geometric approach is independence-free but has not yet been developed for RDL or IDL.

8 Conclusion

In this paper, we present a unified framework for interpreting SDL, RDL, and IDL analyses using the Walsh transform. This framework extends state-of-the-art techniques in SDL analysis to both RDL and IDL analyses, enabling efficient evaluation and discovery of SDL/RDL/IDL distinguishers. From the perspective of the Walsh transform, we propose 2×2 matrix product formulas as well as an approximate CP model for computing the SDL and RDL correlations of modular addition, facilitating the automated search for SDL and RDL distinguishers on ARX primitives. Leveraging these newly developed techniques, we discover improved RDL and SDL distinguishers for *Xoodoo-p*, *Ascon-p*, *Alzette*, *SPECK*, *SipHash*, and *Chaskey*, achieving lower correlations or covering more rounds.

Acknowledgements. The authors thank the anonymous reviewers for their valuable comments. This work is supported by the National Key R&D Program of China (Grant No. 2024YFA1013000) and supported by the Flemish Government through the Cybersecurity Research Program Flanders with grant number: VOEWICS02. Zhongfeng Niu is supported by the Singapore NRF-NRFI08-2022-0013 grant. Kai Hu and Meiqin Wang are supported by the National Cryptologic Science Fund of China (2025NCSF01013), the National Natural Science Foundation of China (Grant No. U2336207), the Department of Science & Technology of Shandong Province (No. SYS202201), and the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM114). Kai Hu is also supported by the National Cryptologic Science Fund of China (2025NCSF02007), the National Natural Science Foundation of China (62402283), the Shandong Provincial Natural Science Foundation (No. 2025HWYQ-025), the Natural Science Foundation of Jiangsu Province (BK20240420), the Program of TaiShan Scholars Special Fund for young scholars (Grant No. tsqn202507063), and the Program of Qilu Young Scholars of Shandong University. Meiqin Wang is further supported by Key R&D Program of Shandong Province, China (Grant No. 2026CXPT037).

References

1. Google OR-Tools. <https://developers.google.com/optimization>
2. Ashur, T., Liu, Y.: Rotational cryptanalysis in the presence of constants. *IACR Trans. Symmetric Cryptol.* **2016**(1), 57–70 (2016). <https://doi.org/10.13154/TOSC.V2016.I1.57-70>, <https://doi.org/10.13154/tosc.v2016.i1.57-70>
3. Aumasson, J., Bernstein, D.J.: SipHash: A fast short-input PRF. In: Galbraith, S.D., Nandi, M. (eds.) *Progress in Cryptology - INDOCRYPT 2012*, 13th International Conference on Cryptology in India, Kolkata, India, December 9–12, 2012. Proceedings. Lecture Notes in Computer Science, vol. 7668, pp. 489–508. Springer (2012). https://doi.org/10.1007/978-3-642-34931-7_28, https://doi.org/10.1007/978-3-642-34931-7_28
4. Baignères, T.: Quantitative security of block ciphers - designs and cryptanalysis tools. Ph.D. thesis, EPFL, Switzerland (2008). <https://doi.org/10.5075/EPFL-THESIS-4208>, <https://doi.org/10.5075/epfl-thesis-4208>
5. Bar-On, A., Dunkelman, O., Keller, N., Weizman, A.: DLCT: A new tool for differential-linear cryptanalysis. In: Ishai, Y., Rijmen, V. (eds.) *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Darmstadt, Germany, May 19–23, 2019, Proceedings, Part I. Lecture Notes in Computer Science, vol. 11476, pp. 313–342. Springer (2019). https://doi.org/10.1007/978-3-030-17653-2_11, https://doi.org/10.1007/978-3-030-17653-2_11
6. Barbero, S., Bellini, E., Makarim, R.H.: Rotational analysis of ChaCha permutation. *Adv. Math. Commun.* **17**(6), 1422–1439 (2023). <https://doi.org/10.3934/AMC.2021057>, <https://doi.org/10.3934/amc.2021057>
7. Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., Wingers, L.: The SIMON and SPECK families of lightweight block ciphers. *IACR Cryptol. ePrint Arch.* p. 404 (2013), <http://eprint.iacr.org/2013/404>
8. Beierle, C., Biryukov, A., dos Santos, L.C., Großschädl, J., Perrin, L., Udovenko, A., Velichkov, V., Wang, Q.: Alzette: A 64-bit ARX-box - (feat. CRAX and TRAX). In: Micciancio, D., Ristenpart, T. (eds.) *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference*, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part III. Lecture Notes in Computer Science, vol. 12172, pp. 419–448. Springer (2020). https://doi.org/10.1007/978-3-030-56877-1_15, https://doi.org/10.1007/978-3-030-56877-1_15
9. Beierle, C., Biryukov, A., dos Santos, L.C., Großschädl, J., Perrin, L., Udovenko, A., Velichkov, V., Wang, Q.: Lightweight AEAD and hashing using the Sparkle permutation family. *IACR Trans. Symmetric Cryptol.* **2020**(S1), 208–261 (2020). <https://doi.org/10.13154/TOSC.V2020.IS1.208-261>, <https://doi.org/10.13154/tosc.v2020.iS1.208-261>
10. Beierle, C., Leander, G., Todo, Y.: Improved differential-linear attacks with applications to ARX ciphers. In: Micciancio, D., Ristenpart, T. (eds.) *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference*, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part III. Lecture Notes in Computer Science, vol. 12172, pp. 329–358. Springer (2020). https://doi.org/10.1007/978-3-030-56877-1_12, https://doi.org/10.1007/978-3-030-56877-1_12
11. Bertoni, G., Daemen, J., Hoffert, S., Peeters, M., Assche, G.V., Keer, R.V.: Farfalle: parallel permutation-based cryptography. *IACR Trans. Symmetric Cryptol.*

- 2017(4), 1–38 (2017), <https://tosc.iacr.org/index.php/ToSC/article/view/801>
12. Beyne, T.: A geometric approach to linear cryptanalysis. In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security*, Singapore, December 6–10, 2021, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 13090, pp. 36–66. Springer (2021). https://doi.org/10.1007/978-3-030-92062-3_2, https://doi.org/10.1007/978-3-030-92062-3_2
 13. Biham, E., Dunkelman, O., Keller, N.: Enhancing differential-linear cryptanalysis. In: Zheng, Y. (ed.) *Advances in Cryptology - ASIACRYPT 2002, 8th International Conference on the Theory and Application of Cryptology and Information Security*, Queenstown, New Zealand, December 1–5, 2002, Proceedings. *Lecture Notes in Computer Science*, vol. 2501, pp. 254–266. Springer (2002). https://doi.org/10.1007/3-540-36178-2_16, https://doi.org/10.1007/3-540-36178-2_16
 14. Biham, E., Shamir, A.: Differential cryptanalysis of DES-like cryptosystems. In: Menezes, A., Vanstone, S.A. (eds.) *CRYPTO '90*. LNCS, vol. 537, pp. 2–21. Springer (1990). https://doi.org/10.1007/3-540-38424-3_1, https://doi.org/10.1007/3-540-38424-3_1
 15. Biryukov, A., Lambin, B., Udovenko, A.: Exact formula for RX-differential probability through modular addition for all rotations. *IACR Trans. Symmetric Cryptol.* **2025**(1), 542–591 (2025). <https://doi.org/10.46586/TOSC.V2025.I1.542-591>, <https://doi.org/10.46586/tosc.v2025.i1.542-591>
 16. Blondeau, C., Leander, G., Nyberg, K.: Differential-linear cryptanalysis revisited. *J. Cryptol.* **30**(3), 859–888 (2017). <https://doi.org/10.1007/S00145-016-9237-5>, <https://doi.org/10.1007/s00145-016-9237-5>
 17. Canteaut, A., Carlet, C., Charpin, P., Fontaine, C.: On cryptographic properties of the cosets of $r(1, m)$. *IEEE Trans. Inf. Theory* **47**(4), 1494–1513 (2001). <https://doi.org/10.1109/18.923730>, <https://doi.org/10.1109/18.923730>
 18. Chen, Y., Bao, Z., Yu, H.: Differential-linear approximation semi-unconstrained searching and partition tree: Application to LEA and Speck. In: Guo, J., Steinfield, R. (eds.) *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security*, Guangzhou, China, December 4–8, 2023, Proceedings, Part III. *Lecture Notes in Computer Science*, vol. 14440, pp. 223–255. Springer (2023). https://doi.org/10.1007/978-981-99-8727-6_8, https://doi.org/10.1007/978-981-99-8727-6_8
 19. Daemen, J., Govaerts, R., Vandewalle, J.: Correlation matrices. In: Preneel, B. (ed.) *Fast Software Encryption: Second International Workshop*. Leuven, Belgium, 14–16 December 1994, Proceedings. *Lecture Notes in Computer Science*, vol. 1008, pp. 275–285. Springer (1994). https://doi.org/10.1007/3-540-60590-8_21, https://doi.org/10.1007/3-540-60590-8_21
 20. Daemen, J., Hoffert, S., Assche, G.V., Keer, R.V.: The design of Xoodoo and Xoofff. *IACR Trans. Symmetric Cryptol.* **2018**(4), 1–38 (2018). <https://doi.org/10.13154/TOSC.V2018.I4.1-38>, <https://doi.org/10.13154/tosc.v2018.i4.1-38>
 21. Dinur, I., Dunkelman, O., Shamir, A.: Collision attacks on up to 5 rounds of SHA-3 using generalized internal differentials. In: Moriai, S. (ed.) *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11–13, 2013. Revised Selected Papers*. *Lecture Notes in Computer Science*, vol. 8424, pp. 219–240. Springer (2013). https://doi.org/10.1007/978-3-662-43933-3_12, https://doi.org/10.1007/978-3-662-43933-3_12

22. Dobraunig, C., Eichlseder, M., Mendel, F., Schl  ffer, M.: Ascon v1.2: Lightweight authenticated encryption and hashing. *J. Cryptol.* **34**(3), 33 (2021). <https://doi.org/10.1007/s00145-021-09398-9>, <https://doi.org/10.1007/s00145-021-09398-9>
23. Fu, K., Wang, M., Guo, Y., Sun, S., Hu, L.: MILP-based automatic search algorithms for differential and linear trails for Speck. In: Peyrin, T. (ed.) *Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers. Lecture Notes in Computer Science*, vol. 9783, pp. 268-288. Springer (2016). https://doi.org/10.1007/978-3-662-52993-5_14, https://doi.org/10.1007/978-3-662-52993-5_14
24. Gong, X., Wang, Q., Hao, Y., Jiao, L., Hu, X.: Persistence of hourglass(-like) structure: Improved differential-linear distinguishers for several ARX ciphers. In: Hanaoka, G., Yang, B. (eds.) *Advances in Cryptology - ASIACRYPT 2025 - 31st International Conference on the Theory and Application of Cryptology and Information Security, Melbourne, VIC, Australia, December 8-12, 2025, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 16245, pp. 317-351. Springer (2025). https://doi.org/10.1007/978-981-95-5018-0_11, https://doi.org/10.1007/978-981-95-5018-0_11
25. Hadipour, H., Derbez, P., Eichlseder, M.: Revisiting differential-linear attacks via a boomerang perspective with application to AES, Ascon, CLEFIA, SKINNY, PRESENT, KNOT, TWINE, WARP, LBlock, Simeck, and SERPENT. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV. Lecture Notes in Computer Science*, vol. 14923, pp. 38-72. Springer (2024). https://doi.org/10.1007/978-3-031-68385-5_2, https://doi.org/10.1007/978-3-031-68385-5_2
26. Hu, K., Niu, Z., Wang, M.: Round-based approximation of (higher-order) differential-linear correlation - A geometric approach perspective. In: Daemen, J., Thom  , E. (eds.) *Advances in Cryptology - EUROCRYPT 2026 - 45th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Rome, Italy, May 10-14, 2026, Proceedings, Part VI. pp. 152-180. Lecture Notes in Computer Science*, Springer (2026). https://doi.org/10.1007/978-3-032-25333-0_6, https://doi.org/10.1007/978-3-032-25333-0_6
27. Hu, K., Zhang, C., Chang, C., Zhang, J., Wang, M., Peyrin, T.: Unlocking mix-basis potential: Geometric approach for combined attacks. In: Kalai, Y.T., Kamara, S.F. (eds.) *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part V. Lecture Notes in Computer Science*, vol. 16004, pp. 293-334. Springer (2025). https://doi.org/10.1007/978-3-032-01901-1_10, https://doi.org/10.1007/978-3-032-01901-1_10
28. Khovratovich, D., Nikolic, I.: Rotational cryptanalysis of ARX. In: Hong, S., Iwata, T. (eds.) *Fast Software Encryption, 17th International Workshop, FSE 2010, Seoul, Korea, February 7-10, 2010, Revised Selected Papers. Lecture Notes in Computer Science*, vol. 6147, pp. 333-346. Springer (2010). https://doi.org/10.1007/978-3-642-13858-4_19, https://doi.org/10.1007/978-3-642-13858-4_19
29. Khovratovich, D., Nikolic, I., Rechberger, C.: Rotational rebound attacks on reduced Skein. In: Abe, M. (ed.) *Advances in Cryptology - ASIACRYPT 2010 - 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings. Lecture Notes*

- in Computer Science, vol. 6477, pp. 1–19. Springer (2010). https://doi.org/10.1007/978-3-642-17373-8_1, https://doi.org/10.1007/978-3-642-17373-8_1
30. Kraleva, L., Ashur, T., Rijmen, V.: Rotational cryptanalysis on MAC algorithm Chaskey. In: Conti, M., Zhou, J., Casalicchio, E., Spognardi, A. (eds.) Applied Cryptography and Network Security - 18th International Conference, ACNS 2020, Rome, Italy, October 19–22, 2020, Proceedings, Part I. Lecture Notes in Computer Science, vol. 12146, pp. 153–168. Springer (2020). https://doi.org/10.1007/978-3-030-57808-4_8, https://doi.org/10.1007/978-3-030-57808-4_8
 31. Lai, X.: Higher order derivatives and differential cryptanalysis. In: Blahut, R.E., Costello, D.J., Maurer, U., Mittelholzer, T. (eds.) Communications and Cryptography, vol. 276, pp. 227–233. Springer (1994). https://doi.org/10.1007/978-1-4615-2694-0_23, https://doi.org/10.1007/978-1-4615-2694-0_23
 32. Lai, X., Massey, J.L., Murphy, S.: Markov ciphers and differential cryptanalysis. In: Davies, D.W. (ed.) Advances in Cryptology - EUROCRYPT '91, Workshop on the Theory and Application of Cryptographic Techniques, Brighton, UK, April 8–11, 1991, Proceedings. Lecture Notes in Computer Science, vol. 547, pp. 17–38. Springer (1991). https://doi.org/10.1007/3-540-46416-6_2, https://doi.org/10.1007/3-540-46416-6_2
 33. Langford, S.K., Hellman, M.E.: Differential-linear cryptanalysis. In: Desmedt, Y. (ed.) Advances in Cryptology - CRYPTO '94, 14th Annual International Cryptology Conference, Santa Barbara, California, USA, August 21–25, 1994, Proceedings. Lecture Notes in Computer Science, vol. 839, pp. 17–25. Springer (1994). https://doi.org/10.1007/3-540-48658-5_3, https://doi.org/10.1007/3-540-48658-5_3
 34. Leurent, G.: Improved differential-linear cryptanalysis of 7-round Chaskey with partitioning. In: Fischlin, M., Coron, J. (eds.) Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8–12, 2016, Proceedings, Part I. Lecture Notes in Computer Science, vol. 9665, pp. 344–371. Springer (2016). https://doi.org/10.1007/978-3-662-49890-3_14, https://doi.org/10.1007/978-3-662-49890-3_14
 35. Liu, Y., Niu, Z., Sun, S., Li, C., Hu, L.: Rotational differential-linear cryptanalysis revisited. *J. Cryptol.* **36**(1), 3 (2023). <https://doi.org/10.1007/S00145-022-09440-4>, <https://doi.org/10.1007/s00145-022-09440-4>
 36. Liu, Y., Sun, S., Li, C.: Rotational cryptanalysis from a differential-linear perspective - practical distinguishers for round-reduced FRIET, Xoodoo, and Alzette. In: Canteaut, A., Standaert, F. (eds.) Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17–21, 2021, Proceedings, Part I. Lecture Notes in Computer Science, vol. 12696, pp. 741–770. Springer (2021). https://doi.org/10.1007/978-3-030-77870-5_26, https://doi.org/10.1007/978-3-030-77870-5_26
 37. Liu, Y., Witte, G.D., Ranea, A., Ashur, T.: Rotational-XOR cryptanalysis of reduced-round SPECK. *IACR Trans. Symmetric Cryptol.* **2017**(3), 24–36 (2017). <https://doi.org/10.13154/TOSC.V2017.I3.24-36>, <https://doi.org/10.13154/tosc.v2017.i3.24-36>
 38. Lu, J., Liu, Y., Ashur, T., Sun, B., Li, C.: Rotational-XOR cryptanalysis of Simon-like block ciphers. In: Liu, J.K., Cui, H. (eds.) Information Security and Privacy - 25th Australasian Conference, ACISP 2020, Perth, WA, Australia, November 30 - December 2, 2020, Proceedings. Lecture Notes in Computer Science, vol. 12248,

- pp. 105–124. Springer (2020). https://doi.org/10.1007/978-3-030-55304-3_6, https://doi.org/10.1007/978-3-030-55304-3_6
39. Lu, J., Liu, Y., Ashur, T., Sun, B., Li, C.: Improved rotational-XOR cryptanalysis of Simon-like block ciphers. *IET Inf. Secur.* **16**(4), 282–300 (2022). <https://doi.org/10.1049/ISE2.12061>, <https://doi.org/10.1049/ise2.12061>
 40. Lv, G., Jin, C., Cui, T.: A MIQCP-based automatic search algorithm for differential-linear trails of ARX ciphers (long paper). *IACR Cryptol. ePrint Arch.* p. 259 (2023), <https://eprint.iacr.org/2023/259>
 41. Matsui, M.: Linear cryptanalysis method for DES cipher. In: Helleseth, T. (ed.) *Advances in Cryptology - EUROCRYPT '93, Workshop on the Theory and Application of Cryptographic Techniques, Lofthus, Norway, May 23–27, 1993, Proceedings. Lecture Notes in Computer Science*, vol. 765, pp. 386–397. Springer (1993). https://doi.org/10.1007/3-540-48285-7_33, https://doi.org/10.1007/3-540-48285-7_33
 42. Morawiecki, P., Pieprzyk, J., Srebrny, M.: Rotational cryptanalysis of round-reduced Keccak. In: Moriai, S. (ed.) *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11–13, 2013. Revised Selected Papers. Lecture Notes in Computer Science*, vol. 8424, pp. 241–262. Springer (2013). https://doi.org/10.1007/978-3-662-43933-3_13, https://doi.org/10.1007/978-3-662-43933-3_13
 43. Mouha, N., Mennink, B., Herrewewe, A.V., Watanabe, D., Preneel, B., Verbauwhede, I.: Chaskey: An efficient MAC algorithm for 32-bit microcontrollers. In: Joux, A., Youssef, A.M. (eds.) *Selected Areas in Cryptography - SAC 2014 - 21st International Conference, Montreal, QC, Canada, August 14–15, 2014, Revised Selected Papers. Lecture Notes in Computer Science*, vol. 8781, pp. 306–323. Springer (2014). https://doi.org/10.1007/978-3-319-13051-4_19, https://doi.org/10.1007/978-3-319-13051-4_19
 44. Mouha, N., Preneel, B.: Towards finding optimal differential characteristics for ARX: Application to Salsa20. *Cryptology ePrint Archive, Paper 2013/328* (2013), <https://eprint.iacr.org/2013/328>
 45. Niu, Z., Sun, S., Liu, Y., Li, C.: Rotational differential-linear distinguishers of ARX ciphers with arbitrary output linear masks. In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15–18, 2022, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 13507, pp. 3–32. Springer (2022). https://doi.org/10.1007/978-3-031-15802-5_1, https://doi.org/10.1007/978-3-031-15802-5_1
 46. Peng, T., Zhang, W., Weng, J., Ding, T.: New approaches for estimating the bias of differential-linear distinguishers. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2024, Proceedings, Part IV. Lecture Notes in Computer Science*, vol. 14923, pp. 174–205. Springer (2024). https://doi.org/10.1007/978-3-031-68385-5_6, https://doi.org/10.1007/978-3-031-68385-5_6
 47. Peyrin, T.: Improved differential attacks for ECHO and Grøstl. In: Rabin, T. (ed.) *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15–19, 2010. Proceedings. Lecture Notes in Computer Science*, vol. 6223, pp. 370–392. Springer (2010). https://doi.org/10.1007/978-3-642-14623-7_20, https://doi.org/10.1007/978-3-642-14623-7_20

48. Preneel, B., Leekwijck, W.V., Linden, L.V., Govaerts, R., Vandewalle, J.: Propagation characteristics of Boolean functions. In: Damgård, I. (ed.) *Advances in Cryptology - EUROCRYPT '90, Workshop on the Theory and Application of Cryptographic Techniques*, Aarhus, Denmark, May 21-24, 1990, Proceedings. *Lecture Notes in Computer Science*, vol. 473, pp. 161–173. Springer (1990). https://doi.org/10.1007/3-540-46877-3_14, https://doi.org/10.1007/3-540-46877-3_14
49. Song, L., Huang, Z., Yang, Q.: Automatic differential analysis of ARX block ciphers with application to SPECK and LEA. In: Liu, J.K., Steinfeld, R. (eds.) *Information Security and Privacy - 21st Australasian Conference, ACISP 2016, Melbourne, VIC, Australia, July 4-6, 2016, Proceedings, Part II*. *Lecture Notes in Computer Science*, vol. 9723, pp. 379–394. Springer (2016). https://doi.org/10.1007/978-3-319-40367-0_24, https://doi.org/10.1007/978-3-319-40367-0_24
50. Sun, S., Hu, L., Wang, P., Qiao, K., Ma, X., Song, L.: Automatic security evaluation and (related-key) differential characteristic search: Application to SIMON, PRESENT, LBlock, DES(L) and other bit-oriented block ciphers. In: Sarkar, P., Iwata, T. (eds.) *ASIACRYPT 2014, Part I*. LNCS, vol. 8873, pp. 158–178. Springer (2014). https://doi.org/10.1007/978-3-662-45611-8_9, https://doi.org/10.1007/978-3-662-45611-8_9
51. Team, M.: MiniZinc: A Free and Open-Source Constraint Modeling Language (2024), <https://www.minizinc.org/>, accessed: 2024-10-01
52. Zhang, Z., Hou, C., Liu, M.: Collision attacks on round-reduced SHA-3 using conditional internal differentials. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Lyon, France, April 23-27, 2023, Proceedings, Part IV. *Lecture Notes in Computer Science*, vol. 14007, pp. 220–251. Springer (2023). https://doi.org/10.1007/978-3-031-30634-1_8, https://doi.org/10.1007/978-3-031-30634-1_8
53. Zhang, Z., Hou, C., Liu, M.: Probabilistic linearization: Internal differential collisions in up to 6 rounds of SHA-3. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV. *Lecture Notes in Computer Science*, vol. 14923, pp. 241–272. Springer (2024). https://doi.org/10.1007/978-3-031-68385-5_8, https://doi.org/10.1007/978-3-031-68385-5_8
54. Zhang, Z., Hou, C., Liu, M.: Preimage attacks on up to 5 rounds of SHA-3 using internal differentials. In: Fehr, S., Fouque, P. (eds.) *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Madrid, Spain, May 4-8, 2025, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 15601, pp. 333–363. Springer (2025). https://doi.org/10.1007/978-3-031-91107-1_12, https://doi.org/10.1007/978-3-031-91107-1_12

Appendix

A SDLCT Definition

Definition 4 (Standard Differential-Linear Connectivity Table [5]). For a vectorial Boolean function $S : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, the SDLCT of S is a $2^n \times 2^m$ table whose rows correspond to the input standard difference Δ to S and whose columns correspond to the output linear mask λ of S . The entry at index (Δ, λ) is

$$\text{SDLCT}_S(\Delta, \lambda) = 2 \cdot \Pr[\langle \lambda, S(x) \oplus S(x \oplus \Delta) \rangle = 0] - 1.$$

B CP Models for SDL/RDL Correlation of Modular Addition

B.1 CP Model for SDL Correlation

The first variant is implemented by Algorithm 1, where we introduce intermediate integer variable pairs $\mathbf{u}_0, \dots, \mathbf{u}_n$ and $\mathbf{v}_0, \dots, \mathbf{v}_{n-1}$, each consisting of two integer variables, to track the running matrix multiplication from left to right. A global scaling factor $2^{r_{\max}}$ is applied to keep all intermediate values as integers. Initially, $\mathbf{u}_n$ is set as $(0, 2^{r_{\max}})$, encoding the left matrix $\begin{bmatrix} 0 & 1 \end{bmatrix}$ scaled by $2^{r_{\max}}$; then at each step of the iteration from $i = n-1$ down to 0, the following relations are encoded by constraints:

$$[\mathbf{v}_i[0] \ \mathbf{v}_i[1]] = [\mathbf{u}_{i+1}[0] \ \mathbf{u}_{i+1}[1]] C_{\Delta_x[i], \Delta_y[i]}, [\mathbf{u}_i[0] \ \mathbf{u}_i[1]] = [\mathbf{v}_i[0] \ \mathbf{v}_i[1]] B_{\lambda_z[i]}.$$

After all iterations, $\mathbf{u}_0 = 2^{r_{\max}} \begin{bmatrix} 0 & 1 \end{bmatrix} \prod_{i=n-1}^0 (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]})$. The absolute value of Expression (11) is $|a \cdot \mathbf{u}_0[0] + \mathbf{u}_0[1]| / 2^{r_{\max}}$, and its negative logarithm r is approximated by the constraint: $2^{r_{\max}-r-1} < |a \cdot \mathbf{u}_0[0] + \mathbf{u}_0[1]| \leq 2^{r_{\max}-r}$, which can be transformed into constraints naturally supported by CP solvers (see Appendix N). The error of this approximation is at most 1.

Note that when $\Delta_x[i] = \Delta_y[i]$, each iteration involves dividing $\mathbf{u}_{i+1}[0]$ by 2 (Lines 15–20), thus the division-by-2 may produce non-integer intermediate values after sufficiently many iterations. Indeed, the scaling factor $2^{r_{\max}}$ guarantees that for bit positions j from $n - r_{\max} - 1$ to $n - 1$, both $\mathbf{u}_j[0]$ and $\mathbf{u}_j[1]$ remain integers regardless of $\Delta_x, \Delta_y, \lambda_z$. Ideally, $r_{\max} = n - 1$ would cover all bit positions. However, for large n (e.g., $n = 64$), this may cause precision overflow since modern CP solvers may support at most 32-bit integer arithmetic, which significantly limits the search space of Algorithm 1 (i.e., values of $\Delta_x, \Delta_y, \lambda_z$ that produce non-integer intermediate values will fall outside the model’s feasible region). Thus, we further sacrifice accuracy for larger search space on the basis of Algorithm 1, developing the second variant implemented by Algorithm 2, where the differences from Algorithm 1 are highlighted in red colors.

Since the values of $j_0, \dots, j_{m-1}$ and m in Proposition 7 are unknown a priori in an automated model, we introduce an auxiliary vector $\mathbf{w}$ of n integer variables to dynamically track factor boundaries. Specifically:

Algorithm 1: Automated Model (Version 1) for Characterizing the Negative Logarithm (Base 2) of the absolute value of Expression (11)

```

1 Procedure SDLModelV1(The model  $\mathcal{M}$ ,  $\Delta_x, \Delta_y, \lambda_z$ , constant  $a$ , the modular
   addition size  $n$ ):
2   Declare  $\mathbf{r}$  as an integer variable
3    $\mathbf{u}_0 = \text{SDLModelV1Core}(\mathcal{M}, \Delta_x, \Delta_y, \lambda_z, n)$ 
4    $\mathcal{M}.\text{constraints} \leftarrow 0 \leq \mathbf{r} \leq r_{\max} - 1 \triangleright$  Values of  $\Delta_x, \Delta_y, \lambda_z$  for which the
   absolute value of Expression (11) is smaller than  $2^{-(r_{\max}-1)}$  are excluded
   from the search space of this algorithm.
5    $\mathcal{M}.\text{constraints} \leftarrow 2^{r_{\max}-\mathbf{r}-1} < |a \cdot \mathbf{u}_0[0] + \mathbf{u}_0[1]| \leq 2^{r_{\max}-\mathbf{r}}$ 
6   return  $\mathbf{r}$  as the variable representing the approximation for the negative
   logarithm of the absolute value of Expression (11)

7 Procedure SDLModelV1Core(The model  $\mathcal{M}$ ,  $\Delta_x, \Delta_y, \lambda_z$ , the modular addition
   size  $n$ ):
8   Declare  $\mathbf{u}_0, \dots, \mathbf{u}_n$  and  $\mathbf{v}_0, \dots, \mathbf{v}_{n-1}$  as vectors of two integer variables
9    $\mathcal{M}.\text{constraints} \leftarrow \mathbf{u}_n[0] = 0, \mathbf{u}_n[1] = 2^{r_{\max}} \triangleright r_{\max}$  is a customizable
   constant that prevents precision overflow. In practice, we typically set
    $r_{\max}$  between 10 and 28, where larger values extend the search range of
    $\Delta_x, \Delta_y, \lambda_z$  at the cost of increased solving time.
10  for  $i$  from  $n-1$  down to 0 do
11    Add the following constraints to  $\mathcal{M}$ :
12    if  $\Delta_x[i] \neq \Delta_y[i]$  then
13       $\mathbf{v}_i[0] = 0$  and  $\mathbf{v}_i[1] = \mathbf{u}_{i+1}[1]$ 
14    else
15       $\mathbf{v}_i[0] = \mathbf{u}_{i+1}[0]/2$  and
16      if  $\Delta_x[i] = 0$  then
17         $\mathbf{v}_i[1] = \mathbf{u}_{i+1}[0]/2 + \mathbf{u}_{i+1}[1]$ 
18      else
19         $\mathbf{v}_i[1] = -\mathbf{u}_{i+1}[0]/2 + \mathbf{u}_{i+1}[1]$ 
20      endif
21    endif
22  for  $i$  from  $n-1$  down to 0 do
23    Add the following constraints to  $\mathcal{M}$ :
24    if  $\lambda_z[i] = 0$  then
25       $\mathbf{u}_i[0] = \mathbf{v}_i[0]$  and  $\mathbf{u}_i[1] = \mathbf{v}_i[1]$ 
26    else
27       $\mathbf{u}_i[0] = \mathbf{v}_i[1]$  and  $\mathbf{u}_i[1] = \mathbf{v}_i[0]$ 
28    endif
29  return  $\mathbf{u}_0$ 

```

- If $\Delta_x[i] \neq \Delta_y[i]$, a factor boundary occurs between positions i and $i+1$. In this case, $\mathbf{w}[i+1]$ is constrained to approximate the negative logarithm of the absolute value of the factor containing position $i+1$, and $[\mathbf{v}_i[0] \ \mathbf{v}_i[1]]$ is reset to $2^{r_{\max}} \begin{bmatrix} 0 & 1 \end{bmatrix}$ $C_{\Delta_x[i], \Delta_y[i]} = \begin{bmatrix} 0 & 2^{r_{\max}} \end{bmatrix}$, starting a new factor with a fresh scaling factor.

Algorithm 2: Automated Model (Version 2) for Characterizing the Negative Logarithm (Base 2) of the absolute value of Expression (11)

```

1 Procedure SDLModelV2(The model  $\mathcal{M}$ ,  $\Delta_x, \Delta_y, \lambda_z$ , constant  $a$ , the modular
   addition size  $n$ ):
2   Declare  $\mathbf{r}$  as an integer variable
3    $\mathbf{u}_0, \mathbf{w} = \text{SDLModelV2Core}(\mathcal{M}, \Delta_x, \Delta_y, \lambda_z, n)$ 
4    $\mathcal{M}.\text{constraints} \leftarrow 2^{r_{\max} - \mathbf{w}[0] - 1} < |a \cdot \mathbf{u}_0[0] + \mathbf{u}_0[1]| \leq 2^{r_{\max} - \mathbf{w}[0]}, \mathbf{r} =$ 
    $\sum_{i=0}^{n-1} \mathbf{w}[i]$ .
5   return  $\mathbf{r}$  representing the approximation for the negative logarithm of the
   absolute value of Expression (11)
6 Procedure SDLModelV2Core(The model  $\mathcal{M}$ ,  $\Delta_x, \Delta_y, \lambda_z$ ,  $n$ ):
7   Declare  $\mathbf{u}_0, \dots, \mathbf{u}_n$  and  $\mathbf{v}_0, \dots, \mathbf{v}_{n-1}$  as vectors of two integer variables
8   Declare  $\mathbf{w}$  as a vector of  $n$  integer variables over  $\{0, 1, \dots, r_{\max} - 1\}$ 
9    $\mathcal{M}.\text{constraints} \leftarrow \mathbf{u}_n[0] = 0, \mathbf{u}_n[1] = 2^{r_{\max}}$ 
10  for  $i = n - 1$  do Add the constraints in Lines 11–21 of Algorithm 1 to  $\mathcal{M}$ ;
11  for  $i$  from  $n - 2$  down to 0 do
12    Add the following constraints to  $\mathcal{M}$ :
13    if  $\Delta_x[i] \neq \Delta_y[i]$  then
14       $\mathbf{v}_i[0] = 0, \mathbf{v}_i[1] = 2^{r_{\max}}, 2^{r_{\max} - \mathbf{w}[i+1] - 1} < |\mathbf{u}_{i+1}[1]| \leq 2^{r_{\max} - \mathbf{w}[i+1]}$ 
15    else
16       $\mathbf{v}_i[0] = \mathbf{u}_{i+1}[0]/2, \mathbf{w}[i+1] = 0$ , and
17      if  $\Delta_x[i] = 0$  then
18         $\mathbf{v}_i[1] = \mathbf{u}_{i+1}[0]/2 + \mathbf{u}_{i+1}[1]$ 
19      else
20         $\mathbf{v}_i[1] = -\mathbf{u}_{i+1}[0]/2 + \mathbf{u}_{i+1}[1]$ 
21      endif
22    endif
23  for  $i$  from  $n - 1$  down to 0 do
24    Add the constraints in Lines 23–28 of Algorithm 1 to  $\mathcal{M}$ 
25  return  $\mathbf{u}_0, \mathbf{w}$ 

```

- If $\Delta_x[i] = \Delta_y[i]$, positions i and $i + 1$ belong to the same factor. $\mathbf{w}[i + 1]$ is constrained to 0, and the constraints between $\mathbf{v}_i, \mathbf{u}_i$ and $\mathbf{u}_{i+1}$ are as in Algorithm 1.

Finally, $\mathbf{w}[0]$ is constrained to be the approximation for $-\log_2 |p_0|$, and the overall approximation is $\mathbf{r} = \sum_{i=0}^{n-1} \mathbf{w}[i]$.

Since each factor is processed with its own scaling factor $2^{r_{\max}}$, the non-integer issue is confined to each individual factor than the entire Expression (11). This allows us to use a much smaller $r_{\max}$ (typically 10–28 in practice) while maintaining a large search space. Indeed, all intermediate values are guaranteed to be integers, as long as the number of bit positions in each factor does not exceed $r_{\max} + 1$. The trade-off is an error of at most $m + 1$ (the number of factors), compared to at most 1 for Algorithm 1. In practice, the error of Algorithm 2

is typically small (less than 2 in our experiments with $n = 64$). A comparison between the two algorithms is provided in Appendix C (see Example 2).

B.2 CP Model for RDL Correlation

Algorithm 3: Automated Model for Approximating the Negative Logarithm of the Absolute RDL Correlation of Modular Addition When t and $n - t$ are Large

```

1 Procedure RDLModel(The model  $\mathcal{M}$ ,  $\Delta_x, \Delta_y, \lambda_z$ , rotational offset  $t$ , the
   modular addition size  $n$ ):
2   Declare  $\mathbf{r}$  as an integer variable
3    $\mathbf{p}' = \text{SDLModelV1/V2}(\mathcal{M}, \lceil \Delta_x \rceil^{n-t}, \lceil \Delta_y \rceil^{n-t}, \lceil \lambda_z \rceil^{n-t}, 0, n-t)$ 
4    $\mathbf{q}' = \text{SDLModelV1/V2}(\mathcal{M}, \lfloor \Delta_x \rfloor^t, \lfloor \Delta_y \rfloor^t, \lfloor \lambda_z \rfloor^t, 0, t)$ 
5    $\triangleright$  See Algorithm 1 and 2 for the definition of SDLModelV1 and SDLModelV2
6    $\mathcal{M}.\text{constraints} \leftarrow \mathbf{r} = \mathbf{p}' + \mathbf{q}'$ 
7   return  $\mathbf{r}$  as the variable representing the approximation for
    $-\log_2 |\mathcal{R}_{\boxplus_n^{(b)}}^{\text{RDL}}((\Delta_x, \Delta_y), \lambda_z)|$ 

```

C Examples

Example 1 (Example for Proposition 7). Consider computing Expression (11) for an $n = 8$ bit modular addition with:

$$\begin{aligned}\Delta_x &= (0, 0, 1, 0, 0, 0, 1, 0), \\ \Delta_y &= (0, 0, 0, 0, 0, 0, 0, 0), \\ \lambda_z &= (0, 1, 0, 0, 1, 0, 0, 0),\end{aligned}$$

and $a = 1$. Since $\Delta_x[i] \neq \Delta_y[i]$ at positions $j_0 = 2$ and $j_1 = 6$, we have $m = 2$ split points with $j_2 = j_m = n - 1 = 7$.

By Proposition 7, Expression (11) factors as $p_2 p_1 p_0$, where

$$\begin{aligned}
p_0 &= [0 \ 1] \prod_{i=j_0}^0 (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} a \\ 1 \end{bmatrix} \\
&= [0 \ 1] \cdot C_{1,0} B_0 \cdot C_{0,0} B_1 \cdot C_{0,0} B_0 \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 1. \\
p_1 &= [0 \ 1] \prod_{i=j_1}^{j_0+1} (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\
&= [0 \ 1] \cdot C_{1,0} B_0 \cdot C_{0,0} B_0 \cdot C_{0,0} B_1 \cdot C_{0,0} B_0 \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{1}{2}. \\
p_2 &= [0 \ 1] \prod_{i=j_2}^{j_1+1} (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} 0 \\ 1 \end{bmatrix} \\
&= [0 \ 1] \cdot C_{0,0} B_0 \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 1.
\end{aligned}$$

Therefore, Expression (11) evaluates to:

$$p_2 p_1 p_0 = 1 \times \frac{1}{2} \times 1 = \frac{1}{2}.$$

Example 2 (Example for comparison between Algorithm 1 and 2). Consider $n = 32, a = 1$ with

$$\begin{cases} \Delta_x = (0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0) \\ \Delta_y = (0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1) \\ \lambda_z = (0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1) \end{cases}.$$

Setting $r_{\max} = 20$, if we temporarily ignore constraint that the elements of $\mathbf{u}$ and $\mathbf{v}$ in the procedure `SDLModelV1Core` must be integers, then theoretically they would be computed as shown in Table 4.

However, since $\mathbf{u}_0$ is declared as a vector of integer variables, the values Δ_x , Δ_y and λ_z in this example lie outside the search space of Algorithm 1.

In contrast, when inputting Δ_x , Δ_y , λ_z , and a to Algorithm 2, the vectors $\mathbf{u}$, $\mathbf{v}$, and $\mathbf{w}$ are computed as shown in Table 5. It can be seen that all elements of $\mathbf{u}$ and $\mathbf{v}$ are integers. The final approximation for the negative base-2 logarithm of the absolute value of Expression (11) is returned as $\sum_{i=0}^{n-1} \mathbf{w}_i = 16$, whereas the exact value is 17.0931.

Table 4: Theoretical values of $\mathbf{v}_0, \dots, \mathbf{v}_{n-1}$ and $\mathbf{u}_0, \dots, \mathbf{u}_n$ in Algorithm 1

i	$\mathbf{u}_i$	$\mathbf{v}_i$
0	(0.9375, 6.5625)	(0.9375, 6.5625)
1	(1.875, 5.625)	(1.875, 5.625)
2	(3.75, 3.75)	(3.75, 3.75)
3	(7.5, 0)	(0, 7.5)
4	(0, 7.5)	(0, 7.5)
5	(52.5, 7.5)	(7.5, 52.5)
6	(15, 45)	(45, 15)
7	(90, -30)	(-30, 90)
8	(-60, 60)	(60, -60)
9	(120, 0)	(0, 120)
10	(0, 120)	(0, 120)
11	(-40, 120)	(-40, 120)
12	(-80, 80)	(-80, 80)
13	(-160, 160)	(160, -160)
14	(320, 0)	(0, 320)
15	(192, 320)	(192, 320)
16	(384, 128)	(128, 384)
17	(256, 256)	(256, 256)
18	(512, 512)	(512, 512)
19	(1024, 1024)	(1024, 1024)
20	(2048, 2048)	(2048, 2048)
21	(4096, 4096)	(4096, 4096)
22	(8192, 8192)	(8192, 8192)
23	(16384, 16384)	(16384, 16384)
24	(32768, 0)	(0, 32768)
25	(0, 32768)	(0, 32768)
26	(32768, 32768)	(32768, 32768)
27	(65536, 65536)	(65536, 65536)
28	(131072, 131072)	(131072, 131072)
29	(262144, 262144)	(262144, 262144)
30	(524288, 524288)	(524288, 524288)
31	(1048576, 0)	(0, 1048576)
32	(0, 1048576)	—

Table 5: Values of $\mathbf{v}_0, \dots, \mathbf{v}_{n-1}$, $\mathbf{u}_0, \dots, \mathbf{u}_n$ and $\mathbf{w}_0, \dots, \mathbf{w}_{n-1}$ in Algorithm 2

i	$\mathbf{u}_i$	$\mathbf{v}_i$	$\mathbf{w}_i$
0	(131072, 917504)	(131072, 917504)	0
1	(262144, 786432)	(262144, 786432)	0
2	(524288, 524288)	(524288, 524288)	0
3	(1048576, 0)	(0, 1048576)	0
4	(0, 1048576)	(0, 1048576)	0
5	(458752, 65536)	(65536, 458752)	4
6	(131072, 393216)	(393216, 131072)	0
7	(786432, -262144)	(-262144, 786432)	0
8	(-524288, 524288)	(524288, -524288)	0
9	(1048576, 0)	(0, 1048576)	0
10	(0, 1048576)	(0, 1048576)	0
11	(-131072, 393216)	(-131072, 393216)	1
12	(-262144, 262144)	(-262144, 262144)	0
13	(-524288, 524288)	(524288, -524288)	0
14	(1048576, 0)	(0, 1048576)	0
15	(6144, 10240)	(6144, 10240)	6
16	(12288, 4096)	(4096, 12288)	0
17	(8192, 8192)	(8192, 8192)	0
18	(16384, 16384)	(16384, 16384)	0
19	(32768, 32768)	(32768, 32768)	0
20	(65536, 65536)	(65536, 65536)	0
21	(131072, 131072)	(131072, 131072)	0
22	(262144, 262144)	(262144, 262144)	0
23	(524288, 524288)	(524288, 524288)	0
24	(1048576, 0)	(0, 1048576)	0
25	(0, 1048576)	(0, 1048576)	0
26	(32768, 32768)	(32768, 32768)	5
27	(65536, 65536)	(65536, 65536)	0
28	(131072, 131072)	(131072, 131072)	0
29	(262144, 262144)	(262144, 262144)	0
30	(524288, 524288)	(524288, 524288)	0
31	(1048576, 0)	(0, 1048576)	0
32	(0, 1048576)	—	—

D Estimate of the SDL Correlation for Multi-round E_M

According to Formula (1), we can express the SDL correlation of a vectorial Boolean function F as

$$\mathcal{R}_F^{\text{SDL}}(\Delta, \lambda) = \sum_{\alpha} \mathcal{P}_F(\Delta, \alpha) (-1)^{\langle \alpha, \lambda \rangle} = \sum_{\beta} \mathcal{C}_F^2(\beta, \lambda) (-1)^{\langle \Delta, \beta \rangle}.$$

Hence, for a fixed input standard difference Δ , if the standard difference of a specific output bit of F is deterministically 0, then the value of λ at that bit position does not influence $\mathcal{R}_F^{\text{SDL}}(\Delta, \lambda)$; conversely, for a fixed output mask λ , if the linear mask of a specific input bit is deterministically 0, then the value of Δ at that bit position does not influence $\mathcal{R}_F^{\text{SDL}}(\Delta, \lambda)$ as well.

Efficient Computation of SDL Correlation [46]. Now let us focus on computing the SDL correlation $\mathcal{R}_{E_M}^{\text{SDL}}(\Delta_m, \lambda_m)$ for a fixed input standard difference Δ_m and a fixed output linear mask λ_m . We decompose E_M as $E_M = E_{ML} \circ E_{MU}$, with y representing the input state of E_{ML} . According to Formula (1),

$$\mathcal{R}_{E_M}^{\text{SDL}}(\Delta_m, \lambda_m) = \sum_{\alpha} \mathcal{P}_{E_{MU}}(\Delta_m, \alpha) \cdot \mathcal{R}_{E_{ML}}^{\text{SDL}}(\alpha, \lambda_m).$$

For ease of demonstration, suppose that the linear mask on y is always of the form $0^s \| *^w$ for the fixed output linear mask λ_m , where $*$ denotes an arbitrary and non-deterministic bit, then the first s standard difference bits of y will not affect the SDL correlation of E_{ML} . Hence, we can rewrite $\mathcal{R}_{E_M}^{\text{SDL}}(\Delta_m, \lambda_m)$ as

$$\mathcal{R}_{E_M}^{\text{SDL}}(\Delta_m, \lambda_m) = \sum_{\alpha \in \mathbb{F}_2^w} \left(\sum_{\beta \in \mathbb{F}_2^s} \mathcal{P}_{E_{MU}}(\Delta_m, \beta \| \alpha) \right) \cdot \mathcal{R}_{E_{ML}}^{\text{SDL}}(0^s \| \alpha, \lambda_m).$$

That is to say, we can compute $\mathcal{R}_{E_M}^{\text{SDL}}(\Delta_m, \lambda_m)$ by precomputing the truncated standard differential probability $\sum_{\beta \in \mathbb{F}_2^s} \mathcal{P}_{E_{MU}}(\Delta_m, \beta \| \alpha)$ and then computing $\mathcal{R}_{E_{ML}}^{\text{SDL}}(0^s \| \alpha, \lambda_m)$ for each $\alpha \in \mathbb{F}_2^w$. The precomputation is feasible when E_{MU} only contains one S-box layer. Thus, this essentially reduces the problem of computing $\mathcal{R}_{E_M}^{\text{SDL}}(\Delta_m, \lambda_m)$ to evaluating 2^w SDL correlations of E_{ML} , which are simpler to handle. This reduction can be applied iteratively (round by round) until the remaining sub-problems become computationally trivial.

Determinism Propagation of Standard Differences and Linear Masks.

We next use two small examples to illustrate how Hadipour et al. and Peng et al. [46, 25] determine which bits of y have deterministically zero standard differences (resp. linear masks) for a fixed input standard difference Δ_m (resp. output linear mask λ_m).

Example 3 (Propagating determinism of standard differences). Let $E_{MU} = S \circ L \circ S$, where S is Xoodoo's 3-bit S-box [20] and $L(x) = (x[0], x[0] \oplus x[1], x[0] \oplus x[1] \oplus x[2])$. Consider $\Delta_m = (1, 0, 0)$.

From the SDDT of Xoodoo's S-box (Table 6), one can derive these rules:

- If the input standard difference is deterministically $(0,0,0)$, the output is deterministically $(0,0,0)$.
- If exactly one bit of the input standard difference is deterministically 1 and the other two are deterministically 0, then that same bit of the output is deterministically 1, but the other two becomes non-deterministically $(*)$.
- Otherwise, all output bits are non-deterministic.

For XOR: if any input bit is non-deterministic, the output is non-deterministic; otherwise, the output is the XOR of the deterministic input values.

Tracing through: $\Delta_m = (1,0,0) \xrightarrow{S} (1,*,*) \xrightarrow{L} (1,*,*) \xrightarrow{S} (*,*,*)$. So no standard difference bit of y has a deterministic difference for the choice of Δ_m .

Modeling the propagation of determinism of standard differences through an S-box and an XOR operation is feasible. In [25], 0, 1 and -1 are used to represent whether the standard difference or the linear mask of a bit is deterministically 0, 1 or non-deterministic. For Xoodoo's S-box, if we use two vectors $\mathbf{X}, \mathbf{Y}$ of three variables over $\{0, 1, -1\}$ to represent the determinism of the input and output standard differences, respectively, then the determinism propagation from the input standard difference to the output one can be characterized by the following constraints:

$$\begin{cases} \text{if } \mathbf{X} = (0, 0, 0) & \text{then } \mathbf{Y} = (0, 0, 0), \\ \text{else if } \mathbf{X} = (1, 0, 0) & \text{then } \mathbf{Y} = (1, -1, -1), \\ \text{else if } \mathbf{X} = (0, 1, 0) & \text{then } \mathbf{Y} = (-1, 1, -1), \\ \text{else if } \mathbf{X} = (0, 0, 1) & \text{then } \mathbf{Y} = (-1, -1, 1), \\ \text{else} & \mathbf{Y} = (-1, -1, -1). \end{cases}$$

For an n -input XOR operation, if we use a vector $\mathbf{X}$ of n variables over $\{0, 1, -1\}$ to represent the determinism of the input standard difference and a variable $\mathbf{Y}$ to represent the determinism of the output standard difference, then the determinism propagation through XOR can be characterized as:

$$\begin{cases} \text{if } \exists i \in \{0, 1, n-1\} \text{ s.t. } \mathbf{X}[i] = -1 & \text{then } \mathbf{Y} = -1, \\ \text{else} & \mathbf{Y} = \bigoplus_{i=0}^{n-1} \mathbf{X}[i], \end{cases}$$

where the existence quantifier is naturally supported by modern CP solvers.

Example 4 (Propagating determinism of linear masks). Let $E_{MU} = S \circ L \circ S$, where S is the parallel application of two Xoodoo's S-boxes and $L(x) = (x[0] \oplus x[3], x[1], x[2], x[3], x[4], x[5])$. Consider $\lambda_m = (1, 0, 0, 0, 0, 0)$.

From the LAT of Xoodoo's S-box (Table 7), one can derive the determinism propagation rule of linear masks through the inverse of the S-box. That is, if the output linear mask of the S-box is deterministically $(0, 0, 0)$, then the input linear mask is deterministically $(0, 0, 0)$ as well; if exactly one bit of the output linear mask is deterministically 1 with the other two being deterministically 0, then that same bit of the input mask is deterministically 1, but the other two becomes non-deterministically $(*)$; otherwise, all output bits are non-deterministic. Regarding the linear layer, we observe that the input mask bits can always be expressed

Table 6: SDDT of Xoodoo’s S-box

Input difference	Output difference							
	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7
0x0	8	0	0	0	0	0	0	0
0x1	0	2	0	2	0	2	0	2
0x2	0	0	2	2	0	0	2	2
0x3	0	2	2	0	0	2	2	0
0x4	0	0	0	0	2	2	2	2
0x5	0	2	0	2	2	0	2	0
0x6	0	0	2	2	2	2	0	0
0x7	0	2	2	0	2	0	0	2

Table 7: LAT of Xoodoo’s S-box

Input mask	Output mask							
	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7
0x0	8	0	0	0	0	0	0	0
0x1	0	4	0	-4	0	4	0	4
0x2	0	0	4	4	0	0	-4	4
0x3	0	-4	4	0	0	4	4	0
0x4	0	0	0	0	4	-4	4	4
0x5	0	4	0	4	-4	0	4	0
0x6	0	0	-4	4	4	4	0	0
0x7	0	4	4	0	4	0	0	-4

as an XOR sum of the output mask bits. Consequently, we can apply the same propagation rules for XOR operations as those used for standard differences to track mask determinism through the inverse of linear layer.

Tracing backward:

$$\lambda_m = (1, 0, 0, 0, 0, 0) \xrightarrow{S^{-1}} (1, *, *, 0, 0, 0) \xrightarrow{L^{-1}} (1, *, *, 1, 0, 0) \xrightarrow{S^{-1}} (*, *, *, 1, *, *).$$

Thus only the mask on $y[3]$ is deterministically 1; all other masks are non-deterministic.

Modeling the determinism propagation of linear masks is analogous to that for standard differences (Example 3).

CP Model for SDL Distinguisher Search [25]. From the analysis above, it is intuitive that the more state bits in E_M with a zero standard difference or a zero linear mask, the lower the complexity of calculating the SDL correlation of E_M . Thus, Hadipour et al. developed a constraint programming (CP) model for the search of potentially effective SDL distinguishers on bit-based primitives.

In this model, the primitive E is decomposed into E_U, E_M and E_L , where the standard differential probability of E_U , the linear correlation of E_L , and the

propagation of determinism of standard differences and linear masks within E_M is characterized. The details of the CP model are provided in Algorithm 4, where we use 0, 1 and -1 to represent the standard difference or the linear mask of a bit is deterministically 0, 1 or non-deterministic. The CP solver is configured to output all the found solutions during the optimization. For each solution with a sufficiently small objective value, we read the values of $\mathbf{x}, \mathbf{y}, \Delta_{\mathbf{i}}, \Delta_{\mathbf{m}}, \lambda_{\mathbf{m}}, \lambda_{\mathbf{o}}$ as $p, q, \Delta_i, \Delta_m, \lambda_m, \lambda_o$. Then, we compute $\mathcal{R}_{E_M}^{\text{SDL}}(\Delta_m, \lambda_m)$ as r using the method mentioned earlier, and estimate $|\mathcal{R}_E^{\text{SDL}}(\Delta_i, \lambda_o)|$ as $|prq^2|$. If this estimated value is large enough, it identifies an effective distinguisher of E .

Algorithm 4: Hadipour et al.'s Bit-wise CP Model for SDL Distinguisher Search

```

1 Procedure SLDistinguisherSearchModel(The target primitive
    $E = E_L \circ E_M \circ E_U$ ):
2   Let us say  $E_M = f^{r-1} \circ \dots \circ f^0$ , with the input and output states of
    $f^i, 0 \leq i < r$  being  $x^i$  and  $x^{i+1}$ , respectively
3   Declare an empty model  $\mathcal{M}$ 
4   Declare  $\Delta_{\mathbf{i}}, \Delta_{\mathbf{m}}, \lambda_{\mathbf{m}}, \lambda_{\mathbf{o}}$  as vectors of binary variables
5   Declare  $\mathbf{v}^0, \dots, \mathbf{v}^r$  as vectors over  $\{0, 1, -1\}$ , where the  $j$ -th bit of  $\mathbf{v}^i$ 
   characterizes the determinism of the standard difference on  $x^i[j]$ .
6   Declare  $\mathbf{u}^0, \dots, \mathbf{u}^r$  as vectors over  $\{0, 1, -1\}$ , where the  $j$ -th bit of  $\mathbf{u}^i$ 
   characterizes the determinism of the linear mask on  $x^i[j]$ .
7   Declare  $\mathbf{x}, \mathbf{N}, \mathbf{y}$  as integer variables
8    $\mathcal{M}.\text{constraints} \leftarrow \mathbf{x} = -\log_2(\mathcal{P}_{E_U}(\Delta_{\mathbf{i}}, \Delta_{\mathbf{m}}))$   $\triangleright$  Typically, we can approximate
    $\mathcal{P}_{E_U}(\Delta_{\mathbf{i}}, \Delta_{\mathbf{m}})$  by the probability of a standard differential trail of  $E_U$  from
    $\Delta_{\mathbf{i}}$  to  $\Delta_{\mathbf{m}}$ 
9    $\mathcal{M}.\text{constraints} \leftarrow \mathbf{y} = -\log_2(|\mathcal{C}_{E_L}(\lambda_{\mathbf{m}}, \lambda_{\mathbf{o}})|)$   $\triangleright$  Typically, we can
   approximate  $\mathcal{C}_{E_L}(\lambda_{\mathbf{m}}, \lambda_{\mathbf{o}})$  by the correlation of a linear trail of  $E_L$  from  $\lambda_{\mathbf{m}}$ 
   to  $\lambda_{\mathbf{o}}$ 
10   $\mathcal{M}.\text{constraints} \leftarrow \Delta_{\mathbf{m}} = \mathbf{v}^0, \lambda_{\mathbf{m}} = \mathbf{u}^r$ 
11  for  $i$  from 0 to  $r-1$  do
12    Add constraints between  $\mathbf{v}^i$  to  $\mathbf{v}^{i+1}$  to encode the determinism
    propagation of standard differences through  $f^i$ 
13  for  $i$  from  $r$  to 1 do
14    Add constraints between  $\mathbf{u}^i$  to  $\mathbf{u}^{i-1}$  to encode the determinism
    propagation of linear masks through the inverse of  $f^{i-1}$ 
15   $\mathcal{M}.\text{constraints} \leftarrow \mathbf{N} =$ 
    $\sum_{i=1}^{r-1} \sum_j \text{bool2int}((\mathbf{v}^i[j] \neq 0) \wedge (\mathbf{u}^i[j] \neq 0) \wedge ((\mathbf{v}^i[j] = -1) \vee (\mathbf{u}^i[j] = -1)))$ 
    $\triangleright \text{bool2int}(\text{true}) = 1$  and  $\text{bool2int}(\text{false}) = 0$ 
16  Choose three integer constants  $w_U, w_M$  and  $w_L$  as the weights for  $\mathbf{x}, \mathbf{N}$  and
    $\mathbf{y}$ , respectively
17   $\mathcal{M}.\text{objective} \leftarrow \min(w_U \mathbf{x} + w_M \mathbf{N} + 2w_L \mathbf{y})$ 
18  Optimize  $\mathcal{M}$ 

```

E Proofs for Deriving Walsh Coefficients of the DTF

E.1 Proof for Proposition 2

Proof. Recall that $\partial_F(x, \Delta) = F(x) \oplus \text{rot}(F(\text{rot}^{-1}(x \oplus \Delta)))$ in RDL analysis. By the definition of Walsh transform, we have

$$(-1)^{\langle \lambda, F(x) \rangle} = \sum_{\alpha} \mathcal{C}_F(\alpha, \lambda) \cdot (-1)^{\langle \alpha, x \rangle}.$$

Substituting λ and x with $\text{rot}^{-1}(\lambda)$ and $\text{rot}^{-1}(x \oplus \Delta)$ respectively into the above equation, we obtain

$$\begin{aligned} (-1)^{\langle \text{rot}^{-1}(\lambda), F(\text{rot}^{-1}(x \oplus \Delta)) \rangle} &= \sum_{\beta} \mathcal{C}_F(\beta, \text{rot}^{-1}(\lambda)) \cdot (-1)^{\langle \beta, \text{rot}^{-1}(x \oplus \Delta) \rangle} \\ &= \sum_{\beta} \mathcal{C}_F(\beta, \text{rot}^{-1}(\lambda)) \cdot (-1)^{\langle \text{rot}(\beta), x \oplus \Delta \rangle} \\ &= \sum_{\beta} \mathcal{C}_F(\text{rot}^{-1}(\beta), \text{rot}^{-1}(\lambda)) \cdot (-1)^{\langle \beta, x \oplus \Delta \rangle}. \end{aligned}$$

Since $\langle \text{rot}^{-1}(\lambda), F(\text{rot}^{-1}(x \oplus \Delta)) \rangle = \langle \lambda, \text{rot}(F(\text{rot}^{-1}(x \oplus \Delta))) \rangle$, multiplying the above two equations yields

$$(-1)^{\langle \lambda, \partial_F(x, \Delta) \rangle} = \sum_{\alpha, \beta} \mathcal{C}_F(\alpha, \lambda) \cdot \mathcal{C}_F(\text{rot}^{-1}(\beta), \text{rot}^{-1}(\lambda)) \cdot (-1)^{\langle \alpha \oplus \beta, x \oplus \Delta \rangle}.$$

Thus, $\mathcal{C}_F(\alpha, \lambda) \cdot \mathcal{C}_F(\text{rot}^{-1}(\beta), \text{rot}^{-1}(\lambda))$ for any α, β and λ is exactly the Walsh coefficient $\mathcal{C}_{\partial_F}((\alpha \oplus \beta, \beta), \lambda)$ of ∂_F in RDL analysis. $\square$

E.2 Proof for Proposition 3

Proof. In IDL analysis, the DTF of F is defined as $\partial_F(x, \Delta) = \mathcal{L}F(y)$, where $y = (\Delta[0]_m \oplus x) \parallel \cdots \parallel (\Delta[\frac{n}{m}-2]_m \oplus x) \parallel x$. By the definition of Walsh transform, we have

$$(-1)^{\langle \mathcal{L}^\top \lambda, F(y) \rangle} = \sum_{\alpha \in \mathbb{F}_2^n} \mathcal{C}_F(\alpha, \mathcal{L}^\top \lambda) \cdot (-1)^{\langle \alpha, y \rangle}.$$

Substituting $y = (\Delta[0]_m \oplus x) \parallel \cdots \parallel (\Delta[\frac{n}{m}-2]_m \oplus x) \parallel x$ into the equation gives

$$\begin{aligned} (-1)^{\langle \mathcal{L}^\top \lambda, F(y) \rangle} &= (-1)^{\langle \lambda, \partial_F(x, \Delta) \rangle} \\ &= \sum_{\alpha \in \mathbb{F}_2^n} \mathcal{C}_F(\alpha, \mathcal{L}^\top \lambda) \cdot (-1)^{\langle \sum_{i=0}^{\frac{n}{m}-1} \alpha[i]_m, x \rangle \oplus \langle \alpha[0]_m \parallel \cdots \parallel \alpha[\frac{n}{m}-2]_m, \Delta \rangle}, \end{aligned}$$

thus proving the proposition. $\square$

F Proofs for Deriving Forward and Backward Reductions

F.1 Proof for Proposition 4

Proof. By Lemma 3, we have

$$\mathcal{R}_{E_M}(\Delta_m, \lambda_m) = \sum_{\alpha \in \mathbb{F}_2^{s+t}, \beta \in \mathbb{F}_2^w} \mathcal{R}_{E_{MU}}(\Delta_m, \alpha \| \beta) \cdot \mathcal{C}_{\partial_{E_{ML}}}((0, \alpha \| \beta), \lambda_m).$$

Since the output difference of E_{MU} is always of the form $0^s \| 1^t \| *^w$ for the input standard difference Δ_m , we can rewrite each term $\mathcal{R}_{E_{MU}}(\Delta_m, \alpha \| \beta)$ in the above equation according to Lemma 1 as

$$\begin{aligned} \mathcal{R}_{E_{MU}}(\Delta_m, \alpha \| \beta) &= \sum_{\gamma \in \mathbb{F}_2^w} \mathcal{P}_{E_{MU}}(\Delta_m, 0^s \| 1^t \| \gamma) \cdot (-1)^{\langle \gamma, \beta \rangle \oplus \bigoplus_{i=s}^{s+t-1} \alpha[i]} \\ &= (-1)^{\bigoplus_{i=s}^{s+t-1} \alpha[i]} \cdot \sum_{\gamma \in \mathbb{F}_2^w} \mathcal{P}_{E_{MU}}(\Delta_m, 0^s \| 1^t \| \gamma) \cdot (-1)^{\langle \gamma, \beta \rangle} \\ &= (-1)^{\bigoplus_{i=s}^{s+t-1} \alpha[i]} \cdot \mathcal{R}_{E_{MU}}(\Delta_m, 0^{s+t} \| \beta) \end{aligned}$$

Substituting this into the initial equation yields:

$$\begin{aligned} \mathcal{R}_{E_M}(\Delta_m, \lambda_m) &= \sum_{\beta \in \mathbb{F}_2^w} \mathcal{R}_{E_{MU}}(\Delta_m, 0^{s+t} \| \beta) \cdot \sum_{\alpha \in \mathbb{F}_2^{s+t}} \mathcal{C}_{\partial_{E_{ML}}}((0, \alpha \| \beta), \lambda_m) (-1)^{\bigoplus_{i=s}^{s+t-1} \alpha[i]}, \end{aligned}$$

where the inner summation is exactly $\mathcal{TC}_{\partial_{E_{ML}}}(\beta, \lambda_m)$ claimed in the proposition. $\square$

F.2 Proof for Proposition 5

Proof. By Lemma 2, we have

$$\mathcal{R}_{E_M}(\Delta_m, \lambda_m) = \sum_{\alpha \in \mathbb{F}_2^{s+t}, \beta \in \mathbb{F}_2^w} \mathcal{R}_{E_{ML}}(\alpha \| \beta, \lambda_m) \cdot \mathcal{P}_{E_{MU}}(\Delta_m, \alpha \| \beta).$$

Since the mask on input difference of E_{ML} is always of the form $0^s \| 1^t \| *^w$ for the mask λ_m on the output difference, we can rewrite each term $\mathcal{R}_{E_{ML}}(\alpha \| \beta, \lambda_m)$ in the above equation according to Corollary 1 as

$$\begin{aligned} \mathcal{R}_{E_{ML}}(\alpha \| \beta, \lambda_m) &= \sum_{\gamma \in \mathbb{F}_2^w} \mathcal{C}_{\partial_{E_{ML}}}((0, 0^s \| 1^t \| \gamma), \lambda_m) \cdot (-1)^{\langle \gamma, \beta \rangle \oplus \bigoplus_{i=s}^{s+t-1} \alpha[i]} \\ &= (-1)^{\bigoplus_{i=s}^{s+t-1} \alpha[i]} \cdot \sum_{\gamma \in \mathbb{F}_2^w} \mathcal{C}_{\partial_{E_{ML}}}((0, 0^s \| 1^t \| \gamma), \lambda_m) \cdot (-1)^{\langle \gamma, \beta \rangle} \\ &= (-1)^{\bigoplus_{i=s}^{s+t-1} \alpha[i]} \cdot \mathcal{R}_{E_{ML}}(0^{s+t} \| \beta, \lambda_m) \end{aligned}$$

Substituting this into the initial equation yields:

$$\mathcal{R}_{E_M}(\Delta_m, \lambda_m) = \sum_{\beta \in \mathbb{F}_2^{s+t}} \mathcal{R}_{E_{ML}}(0^{s+t} \parallel \beta, \lambda_m) \cdot \sum_{\alpha \in \mathbb{F}_2^{s+t}} \mathcal{P}_{E_{MU}}(\Delta_m, \alpha \parallel \beta) (-1)^{\bigoplus_{i=s}^{s+t-1} \alpha[i]},$$

where the inner summation is exactly $\mathcal{TP}_{E_{MU}}(\Delta_m, \beta)$ claimed in the proposition. $\square$

G Proofs for Walsh Transform on Majority Function

G.1 Proof of Lemma 4

Proof. Since $c[i+1] = \text{maj}(x[i], y[i], c[i])$ and $c[i+1] \oplus \Delta_c[i+1] = \text{maj}(x[i] \oplus \Delta_x[i], y[i] \oplus \Delta_y[i], c[i] \oplus \Delta_c[i])$, the Walsh transform gives

$$\begin{aligned} (-1)^{c[i+1]} &= -\frac{1}{2}(-1)^{x[i] \oplus y[i] \oplus c[i]} + \frac{1}{2}(-1)^{x[i]} + \frac{1}{2}(-1)^{y[i]} + \frac{1}{2}(-1)^{c[i]} \\ (-1)^{c[i+1] \oplus \Delta_c[i+1]} &= -\frac{1}{2}(-1)^{x[i] \oplus y[i] \oplus c[i] \oplus \Delta_x[i] \oplus \Delta_y[i] \oplus \Delta_c[i]} + \frac{1}{2}(-1)^{x[i] \oplus \Delta_x[i]} \\ &\quad + \frac{1}{2}(-1)^{y[i] \oplus \Delta_y[i]} + \frac{1}{2}(-1)^{c[i] \oplus \Delta_c[i]}. \end{aligned}$$

Multiplying the two equations above gives the lemma. $\square$

G.2 Proof of Lemma 5

Proof. Starting from Lemma 4, a term $\epsilon_{\alpha, \beta} (-1)^{\langle \alpha, x[i] \parallel y[i] \parallel c[i] \rangle \oplus \langle \beta, \Delta_x[i] \parallel \Delta_y[i] \parallel \Delta_c[i] \rangle}$ has a non-zero sum over $x[i], y[i]$ only if the first two bits of α are both 0 (otherwise the sum over $x[i], y[i]$ vanishes). For $\epsilon_{\alpha, \beta} \neq 0$, both $wt(\alpha \oplus \beta)$ and $wt(\beta)$ must be odd. With the first two bits of α forced to 0, this results in $\alpha = (0, 0, 0)$ and $\beta \in \{(1, 0, 0), (0, 1, 0), (0, 0, 1), (1, 1, 1)\}$. For all four choices, $\epsilon_{0, \beta} = \frac{1}{4}$, and summing over $x[i], y[i]$ gives a factor of 4, yielding the result. $\square$

H Proofs for Deriving SDL/RDL Correlations of Modular Addition

H.1 Proof for Lemma 7

Proof. Since $x[i]$ and $y[i]$ do not affect $\lfloor \Delta_c \rfloor^{i+1}$, we have

$$S_0^i = 4 \sum_{\lfloor x \rfloor^i, \lfloor y \rfloor^i} (-1)^{\langle \lfloor \lambda_z \rfloor^i, \lfloor \Delta_c \rfloor^i \rangle}. \quad (14)$$

Similarly,

$$S_1^i = 4 \sum_{\lfloor x \rfloor^i, \lfloor y \rfloor^i} (-1)^{\langle \lfloor \lambda_z \rfloor^i, \lfloor \Delta_c \rfloor^i \rangle \oplus \Delta_c[i]}.$$

Since $\lfloor \Delta_c \rfloor^i$ does not depend on $x[i-1], y[i-1]$, we can separate the sum over $x[i-1], y[i-1]$, namely

$$S_1^i = 4 \sum_{\lfloor x \rfloor^{i-1}, \lfloor y \rfloor^{i-1}} \left((-1)^{\langle \lfloor \lambda_z \rfloor^i, \lfloor \Delta_c \rfloor^i \rangle} \cdot \sum_{x[i-1], y[i-1]} (-1)^{\Delta_c[i]} \right).$$

By Lemma 5, the inner summation equals $((-1)^{\Delta_x[i-1] \oplus \Delta_y[i-1]} + 1) \cdot (-1)^{\Delta_c[i-1]} + (-1)^{\Delta_x[i-1]} + (-1)^{\Delta_y[i-1]}$. Substituting this into the RHS of above equation, rewriting the RHS as a summation over $\lfloor x \rfloor^i, \lfloor y \rfloor^i$ and then combining Eqn. (14), we obtain

$$\begin{bmatrix} S_1^i \\ S_0^i \end{bmatrix} = 2^2 C_{\Delta_x[i-1], \Delta_y[i-1]} \begin{bmatrix} \sum_{\lfloor x \rfloor^i, \lfloor y \rfloor^i} (-1)^{\langle \lfloor \lambda_z \rfloor^i, \lfloor \Delta_c \rfloor^i \rangle \oplus \Delta_c[i-1]} \\ \sum_{\lfloor x \rfloor^i, \lfloor y \rfloor^i} (-1)^{\langle \lfloor \lambda_z \rfloor^i, \lfloor \Delta_c \rfloor^i \rangle} \end{bmatrix}.$$

By Lemma 6 (applied with $i-1$ in place of i), the RHS of above equation is $2^2 C_{\Delta_x[i-1], \Delta_y[i-1]} B_{\lambda_z[i-1]} \begin{bmatrix} S_1^{i-1} \\ S_0^{i-1} \end{bmatrix}$, producing the claimed matrix recurrence. $\square$

H.2 Proof for Lemma 8

Proof. Treating $c'[n-t]$ as a constant, $\lfloor \Delta_z \rfloor^t$ can be expressed as $\lfloor \Delta_z \rfloor^t = \boxplus_t^{(b)}(\lfloor x \rfloor^t, \lfloor y \rfloor^t) \oplus \boxplus_t^{(c'[n-t])}(\lceil x' \rceil^t, \lceil y' \rceil^t)$ depending on $\lfloor x \rfloor^t$ and $\lfloor y \rfloor^t$. Since $\lfloor \Delta_x \rfloor^t = \lfloor x \rfloor^t \oplus \lceil x' \rceil^t$, $\lfloor \Delta_y \rfloor^t = \lfloor y \rfloor^t \oplus \lceil y' \rceil^t$ and $\lfloor \Delta_c \rfloor^t = \lfloor \Delta_x \rfloor^t \oplus \lfloor \Delta_y \rfloor^t \oplus \lfloor \Delta_z \rfloor^t$, we have

$$\mathcal{R}_{\boxplus_t^{(b, c'[n-t])}}^{\text{SDL}}((\lfloor \Delta_x \rfloor^t, \lfloor \Delta_y \rfloor^t), \lfloor \lambda_z \rfloor^t) = \frac{(-1)^{\langle \lfloor \lambda_z \rfloor^t, \lfloor \Delta_x \rfloor^t \oplus \lfloor \Delta_y \rfloor^t \rangle}}{2^{2t}} \sum_{\lfloor x \rfloor^t, \lfloor y \rfloor^t} (-1)^{\langle \lfloor \lambda_z \rfloor^t, \lfloor \Delta_c \rfloor^t \rangle}. \quad (15)$$

By Corollary 2, the LHS of above equation can be expressed as

$$(-1)^{\langle \lfloor \lambda_z \rfloor^t, \lfloor \Delta_x \rfloor^t \oplus \lfloor \Delta_y \rfloor^t \rangle} \begin{bmatrix} 0 & 1 \end{bmatrix} \prod_{i=t-1}^0 (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} (-1)^{\Delta_c[0]} \\ 1 \end{bmatrix}.$$

Substituting the above expression into Eqn. (15) and canceling the common factor $(-1)^{\langle \lfloor \lambda_z \rfloor^t, \lfloor \Delta_x \rfloor^t \oplus \lfloor \Delta_y \rfloor^t \rangle}$ of both sides yields

$$\frac{1}{2^{2t}} \sum_{\lfloor x \rfloor^t, \lfloor y \rfloor^t} (-1)^{\langle \lfloor \lambda_z \rfloor^t, \lfloor \Delta_c \rfloor^t \rangle} = \begin{bmatrix} 0 & 1 \end{bmatrix} \prod_{i=t-1}^0 (C_{\Delta_x[i], \Delta_y[i]} B_{\lambda_z[i]}) \begin{bmatrix} (-1)^{\Delta_c[0]} \\ 1 \end{bmatrix}.$$

Since $(-1)^{\langle \lfloor \lambda_z \rfloor^t, \lfloor \Delta_c \rfloor^t \rangle} = f_0(\lfloor x \rfloor^t, \lfloor y \rfloor^t) (-1)^{\Delta_c[0]} + f_1(\lfloor x \rfloor^t, \lfloor y \rfloor^t)$, and $\Delta_c[0] = b \oplus c'[n-t]$ is independent of $\lfloor x \rfloor^t, \lfloor y \rfloor^t$, matching the coefficients of $(-1)^{\Delta_c[0]}$ and the constant terms on both sides yields the first equation in the lemma. The second equation can be derived analogously. $\square$

H.3 Proof for Lemma 9

Proof. We prove the first equation; the second is analogous. Define auxiliary sequences U_i, V_i for $0 \leq i \leq n-t$ as follows:

$$U_i = \frac{1}{2^i} \sum_{\lfloor [x]^{n-t} \rfloor^i, \lfloor [y]^{n-t} \rfloor^i} (-1)^{c'[i] \oplus \langle \lfloor [\lambda_z]^{n-t} \rfloor^i, \lfloor [\Delta_c]^{n-t} \rfloor^i \rangle},$$

$$V_i = \frac{1}{2^i} \sum_{\lfloor [x]^{n-t} \rfloor^i, \lfloor [y]^{n-t} \rfloor^i} (-1)^{c[i+t] \oplus \langle \lfloor [\lambda_z]^{n-t} \rfloor^i, \lfloor [\Delta_c]^{n-t} \rfloor^i \rangle}.$$

We claim

$$\frac{1}{2^{2n-2t}} \sum_{\substack{[x]^{n-t} \\ [y]^{n-t}}} (-1)^{c'[n-t] \oplus \langle [\lambda_z]^{n-t}, [\Delta_c]^{n-t} \rangle} = \begin{cases} \frac{(-1)^{c[t]}}{2^{n-t}}, & \text{if } wt([\lambda_z]^{n-t}) \text{ is odd,} \\ \frac{(-1)^b}{2^{n-t}}, & \text{otherwise.} \end{cases} \quad (16)$$

The left-hand side of above equation is exactly $\frac{1}{2^{n-t}} U_{n-t}$. For $i \geq 1$, since $c'[i]$ is dependent on $\lfloor [x]^{n-t} \rfloor^i$ and $\lfloor [y]^{n-t} \rfloor^i$ while $\lfloor [\Delta_c]^{n-t} \rfloor^i$ is dependent on $\lfloor [x]^{n-t} \rfloor^{i-1}$ and $\lfloor [y]^{n-t} \rfloor^{i-1}$, we can rewrite U_i as

$$U_i = \frac{1}{2^i} \sum_{\substack{\lfloor [x]^{n-t} \rfloor^{i-1} \\ \lfloor [y]^{n-t} \rfloor^{i-1}}} \left(\sum_{x[t+i-1], y[t+i-1]} (-1)^{c'[i]} \right) \cdot (-1)^{\langle \lfloor [\lambda_z]^{n-t} \rfloor^i, \lfloor [\Delta_c]^{n-t} \rfloor^i \rangle}.$$

Expanding $(-1)^{c'[i]}$ as $-\frac{1}{2}(-1)^{x'[i-1] \oplus y'[i-1] \oplus c'[i-1]} + \frac{1}{2}(-1)^{x'[i-1]} + \frac{1}{2}(-1)^{y'[i-1]} + \frac{1}{2}(-1)^{c'[i-1]}$ according to the Walsh transform and summing it over $x[t+i-1], y[t+i-1]$, we obtain

$$U_i = \frac{1}{2^{i-1}} \sum_{\lfloor [x]^{n-t} \rfloor^{i-1}, \lfloor [y]^{n-t} \rfloor^{i-1}} (-1)^{\langle \lfloor [\lambda_z]^{n-t} \rfloor^i, \lfloor [\Delta_c]^{n-t} \rfloor^i \rangle \oplus c'[i-1]}.$$

Since $\Delta_c[t+i-1] = c[t+i-1] \oplus c'[i-1]$, we have $U_i = V_{i-1}$ and similarly $V_i = U_{i-1}$ if $\lambda_z[t+i-1] = 1$, and otherwise $U_i = U_{i-1}, V_i = V_{i-1}$. The base cases are $U_0 = (-1)^b, V_0 = (-1)^{c[t]}$. Solving the recurrence yields $U_{n-t} = (-1)^b$ if $wt([\lambda_z]^{n-t})$ is even, otherwise $U_{n-t} = (-1)^{c[t]}$. This proves Eqn. (16). Now, since $\Delta_c[0] = b \oplus c'[n-t]$, $(-1)^{\langle [\lambda_z]^{n-t}, [\Delta_c]^{n-t} \rangle} = g_0([x]^{n-t}, [y]^{n-t})(-1)^{\Delta_c[t]} + g_1([x]^{n-t}, [y]^{n-t})$ and $\Delta_c[t] = c[t] \oplus b$, the LHS of Eqn. (16) can be rewritten as

$$\frac{(-1)^b}{2^{2n-2t}} \left(\sum_{[x]^{n-t}, [y]^{n-t}} (-1)^{\Delta_c[0]} [g_0 \ g_1] \right) \begin{bmatrix} (-1)^{c[t] \oplus b} \\ 1 \end{bmatrix}.$$

Compare the coefficients of $(-1)^{c[t]}$ and the constant terms on both sides of Eqn. (16) yields the first equation of the lemma. $\square$

H.4 Proof for Proposition 8

Proof. Starting from Equation (12), $\mathcal{R}_{\boxplus_n^{(b)}}^{\text{RDL}}((\Delta_x, \Delta_y), \lambda_z)$ can be expressed as

$$\begin{aligned} & \frac{(-1)^{\langle \lambda_z, \Delta_x \oplus \Delta_y \rangle}}{2^{2n}} \sum_{x,y} [f_0 \ f_1] \begin{bmatrix} (-1)^{\Delta_c[0]} \\ 1 \end{bmatrix} [g_0 \ g_1] \begin{bmatrix} (-1)^{\Delta_c[t]} \\ 1 \end{bmatrix} \\ &= \frac{(-1)^{\langle \lambda_z, \Delta_x \oplus \Delta_y \rangle}}{2^{2n}} \sum_{x,y} \left(f_0 g_0 (-1)^{\Delta_c[0] \oplus \Delta_c[t]} + f_0 g_1 (-1)^{\Delta_c[0]} + f_1 g_0 (-1)^{\Delta_c[t]} + f_1 g_1 \right). \end{aligned}$$

We evaluate each term by factoring the sum over $[x]^t, [y]^t$ and $[x]^{n-t}, [y]^{n-t}$.

For the $f_0 g_0 (-1)^{\Delta_c[0] \oplus \Delta_c[t]}$ term,

$$\frac{1}{2^{2n}} \sum_{x,y} f_0 g_0 (-1)^{\Delta_c[0] \oplus \Delta_c[t]} = \frac{1}{2^{2n}} \sum_{[x]^t, [y]^t} f_0 (-1)^{\Delta_c[t]} \cdot \sum_{[x]^{n-t}, [y]^{n-t}} g_0 (-1)^{\Delta_c[0]}.$$

Applying Lemma 9 to both sums yields α_{00} . For the $f_0 g_1 (-1)^{\Delta_c[0]}$ term,

$$\frac{1}{2^{2n}} \sum_{x,y} f_0 g_1 (-1)^{\Delta_c[0]} = \frac{1}{2^{2n}} \sum_{[x]^t, [y]^t} f_0 \cdot \sum_{[x]^{n-t}, [y]^{n-t}} g_1 (-1)^{\Delta_c[0]}.$$

Apply Lemma 8 to the first sum and Lemma 9 to the second sum yields α_{10} . Similarly, for the term $f_1 g_0 (-1)^{\Delta_c[t]}$, $\frac{1}{2^{2n}} \sum_{x,y} f_1 g_0 (-1)^{\Delta_c[t]}$ yields α_{01} . For the last term $f_1 g_1$,

$$\frac{1}{2^{2n}} \sum_{x,y} f_1 g_1 = \frac{1}{2^{2n}} \sum_{[x]^t, [y]^t} f_1 \cdot \sum_{[x]^{n-t}, [y]^{n-t}} g_1.$$

Applying Lemma 8 yields pq . $\square$

I Validity of Proposition 1 Under Various Key Models

The derivation of Proposition 1 relies solely on the differential independence assumption. We discuss when this assumption holds under different key models.

Fixed-key vs. key-averaged. When E is keyed, the proposition is conditional on Definition 3 in every key model. In the fixed-key model there is no averaging, and differential independence is a working assumption that has to be validated, e.g., experimentally. In the key-averaged case it may fail: correlations between the keys of E_U , E_M , and E_L can break the differential independence assumption under averaging. If, however, the keys of E_U , E_M , and E_L are independent and random, then for standard differences the three parts remain differentially independent in expectation over the keys, and the derivation carries through term by term, so the proposition continues to hold. For rotational and internal differences this is not the case, since the key addition itself shifts the difference by $k \oplus \text{rot}(k)$ or $\mathcal{L}k$ (see Remark 3). In practice, most ciphers are not genuine

Markov ciphers, because their key schedules induce dependencies among round keys. Existing SDL analyses therefore implicitly rely on a working assumption, analogous to the hypothesis of stochastic equivalence due to Lai et al. [32]: the key-averaged SDL correlation closely approximates the fixed-key SDL correlation for almost all keys. Whether this hypothesis is realistic in practical scenarios is a subject that requires further investigation.

Related-key setting. In the related-key setting (applicable to SDL and RDL analyses; for IDL, imposing an internal differential on the key reduces the analysis to a weak-key model), we treat both the key and its (related-key) difference as part of the input to ∂_E , so the proposition continues to hold.

J Extending Hadipour et al.’s CP Model to RDL/IDL Analysis

Note that Algorithm 4 (Hadipour et al.’s SDL model, Line 15) counts standard difference bits throughout E_M whose values and masks are non-zero (including the deterministically 1 case) simultaneously, while Algorithm 5 (our unified model, Line 15) counts (standard, rotational or internal) difference bits throughout E_M whose values and masks are non-deterministic (excluding the deterministically 1 case) simultaneously, because we have proved in Section 3.4 that difference bits with deterministically 1 values or deterministically 1 masks can be bypassed for free by backward and forward reductions.

Algorithm 5: Unified Bit-wise CP Model for SDL/RDL/IDL Distinguisher Search

```

1 Procedure UnifiedDLDistinctSearchModel(The target primitive
    $E = E_L \circ E_M \circ E_U$ ):
2   Let us say  $E_M = f^{r-1} \circ \dots \circ f^0$ , with the input and output states of
    $f^i, 0 \leq i < r$  being  $x^i$  and  $x^{i+1}$ , respectively
3   Declare an empty model  $\mathcal{M}$ 
4   Declare  $\Delta_i, \Delta_m, \lambda_m, \lambda_o$  as vectors of binary variables
5   Declare  $v^0, \dots, v^r$  as vectors over  $\{0, 1, -1\}$ , where  $v^i[j]$  characterizes the
   determinism of the  $j$ -th bit of the (standard, rotational or internal)
   difference at state  $x^i$ 
6   Declare  $u^0, \dots, u^r$  as vectors over  $\{0, 1, -1\}$ , where  $u^i[j]$  characterizes the
   determinism of the  $j$ -th bit of the mask on the (standard, rotational or
   internal) difference at state  $x^i$ .
7   Declare  $x, N, y$  as integer variables
8    $\mathcal{M}.\text{constraints} \leftarrow x = -\log_2(\mathcal{P}_{E_U}(\Delta_i, \Delta_m))$   $\triangleright$  Typically, we can approximate
    $\mathcal{P}_{E_U}(\Delta_i, \Delta_m)$  by the probability of a (standard, rotational or internal)
   differential trail of  $E_U$  from  $\Delta_i$  to  $\Delta_m$ 
9    $\mathcal{M}.\text{constraints} \leftarrow y = -\log_2(|\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o)|)$   $\triangleright$  Typically, we can
   approximate  $\mathcal{C}_{\partial_{E_L}}((0, \lambda_m), \lambda_o)$  by the correlations of linear trails of  $E_L$ 
   according to Proposition 2 and 3
10   $\mathcal{M}.\text{constraints} \leftarrow \Delta_m = v^0, \lambda_m = u^r$ 
11  for  $i$  from 0 to  $r-1$  do
12    Add constraints between  $v^i$  to  $v^{i+1}$  to encode the determinism
    propagation of differences through  $f^i$ 
13  for  $i$  from  $r$  down to 1 do
14    Add constraints between  $u^i$  to  $u^{i-1}$  to encode the determinism
    propagation of masks on differences through the inverse of  $f^{i-1}$ 
15   $\mathcal{M}.\text{constraints} \leftarrow N = \sum_{i=1}^{r-1} \sum_j \text{bool2int}((v^i[j] = -1) \wedge (u^i[j] = -1))$   $\triangleright$ 
    $\text{bool2int}(\text{true}) = 1$  and  $\text{bool2int}(\text{false}) = 0$ 
16  Choose three integer constants  $w_U, w_M$  and  $w_L$  as the weights for  $x, N$  and
    $y$ , respectively
17   $\mathcal{M}.\text{objective} \leftarrow \min(w_U x + w_M N + w_L y)$ 
18  Optimize  $\mathcal{M}$ 

```

K Target Primitives

K.1 Xoodoo

Xoodoo [20] is a 384-bit lightweight cryptographic permutation, primarily designed for use in the Farfalle construction [11]. The state of Xoodoo is represented as a $4 \times 3 \times 32$ cuboid a , with each bit accessed as $a[x][y][z]$. The round

function of **Xoodoo** is defined as follows (the z -index is taken modulo 32):

$$\begin{aligned}
\theta : a[x][y][z] &= a[x][y][z] \oplus \sum_y a[x-1][y][z-5] \oplus \sum_y a[x-1][y][z-14], \\
\rho_w : a[x][1][z] &= a[x-1][1][z], \quad a[x][2][z] = a[x][2][z-11], \\
\iota : a[0][0] &= a[0][0] \oplus RC_i, \\
\chi : a[x][y][z] &= a[x][y][z] \oplus ((a[x][y+1][z] \oplus 1) \cdot a[x][y+2][z]), \\
\rho_e : a[x][1][z] &= a[x][1][z-1], \quad a[x][2][z] = a[x-2][2][z-8].
\end{aligned}$$

The total number of rounds for **Xoodoo** is 12, but in some modes (e.g., Farfalle [11]), the core permutation calls a 6-round **Xoodoo** permutation. For **Xoodoo** reduced to r rounds, the round constants used are $c_{-(r-1)}, \dots, c_0$ by default, where

$$\begin{aligned}
c_{-11} &= 00000058, & c_{-8} &= 000000D0, & c_{-5} &= 00000060, & c_{-2} &= 000000F0, \\
c_{-10} &= 00000038, & c_{-7} &= 00000120, & c_{-4} &= 0000002C, & c_{-1} &= 000001A0, \\
c_{-9} &= 000003C0, & c_{-6} &= 00000014, & c_{-3} &= 00000380, & c_0 &= 00000012.
\end{aligned}$$

5-Round RDL distinguisher. The rotational offset is set as 1. We found a 5-round (composed of $S \circ (L \circ S)^4$, where L is the linear layer, and S is the S-box layer) distinguisher with the round constants being $c_{-5}, \dots, c_{-1}$. The input rotational difference of this distinguisher is all-zero, and the mask on the output rotational difference is

$$\begin{aligned}
&00000000, 00000000, 04000000 \\
&00000000, 00000000, 00000000 \\
&00000000, 00000002, 00000000 \\
&00000000, 00000000, 00000000
\end{aligned}$$

The RDL correlation of this distinguisher is computed directly as 2^{-33} using the techniques in Section 3.4. Specifically, we decompose this 5-round RDL correlation into 4096 2-round RDL correlations through a sequence of one forward reduction, one backward reduction, and one further forward reduction; each resulting 2-round RDL correlation can then be evaluated efficiently. Although this correlation is too small to be verified experimentally, our techniques compute it within minutes. This strongly demonstrates the powerfulness and effectiveness of our approach, even in low-correlation regimes.

IDL analysis on Xoodoo. Consider an r -round primitive E operating on a $4 \times 3 \times 16$ cuboid a , whose round functions is defined as follows (the z -index is taken modulo 16):

$$\begin{aligned}
\hat{\theta} : a[x][y][z] &= a[x][y][z] \oplus \sum_y a[x-1][y][z-5] \oplus \sum_y a[x-1][y][z-14], \\
\hat{\rho}_w : a[x][1][z] &= a[x-1][1][z], \quad a[x][2][z] = a[x][2][z-11], \\
\hat{\iota} : a[0][0] &= a[0][0] \oplus K_i, \\
\hat{\chi} : a[x][y][z] &= a[x][y][z] \oplus ((a[x][y+1][z] \oplus 1) \cdot a[x][y+2][z]), \\
\hat{\rho}_e : a[x][1][z] &= a[x][1][z-1], \quad a[x][2][z] = a[x-2][2][z-8],
\end{aligned}$$

where K_i is the key at round i , $0 \leq i < r$. Then, the IDL correlation of r -round Xoodoo by XORing S-boxes at z -coordinates 0-15 with those at 16-31, with an input internal difference Δ and a output mask λ_m on the output internal difference, can be transformed into the SDL correlation of E under the related-key setting, where the input standard difference is Δ , the mask on the output standard difference is λ , and the standard difference on each K_i is $RC_i[0]_{16} \oplus RC_i[1]_{16}$.

K.2 Ascon

Ascon is a family of authenticated encryption and hashing designed by the Ascon team [22] and has been selected by NIST as the new standard for lightweight cryptography. Both the authenticated encryption and the hashing rely on a 320-bit permutation, whose number of rounds differs with the phases of the encryption scheme. Each round function p of the permutation consists of three steps, $p = p_L \circ p_S \circ p_C$. Ascon's 320-bit state S is arranged as 5 64-bit words x_i , $S = x_0 \| x_1 \| x_2 \| x_3 \| x_4$.

The step p_C adds an 8-bit constant c_r to $x[2]$, where the constants are defined in Table 8. The S-box layer p_S updates the state S with 64 parallel applications of the 5-bit S-box (see Table 9) to $x_0[i] \| x_1[i] \| x_2[i] \| x_3[i] \| x_4[i]$, $0 \leq i < 64$. The step p_L applies a linear function $\sum_i(x_i)$ defined as follows to each word x_i .

$$\begin{aligned} x_0 &\leftarrow \Sigma_0(x_0) = x_0 \oplus (x_0 \ggg 19) \oplus (x_0 \ggg 28) \\ x_1 &\leftarrow \Sigma_1(x_1) = x_1 \oplus (x_1 \ggg 61) \oplus (x_1 \ggg 39) \\ x_2 &\leftarrow \Sigma_2(x_2) = x_2 \oplus (x_2 \ggg 1) \oplus (x_2 \ggg 6) \\ x_3 &\leftarrow \Sigma_3(x_3) = x_3 \oplus (x_3 \ggg 10) \oplus (x_3 \ggg 17) \\ x_4 &\leftarrow \Sigma_4(x_4) = x_4 \oplus (x_4 \ggg 7) \oplus (x_4 \ggg 41) \end{aligned}$$

Table 8: The round constants c_r used in each round i of p^a and p^b .

$p^{12} p^8 p^6$	Constant c_r	$ p^{12} p^8 p^6 $	Constant c_r
0	00000000000000000000f0	6 2 0	00000000000000000000096
1	00000000000000000000e1	7 3 1	00000000000000000000087
2	00000000000000000000d2	8 4 2	00000000000000000000078
3	00000000000000000000c3	9 5 3	00000000000000000000069
4 0	00000000000000000000b4	10 6 4	0000000000000000000005a
5 1	00000000000000000000a5	11 7 5	0000000000000000000004b

K.3 Alzette

Alzette [8] is a 64-bit ARX-based S-box designed by Beierle et al. as the main building block of the Sparkle-suite [9]. The instance of Alzette takes $(x, y) \in$

Table 9: Ascon’s 5-bit S-box $\mathcal{S}$ as a lookup table.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	10	11	12	13	14	15	16	17	18	19	1a	1b	1c	1d	1e	1f
$S(x)$	4	b	1f	14	1a	15	9	2	1b	5	8	12	1d	3	6	1c	1e	13	7	e	0	d	11	18	10	c	1	19	16	a	f	17

$\mathbb{F}_2^{32} \times \mathbb{F}_2^{32}$ as input and then is updated as follows (see Figure 3):

$$\begin{aligned}
 x &\leftarrow x \boxplus_{32} (y \ggg 31), y \leftarrow y \oplus (x \ggg 24), x \leftarrow x \oplus c \\
 x &\leftarrow x \boxplus_{32} (y \ggg 17), y \leftarrow y \oplus (x \ggg 17), x \leftarrow x \oplus c \\
 x &\leftarrow x \boxplus_{32} (y \ggg 0), y \leftarrow y \oplus (x \ggg 31), x \leftarrow x \oplus c \\
 x &\leftarrow x \boxplus_{32} (y \ggg 24), y \leftarrow y \oplus (x \ggg 16), x \leftarrow x \oplus c
 \end{aligned}$$

13-Round SDL distinguisher. For 13-round Alzette with $c = b7e15162$, we first model an SDL trail through 6-round E_U , 1-round E_M and 6-round E_L , where Algorithm 1 is applied on E_M . This gives an SDL trail whose correlation is approximated as 2^{-39} by the model. Then, we extend E_M to 7 rounds, such that the decomposition becomes 3-round E_U , 7-round E_M and 3-round E_L . The final SDL distinguisher consists of a standard differential $(10080400, 08040000) \xrightarrow{2^{-5}} (08000000, 18000000)$ of E_U and a linear approximation $(00000001, 00000000) \xrightarrow{2^{-4}} (01c081c1, c180c081)$ of E_L , with the SDL correlation of E_M being experimentally evaluated as $2^{-12.36}$ using 2^{35} pairs, which gives a trail-based estimate of $2^{-5-8-12.36} = 2^{-25.36}$. Measuring the 10-round $E_L \circ E_M$ directly with 2^{48} pairs gives $2^{-21.31}$ instead of $2^{-8-12.36} = 2^{-20.36}$, whereas the 10-round $E_M \circ E_U$ gives $2^{-17.19}$, in agreement with $2^{-5-12.36} = 2^{-17.36}$ (see Table 10). We therefore estimate the overall 13-round SDL correlation as $2^{-5-21.31} = 2^{-26.31}$.

K.4 SPECK

SPECK [7] is a family of ARX block ciphers designed by the U.S. National Security Agency (NSA). The round function of SPECK is depicted in Figure 4. In this work, we focus on the variants with 48-bit and 64-bit block sizes, where the rotation constants, α and β , are set to 8 and 3, respectively.

K.5 SipHash

SipHash [3] is a family of ARX-based pseudorandom functions optimized for short inputs. Let the round state of SipHash be represented as (a, b, c, d) , where $a, b, c, d \in \mathbb{F}_2^{64}$, then the round function of SipHash (see Figure 5) can be described as

$$\begin{aligned}
 a &= a \boxplus_{64} b, & b &= b \lll 13, & c &= d \boxplus_{64} c, & d &= d \lll 16; \\
 b &= b \oplus a, & d &= d \oplus c, & a &= c, & c &= a \lll 32; \\
 a &= a \boxplus_{64} b, & b &= b \lll 17, & c &= d \boxplus_{64} c, & d &= d \lll 21; \\
 b &= b \oplus a, & d &= d \oplus c, & a &= c, & c &= a \lll 32.
 \end{aligned}$$

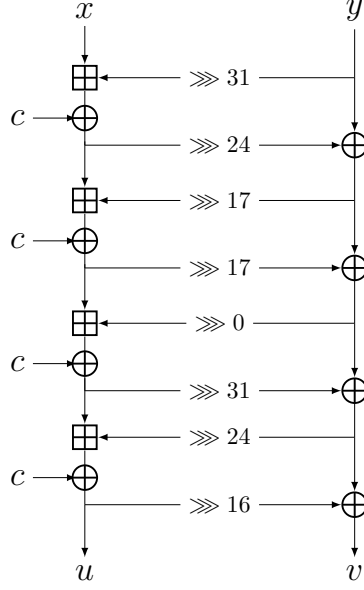


Fig. 3: The round function of Alzette

As in [45], we focus on the finalization process of SipHash, where multiple rounds are applied, and the output branches are combined using XOR operations.

K.6 Chaskey

Chaskey [43] is a lightweight ARX-based cipher designed specifically for use as a MAC algorithm. It has been standardized by ISO and is optimized for resource-constrained devices, making it highly efficient in environments with limited computational resources. The round function of Chaskey operates on four 32-bit words (a, b, c, d) and employ 8 rounds, described as follows (see Figure 6):

$$\begin{aligned}
 a &= a \boxplus_{32} b, & b &= b \lll 8, & c &= d \boxplus_{32} c, & d &= d \lll 5; \\
 b &= b \oplus a, & d &= d \oplus c, & a &= c \lll 16, & c &= a; \\
 a &= a \boxplus_{32} b, & b &= b \lll 13, & c &= d \boxplus_{32} c, & d &= d \lll 7; \\
 b &= b \oplus a, & d &= d \oplus c, & a &= c \lll 16, & c &= a.
 \end{aligned}$$

7-round SDL Distinguisher of Chaskey. For 7-round SDL distinguisher of Chaskey, we model an SDL trail through 3.5-round E_U , 0.5-round E_M and 3-round E_L , where Algorithm 2 is applied on E_M . This gives an SDL trail whose correlation is approximated as 2^{-51} by the model. We then extend E_M to 5 rounds, such that the decomposition becomes 2-round E_U , 5-round E_M and 0-round E_L . The final SDL distinguisher consists of the following standard dif-

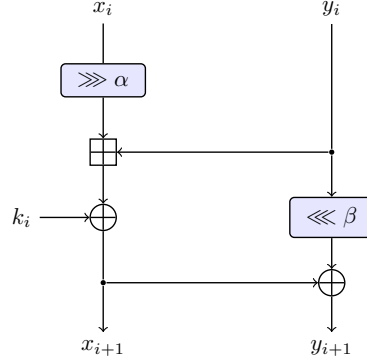


Fig. 4: The round function of SPECK

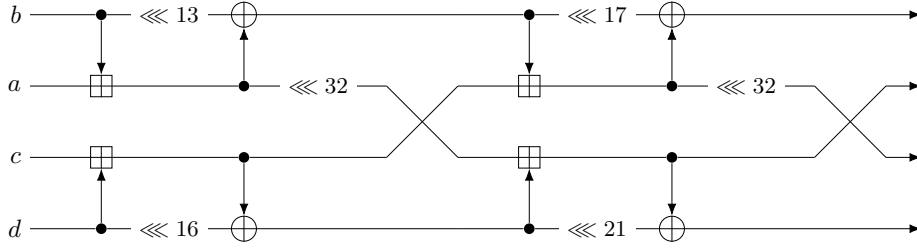


Fig. 5: The round function of SipHash

ferential of E_U :

$$\begin{array}{ccc}
 80048100 & & 80000000 \\
 00848100 & \xrightarrow{2^{-16}} & 00000000 \\
 00080c20 & & 00000000 \\
 00080820 & & 00008000
 \end{array}$$

and the output linear mask of E_L is $(04001098, 0001a300, 0001a301, 08800408)$. The SDL correlation of E_M is experimentally evaluated as $2^{-17.48}$. Thus, the overall SDL correlation of the 7-round distinguisher is estimated as $2^{-16-17.48} = 2^{-33.48}$.

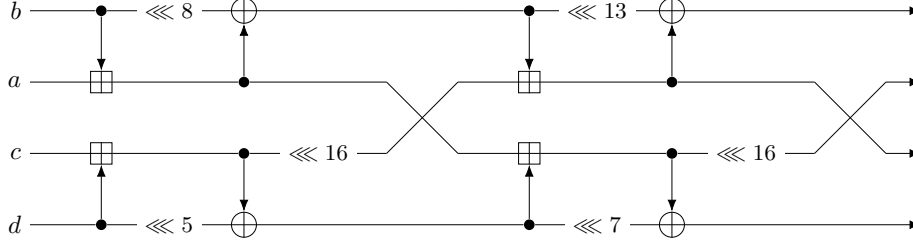


Fig. 6: The round function of Chaskey

L Experimental Verification

All (standard, rotational, internal) differentials of E_U and all middle-part correlations used in this paper, as well as the full distinguishers whenever their correlations are large enough, have been verified experimentally; Table 10 lists the measured values against the theoretical ones together with the sample sizes. For the unkeyed permutations (Xoodoo- p , Ascon- p , Alzette, and the finalization of SipHash and Chaskey), we sample N inputs (IDL) or input pairs (SDL/RDL) uniformly at random and compute the empirical probability or correlation directly. For SPECK, which is keyed, we use the real key schedule, sample K random master keys, and report the mean absolute correlation (resp. the mean probability) over the keys with N pairs per key, written as $K \times N$ in the table. The experiments were run on a laptop GPU (NVIDIA RTX 4080). The measured differential probabilities of E_U match the theoretical values in all cases except for the 8-round Alzette RDL distinguisher, and the measured middle-part correlations match the values computed with the techniques of Section 3.4 to within 0.2 in the exponent. For the composed parts, the largest deviations occur for Alzette and SPECK48; in these cases we base the reported correlations on the measured values, as described in Sections 6.1 and 6.2 and Appendix K.3.

Table 10: Experimental verification of the differential paths and of the (differential-)linear correlations. “diff.” denotes a (standard, rotational, internal) differential probability and “cor.” an SDL/RDL/IDL correlation. Theoretical values of composed parts are the products of the values of their components; “ $-$ ” means that the middle part is evaluated experimentally only. For SPECK, $K \times N$ means N pairs for each of K random keys.

Target (distinguisher)	Part (quantity)	#Rounds	Theoretical	Measured	Samples
Ascon- p (6-round RDL)	E_U (rot. diff.)	2	2^{-8}	$2^{-8.00}$	2^{32}
	E_M (RDL cor.)	3	2^{-11}	$2^{-11.12}$	2^{34}
	$E_L \circ E_M$ (RDL cor.)	4	2^{-13}	$2^{-13.14}$	2^{34}
	$E_M \circ E_U$ (RDL cor.)	5	2^{-19}	$2^{-19.21}$	2^{44}
	E (RDL cor.)	6	2^{-21}	$2^{-20.77}$	2^{47}
Xoodoo- p (6-round RDL)	E_U (rot. diff.)	2	2^{-24}	$2^{-24.00}$	2^{38}
	E_M (RDL cor.)	3	$2^{-16.10}$	$2^{-15.95}$	2^{37}
	$E_L \circ E_M$ (RDL cor.)	4	$2^{-20.09}$	$2^{-20.21}$	2^{46}
Xoodoo- p (6-round IDL)	E_U (int. diff.)	2	2^{-32}	$2^{-32.00}$	2^{42}
	E_M (IDL cor.)	3	2^{-9}	$2^{-9.00}$	2^{31}
	$E_L \circ E_M$ (IDL cor.)	4	2^{-13}	$2^{-12.99}$	2^{34}
Alzette (8-round RDL)	E_U (rot. diff.)	2	2^{-15}	$2^{-16.06}$	2^{36}
	E_M (RDL cor.)	1	$2^{-0.91}^\dagger$	$2^{-0.91}$	2^{31}
	$E_M \circ E_U$ (RDL cor.)	3	$2^{-16.97}^\ddagger$	$2^{-10.30}$	2^{40}
	$E_L \circ E_M$ (RDL cor.)	6	$2^{-14.91}$	$2^{-12.39}$	2^{38}
Alzette (13-round SDL)	E_U (std. diff.)	3	2^{-5}	$2^{-5.00}$	2^{28}
	E_M (SDL cor.)	7	$-$	$2^{-12.36}$	2^{35}
	$E_M \circ E_U$ (SDL cor.)	10	$2^{-17.36}$	$2^{-17.19}$	2^{42}
	$E_L \circ E_M$ (SDL cor.)	10	$2^{-20.36}$	$2^{-21.31}$	2^{48}
SPECK48 (12-round SDL)	E_U (std. diff.)	3	2^{-6}	$2^{-5.99}$	1024×2^{32}
	E_M (SDL cor.)	6	$-$	$2^{-9.90}$	1024×2^{30}
	$E_L \circ E_M$ (SDL cor.)	9	$2^{-15.90}$	$2^{-16.44}$	1024×2^{38}
SPECK64 (14-round SDL)	E_U (std. diff.)	3	2^{-6}	$2^{-6.00}$	1024×2^{28}
	E_M (SDL cor.)	7	$-$	$2^{-13.33}$	1024×2^{30}
	$E_M \circ E_U$ (SDL cor.)	10	$2^{-19.33}$	$2^{-19.15}$	256×2^{40}
SipHash (5-round SDL)	E_U (std. diff.)	1	2^{-15}	$2^{-15.00}$	2^{32}
	E_M (SDL cor.)	4	$-$	$2^{-5.15}$	2^{30}
	E (SDL cor.)	5	$2^{-20.15}$	$2^{-20.17}$	2^{46}
Chaskey (7-round SDL)	E_U (std. diff.)	2	2^{-16}	$2^{-16.00}$	2^{34}
	E_M (SDL cor.)	5	$-$	$2^{-17.48}$	2^{40}

† Exact value computed with Proposition 8.

‡ Product of the measured probability of E_U and the correlation of E_M .

M Niu et al.'s Work [45] on ARX Ciphers

From the propagation of standard differences, Niu et al. define the 4×4 matrix $M_{\alpha,\beta,\lambda}$ for $(\alpha, \beta, \lambda) \in \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2$ as follows:

$$\begin{aligned} M_{0,0,0} &= \begin{bmatrix} 3 & 1 & 1 & 1 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 1 & 1 & 1 & 3 \end{bmatrix} & M_{0,0,1} &= \begin{bmatrix} 3 & -1 & -1 & 1 \\ 0 & -2 & 0 & 0 \\ 0 & 0 & -2 & 0 \\ 1 & -1 & -1 & 3 \end{bmatrix} & M_{0,1,0} &= \begin{bmatrix} 2 & 1 & 1 & 0 \\ 1 & 2 & 0 & 1 \\ 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 2 \end{bmatrix} & M_{0,1,1} &= \begin{bmatrix} -2 & 1 & 1 & 0 \\ -1 & 2 & 0 & -1 \\ -1 & 0 & 2 & -1 \\ 0 & 1 & 1 & -2 \end{bmatrix} \\ M_{1,0,0} &= \begin{bmatrix} 2 & 1 & 1 & 0 \\ 1 & 2 & 0 & 1 \\ 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 2 \end{bmatrix} & M_{1,0,1} &= \begin{bmatrix} -2 & 1 & 1 & 0 \\ -1 & 2 & 0 & -1 \\ -1 & 0 & 2 & -1 \\ 0 & 1 & 1 & -2 \end{bmatrix} & M_{1,1,0} &= \begin{bmatrix} 2 & 0 & 0 & 0 \\ 1 & 3 & 1 & 1 \\ 1 & 1 & 3 & 1 \\ 0 & 0 & 0 & 2 \end{bmatrix} & M_{1,1,1} &= \begin{bmatrix} 2 & 0 & 0 & 0 \\ 1 & -3 & -1 & 1 \\ 1 & -1 & -3 & 1 \\ 0 & 0 & 0 & 2 \end{bmatrix}. \end{aligned}$$

Niu et al. [45] established the following two fundamental theorems for computing the SDL and RDL correlations of modular addition, respectively.

Theorem 1 (SDL correlation of modular addition [45]). *Given the input standard difference $\alpha, \beta \in \mathbb{F}_2^n$ and the output linear mask $\lambda \in \mathbb{F}_2^n$ of the modular addition $\boxplus_n^{(b)}$, the SDL correlation can be computed as*

$$\mathcal{R}_{\boxplus_n^{(b)}}^{\text{SDL}}((\alpha, \beta), \lambda) = \frac{1}{2^{2n}} [1 \ 1 \ 1 \ 1] M_{\alpha[n-1], \beta[n-1], \lambda[n-1]} \cdots M_{\alpha[0], \beta[0], \lambda[0]} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

Theorem 2 (RDL correlation of modular addition [45]). *Given the input rotational difference $\alpha, \beta \in \mathbb{F}_2^n$ and the output linear mask $\lambda \in \mathbb{F}_2^n$ of the modular addition $\boxplus_n^{(b)}$ with rotational offset t , the RDL correlation can be computed as*

$$\mathcal{R}_{\boxplus_n^{(b)}}^{\text{RDL}}((\alpha, \beta), \lambda) = \frac{1}{2^{2n}} [1 \ 0 \ 1 \ 0] C_{\alpha, \beta, \lambda} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \frac{1}{2^{2n}} [0 \ 1 \ 0 \ 1] C_{\alpha, \beta, \lambda} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix},$$

where

$$C_{\alpha, \beta, \lambda} = \prod_{i=0}^{t-1} M_{\alpha[i], \beta[i], \lambda[i]} \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \prod_{j=t}^{n-1} M_{\alpha[j], \beta[j], \lambda[j]}.$$

Example 5 (SDL correlation of modular addition [45]). Consider the 32-bit modulo addition. Let $\alpha, \beta \in \mathbb{F}_2^{32}$ be the input standard difference with

$$\begin{cases} \alpha = (1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1) \\ \beta = (0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1) \end{cases}.$$

Then, the SDL correlations $\mathcal{R}_{\boxplus_{32}^{(b)}}^{\text{SDL}}((\alpha, \beta), e_i \oplus e_{i+1})$ for $0 \leq i < 31$ can be computed with Theorem 1, and the results are listed in Table 11, where e_i is a unit vector in which all entries are zero except the entry at index i .

Table 11: The example SDL correlations of $\boxplus_{32}^{(b)}$ [45].

i	0	1	2	3	4	5	6	7	8	9	10	11
$\mathcal{R}_{\boxplus_{32}^{(b)}}^{\text{SDL}}((\alpha, \beta), e_i \oplus e_{i+1})$	0	$\frac{1}{2}$	$\frac{3}{4}$	$\frac{7}{8}$	$\frac{15}{16}$	$\frac{31}{32}$	$\frac{1}{64}$	$-\frac{65}{128}$	0	$\frac{1}{2}$	$-\frac{3}{4}$	0
i	12	13	14	15	16	17	18	19	20	21	22	23
$\mathcal{R}_{\boxplus_{32}^{(b)}}^{\text{SDL}}((\alpha, \beta), e_i \oplus e_{i+1})$	0	$-\frac{1}{2}$	0	0	0	0	$\frac{1}{2}$	$\frac{1}{4}$	$-\frac{5}{8}$	0	0	0
i	24	25	26	27	28	29	30					
$\mathcal{R}_{\boxplus_{32}^{(b)}}^{\text{SDL}}((\alpha, \beta), e_i \oplus e_{i+1})$	$-\frac{1}{2}$	0	0	0	0	0	$\frac{1}{2}$					

Example 6 (RDL correlation of modular addition [45]). Consider the 32-bit modulo addition. Let $\alpha, \beta \in \mathbb{F}_2^{32}$ be the input rotational difference with

$$\begin{cases} \alpha = (0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1) \\ \beta = (1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1) \end{cases}.$$

Then, the RDL correlations $\mathcal{R}_{\boxplus_{32}^{(b)}}^{\text{RDL}}((\alpha, \beta), e_i \oplus e_{i+1})$ for $0 \leq i < 31$ with rotational offset $t = 30$ can be computed with Theorem 2, and the results are listed in Table 12.

Table 12: The example RDL correlations of $\boxplus_{32}^{(b)}$ [45].

i	0	1	2	3	4	5	6	7	8	9	10	11
$\mathcal{R}_{\boxplus_{32}^{(b)}}^{\text{RDL}}((\alpha, \beta), e_i \oplus e_{i+1})$	0	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{5}{8}$	$\frac{3}{16}$	$-\frac{19}{32}$	0	$\frac{1}{2}$	$\frac{3}{4}$	$-\frac{7}{8}$	0	$-\frac{1}{2}$
i	12	13	14	15	16	17	18	19	20	21	22	23
$\mathcal{R}_{\boxplus_{32}^{(b)}}^{\text{RDL}}((\alpha, \beta), e_i \oplus e_{i+1})$	0	$\frac{1}{2}$	$\frac{1}{4}$	$-\frac{3}{8}$	0	$-\frac{1}{2}$	0	$\frac{1}{2}$	$-\frac{1}{4}$	0	0	$-\frac{1}{2}$
i	24	25	26	27	28	29	30					
$\mathcal{R}_{\boxplus_{32}^{(b)}}^{\text{RDL}}((\alpha, \beta), e_i \oplus e_{i+1})$	0	$\frac{1}{2}$	$\frac{3}{4}$	$-\frac{7}{8}$	0	$\frac{1}{2^{32}}$	$\frac{1073741825}{2^{31}}$					

N Efficient Handling of Exponential Constraints in Modern Solvers

In modern CP solvers (e.g., OR-Tools invoked by Minizinc), the constraints in Line 5 of Algorithm 1 can be reformulated to avoid explicit exponentiation at runtime. The original constraints are as follows:

$$2^{r_{\max} - r - 1} < |a \cdot u_0[0] + u_0[1]| \leq 2^{r_{\max} - r}.$$

We can reformulate the constraints above using the table constraint as follows:

1. Define a table

$$T = \{(i, L[i], U[i]) : i \in \{0, \dots, r_{\max} - 1\}, L[i] = 2^{r_{\max} - i - 1}, U[i] = 2^{r_{\max} - i}\}$$

2. Introduce auxiliary variables **lower** and **upper**
3. Add constraints $(\mathbf{r}, \mathbf{lower}, \mathbf{upper}) \in T$, and

$$\mathbf{lower} < |a \cdot \mathbf{u}_0[0] + \mathbf{u}_0[1]| \leq \mathbf{upper}$$

Most modern solvers can implement this efficiently as it requires only table lookups and element constraints. Both of which are highly optimized primitives.